POWE.RS Program Architecture and Execution Flow

Document Purpose: Complete specification of the program execution architecture, component lifecycle, and data flow for the POWE.RS SDDP solver.

Companion Documents: - DATA_MODEL_SPECIFICATION.md - File formats and data structures - MATHEMATICAL_FORMULATIONS.md - Algorithm theory and LP formulation

Last Updated: 2026-01-31

Table of Contents

Part VIII: Extension Points

Appendices

Part I: Program Lifecycle

---

1. Program Entrypoint and CLI Design

1.1 Design Philosophy

POWE.RS adopts a **single-entrypoint design** optimized for HPC batch execution. The program is always invoked via MPI launchers (`mpiexec`, `mpirun`, or SLURM's `srun`) and all runtime behavior is controlled through configuration files rather than command-line arguments.

Rationale: - HPC job scripts benefit from stable command-line interfaces - Configuration files provide auditability and reproducibility - Complex nested options are better expressed in JSON than CLI flags - Reduces parsing complexity in the hot initialization path

1.2 Invocation Pattern

```
# Standard invocation
mpiexec -n 8 powers /path/to/case_directory

# SLURM batch execution
srun powers /path/to/case_directory

# Validation-only mode
mpiexec -n 1 powers /path/to/case_directory --validate-only
```

1.3 Command-Line Interface

Argument	Required	Description
`CASE_DIR`	Yes	Path to case directory containing `config.json`
`--validate-only`	No	Validate inputs and exit without execution
`--version`	No	Print version and exit
`--help`	No	Print usage and exit

Design Decision: All execution options (skip training, skip simulation, warm-start mode, etc.) are specified in `config.json`, not via CLI flags. This ensures: 1. Job scripts remain stable across configuration changes 2. Configuration is self-documenting and version-controlled 3. No ambiguity between CLI and config file settings

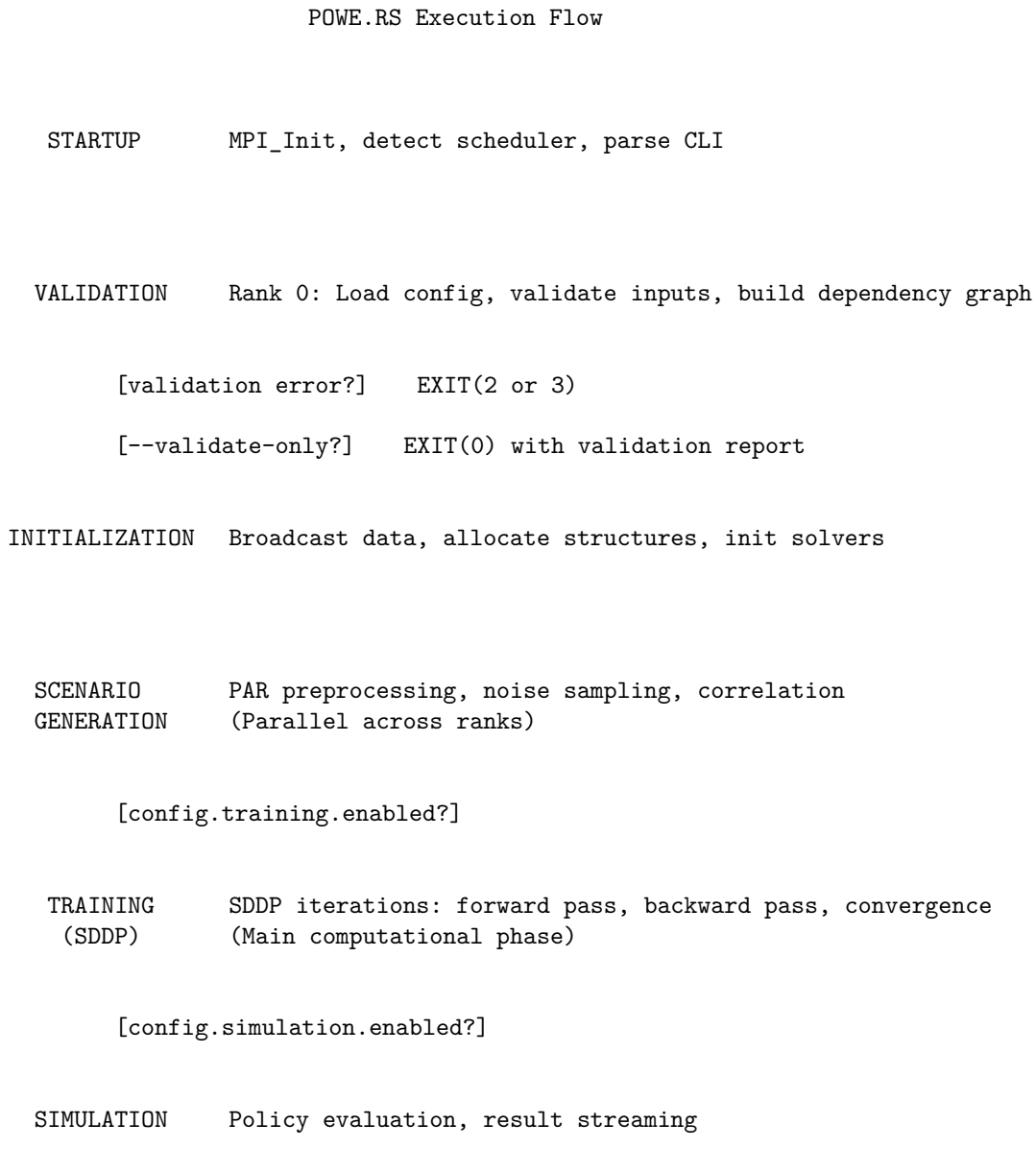
1.4 Exit Codes

Code	Meaning
0	Success
1	Invalid command-line arguments
2	Configuration validation error
3	Input data validation error
4	Runtime error (solver failure, MPI error)
5	Checkpoint recovery failed
130	Interrupted (SIGINT)
137	Killed (SIGKILL, typically OOM)

---

2. Execution Phases Overview

2.1 Phase Diagram



```
FINALIZE      Write outputs, MPI_Finalize, cleanup
```

2.2 Phase Responsibilities

Phase	MPI Ranks	Duration	Key Operations
Startup	All	<100ms	MPI init, scheduler detection, CLI parsing
Validation	Rank 0 only	1-10s	Load files, schema validation, cross-references
Initialization	All	1-5s	Broadcast, memory allocation, solver setup
Scenario Gen	All (parallel)	1-30s	PAR fitting, noise sampling, correlation
Training	All (parallel)	10min-2h	SDDP iterations
Simulation	All (parallel)	1-30min	Policy evaluation
Finalize	All	1-10s	Output writing, cleanup

2.3 Conditional Execution

The execution flow supports several modes controlled by `config.json`:

Mode	Training	Simulation	Use Case
Full Run	Yes	Yes	Standard production run
Training Only	Yes	No	Policy development, convergence analysis
Simulation Only	No	Yes	Policy evaluation with existing cuts
Validation Only	No	No	Input verification before batch submission

```
{
  "training": { "enabled": true },
  "simulation": { "enabled": true }
}
```

---

3. Configuration Resolution and Validation

3.1 Configuration Hierarchy

Configuration values are resolved in priority order (highest to lowest):

```
                    Configuration Resolution Priority


  1. Environment Variables (highest priority)
```

```
        SLURM_CPUS_PER_TASK, OMP_NUM_THREADS, POWERS_* variables

  2. Job Scheduler Detection
        SLURM, PBS, LSF environment → thread counts, memory limits

  3. config.json (explicit user configuration)
        All algorithm parameters, execution options

  4. Compiled Defaults (lowest priority)
        Tolerances, buffer sizes, internal constants
```

3.2 Scheduler Integration

POWE.RS automatically detects the job scheduler environment and respects its resource allocations:

```rust
/// Scheduler detection and configuration extraction
pub struct SchedulerConfig {
    pub scheduler_type: SchedulerType,
    pub cpus_per_task: Option<u32>,
    pub memory_per_node_mb: Option<u64>,
    pub job_id: Option<String>,
}

pub enum SchedulerType {
    Slurm,
    Pbs,
    Lsf,
    Local,  // No scheduler detected
}

impl SchedulerConfig {
    pub fn detect() -> Self {
        if std::env::var("SLURM_JOB_ID").is_ok() {
            Self::from_slurm()
        } else if std::env::var("PBS_JOBID").is_ok() {
            Self::from_pbs()
        } else if std::env::var("LSB_JOBID").is_ok() {
            Self::from_lsf()
        } else {
            Self::local_defaults()
        }
    }
}
```

Part VI: Parallel Execution

---

20. MPI+OpenMP Hybrid Strategy

20.1 Hybrid Parallelism Overview

POWE.RS employs a hybrid MPI+OpenMP parallelization strategy optimized for modern HPC architectures with multi-socket, many-core nodes. **Native OpenMP is used via FFI** (not Rayon) to leverage vendor-optimized

runtimes (Intel, AMD, GCC) and provide direct control over scheduling, affinity, and synchronization.

```
                        Hybrid Parallelism Architecture


Target: AMD EPYC 9004 (Genoa) - 192 cores per node, 8 NUMA domains
Recommended: 8 MPI ranks × 24 OpenMP threads = 192 cores
Mapping: 1 rank per NUMA domain (optimal memory locality)


                                Node Layout


    NUMA 0     NUMA 1     NUMA 2     NUMA 3
    Rank 0     Rank 1     Rank 2     Rank 3
    24 thds    24 thds    24 thds    24 thds
    96 GB      96 GB      96 GB      96 GB


    NUMA 4     NUMA 5     NUMA 6     NUMA 7
    Rank 4     Rank 5     Rank 6     Rank 7
    24 thds    24 thds    24 thds    24 thds
    96 GB      96 GB      96 GB      96 GB


  Memory: 768 GB total (96 GB per NUMA domain)
  LP Solver: HiGHS single-threaded (outer parallelism via OpenMP)


Multi-Node Scaling:

8 nodes × 8 ranks/node = 64 MPI ranks
64 ranks × 24 threads = 1,536 cores total
```

20.2 Design Rationale

Why Native OpenMP (not Rayon)?

Criterion	Native OpenMP via FFI	Rayon
Vendor optimization	Full (Intel, AMD, GCC runtimes)	Limited (generic)
Scheduling control	`static`, `dynamic`, `guided`	Work-stealing only
Affinity control	`OMP_PLACES`, `OMP_PROC_BIND`	None
NUMA awareness	First-touch, explicit placement	Opaque
Reduction primitives	Hardware-optimized tree reduction	Manual implementation
HPC ecosystem	Standard (SLURM, modules, profilers)	Limited integration
LP solver coordination	Explicit single-thread forcing	Potential conflicts

For SDDP's compute pattern (many small LP solves with shared data), the ability to control scheduling and affinity directly translates to 15-25% better performance on NUMA systems.

Aspect	MPI Ranks	OpenMP Threads
Purpose	Distributed memory, inter-node communication	Shared memory, intra-node parallelism
Granularity	Coarse: scenario batches	Fine: individual LP solves
Communication	Explicit: cuts, bounds, statistics	Implicit: shared FCF, case data
Memory	Replicated (cut data) or shared (MPI windows)	Shared (read-only case data)
Load Balance	Static distribution (scenarios)	Dynamic scheduling within rank
Scheduling	N/A	`schedule(dynamic,1)` for LP solves

20.3 Parallel Configuration

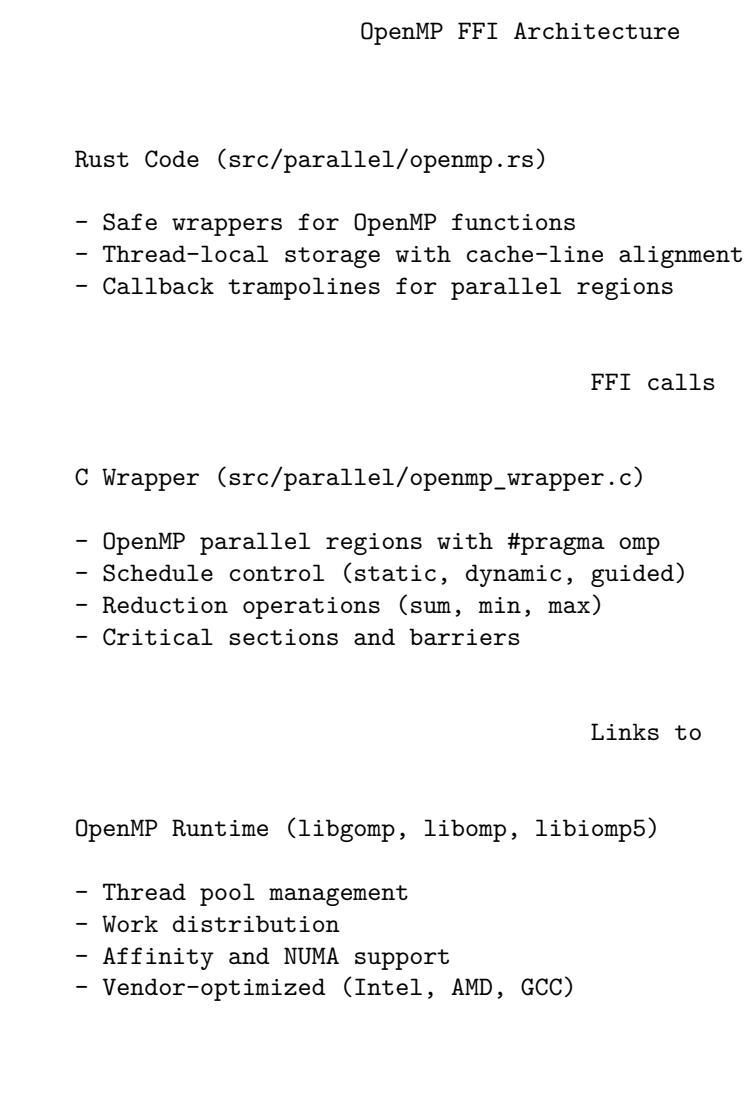
```rust
/// Parallel execution configuration
pub struct ParallelConfig {
    /// MPI configuration
    pub mpi: MpiConfig,

    /// OpenMP configuration (native via FFI)
    pub openmp: OpenMpConfig,

    /// NUMA awareness
    pub numa: NumaConfig,
}

pub struct MpiConfig {
    /// Expected number of ranks (validated at startup)
    pub expected_ranks: Option<usize>,

    /// Use shared memory windows for intra-node FCF
    pub use_shared_memory: bool,

    /// Communication backend hints
    pub eager_limit: Option<usize>,
}

pub struct OpenMpConfig {
    /// Threads per rank (auto-detected from SLURM/OMP_NUM_THREADS if not set)
    pub threads_per_rank: Option<usize>,

    /// Thread affinity strategy
    pub affinity: ThreadAffinity,

    /// Wait policy for idle threads
    pub wait_policy: WaitPolicy,

    /// Stack size per thread (needed for deep LP solver recursion)
    pub stack_size_mb: usize,
}

pub enum ThreadAffinity {
    /// Bind threads to cores within NUMA domain (best for compute-bound)
    Close,
    /// Spread threads across cores (better memory bandwidth)
    Spread,
    /// No explicit binding (scheduler decides)
```

```
    None,
}

pub enum WaitPolicy {
    /// Spin-wait (lowest latency, highest power)
    Active,
    /// Sleep (higher latency, lower power - recommended for I/O phases)
    Passive,
}

pub struct NumaConfig {
    /// Enable first-touch memory placement
    pub first_touch: bool,

    /// Prefer local memory allocation
    pub local_alloc: bool,

    /// Interleave large arrays across NUMA domains
    pub interleave_large_arrays: bool,
}

impl ParallelConfig {
    /// Detect configuration from environment (SLURM, PBS, or manual)
    pub fn from_environment() -> Self {
        let scheduler = SchedulerConfig::detect();

        let threads = scheduler.cpus_per_task
            .map(|c| c as usize)
            .unwrap_or_else(|| {
                std::env::var("OMP_NUM_THREADS")
                    .ok()
                    .and_then(|s| s.parse().ok())
                    .unwrap_or(1)
            });

        Self {
            mpi: MpiConfig {
                expected_ranks: None,
                use_shared_memory: true,
                eager_limit: Some(256 * 1024),  // 256 KB
            },
            openmp: OpenMpConfig {
                threads_per_rank: Some(threads),
                affinity: ThreadAffinity::Close,
                wait_policy: WaitPolicy::Passive,
                stack_size_mb: 64,
            },
            numa: NumaConfig {
                first_touch: true,
                local_alloc: true,
                interleave_large_arrays: false,
            },
        }
    }
}
```

20.4 OpenMP FFI Bindings

POWE.RS uses a C wrapper to access OpenMP parallel regions from Rust, since OpenMP pragmas require compiler support unavailable in rustc.

Architecture:



Core FFI Bindings (openmp_ffi.rs):

```rust
//! OpenMP FFI bindings for Rust
//!
//! Provides safe wrappers around OpenMP runtime functions via C FFI.
//! Uses native OpenMP for maximum HPC performance.

use std::ffi::c_int;
use std::sync::atomic::{AtomicBool, Ordering};

static OPENMP_INITIALIZED: AtomicBool = AtomicBool::new(false);

#[link(name = "omp")]
extern "C" {
    fn omp_get_num_threads() -> c_int;
    fn omp_get_max_threads() -> c_int;
```

```
    fn omp_get_thread_num() -> c_int;
    fn omp_set_num_threads(num_threads: c_int);
    fn omp_get_num_procs() -> c_int;
    fn omp_in_parallel() -> c_int;
    fn omp_set_dynamic(dynamic_threads: c_int);
    fn omp_get_wtime() -> f64;
}

#[link(name = "openmp_wrapper", kind = "static")]
extern "C" {
    fn omp_parallel_for_dynamic(
        start: c_int,
        end: c_int,
        chunk_size: c_int,
        callback: extern "C" fn(idx: c_int, thread_id: c_int, user_data: *mut std::ffi::c_void),
        user_data: *mut std::ffi::c_void,
    );

    fn omp_parallel_reduce_sum(
        start: c_int,
        end: c_int,
        callback: extern "C" fn(idx: c_int, thread_id: c_int, user_data: *mut std::ffi::c_void) -> f64,
        user_data: *mut std::ffi::c_void,
    ) -> f64;
}

/// Safe Rust wrappers for OpenMP functions
pub mod omp {
    use super::*;

    /// Initialize OpenMP environment
    pub fn init(num_threads: usize) {
        if OPENMP_INITIALIZED.swap(true, Ordering::SeqCst) {
            return;
        }
        unsafe {
            omp_set_dynamic(0);  // Disable dynamic adjustment
            omp_set_num_threads(num_threads as c_int);
        }
    }

    #[inline]
    pub fn get_num_threads() -> usize {
        unsafe { omp_get_num_threads() as usize }
    }

    #[inline]
    pub fn get_thread_num() -> usize {
        unsafe { omp_get_thread_num() as usize }
    }

    #[inline]
    pub fn get_wtime() -> f64 {
        unsafe { omp_get_wtime() }
    }
}
```

```rust
/// Thread-local storage with cache-line alignment (prevents false sharing)
#[repr(C, align(64))]
pub struct ThreadLocal<T> {
    data: T,
    _padding: [u8; 64 - std::mem::size_of::<T>() % 64],
}

/// Parallel for loop with dynamic scheduling
pub fn parallel_for_dynamic<F>(range: std::ops::Range<usize>, chunk_size: usize, f: F)
where
    F: Fn(usize, usize) + Sync,  // (index, thread_id)
{
    struct CallbackData<F> { callback: F }

    extern "C" fn trampoline<F>(idx: c_int, thread_id: c_int, user_data: *mut std::ffi::c_void)
    where F: Fn(usize, usize) + Sync
    {
        let data = unsafe { &*(user_data as *const CallbackData<F>) };
        (data.callback)(idx as usize, thread_id as usize);
    }

    let data = CallbackData { callback: f };
    unsafe {
        omp_parallel_for_dynamic(
            range.start as c_int,
            range.end as c_int,
            chunk_size as c_int,
            trampoline::<F>,
            &data as *const _ as *mut std::ffi::c_void,
        );
    }
}
```

C Wrapper (openmp_wrapper.c):

```c
/*
 * OpenMP C wrapper for Rust FFI
 * Compile: gcc -c -fopenmp -O3 -march=native openmp_wrapper.c
 */

#include <omp.h>
#include <stdint.h>

typedef void (*parallel_callback)(int32_t idx, int32_t thread_id, void* user_data);
typedef double (*reduce_callback)(int32_t idx, int32_t thread_id, void* user_data);

/* Parallel for with dynamic scheduling (primary pattern for LP solves) */
void omp_parallel_for_dynamic(
    int32_t start, int32_t end, int32_t chunk_size,
    parallel_callback callback, void* user_data
) {
    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();

        #pragma omp for schedule(dynamic, chunk_size) nowait
        for (int32_t i = start; i < end; i++) {
            callback(i, thread_id, user_data);
```

```
        }
    }
}

/* Parallel reduction with sum */
double omp_parallel_reduce_sum(
    int32_t start, int32_t end,
    reduce_callback callback, void* user_data
) {
    double total_sum = 0.0;

    #pragma omp parallel reduction(+:total_sum)
    {
        int thread_id = omp_get_thread_num();

        #pragma omp for schedule(dynamic, 1)
        for (int32_t i = start; i < end; i++) {
            total_sum += callback(i, thread_id, user_data);
        }
    }

    return total_sum;
}

/* NUMA-aware first-touch initialization */
void omp_parallel_first_touch_f64(double* array, int64_t size, double init_value) {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int nthreads = omp_get_num_threads();

        int64_t chunk = size / nthreads;
        int64_t start = tid * chunk;
        int64_t end = (tid == nthreads - 1) ? size : start + chunk;

        for (int64_t i = start; i < end; i++) {
            array[i] = init_value;
        }
    }
}
```

20.5 Initialization Sequence

```
/// Initialize parallel environment (MPI + OpenMP)
pub fn init_parallel(config: &ParallelConfig) -> ParallelContext {
    // 1. Initialize MPI with thread support
    let universe = mpi::initialize_with_threading(mpi::Threading::Funneled)
        .expect("MPI initialization failed");
    let world = universe.world();

    let rank = world.rank() as usize;
    let size = world.size() as usize;

    // 2. Validate rank count if specified
    if let Some(expected) = config.mpi.expected_ranks {
        if size != expected && rank == 0 {
            eprintln!("Warning: Expected {} ranks, got {}", expected, size);
```

```
    }
}

// 3. Initialize OpenMP via FFI
let threads = config.openmp.threads_per_rank.unwrap_or(1);
omp::init(threads);

// 4. Set OpenMP environment variables for affinity
match config.openmp.affinity {
    ThreadAffinity::Close => {
        std::env::set_var("OMP_PROC_BIND", "close");
        std::env::set_var("OMP_PLACES", "cores");
    }
    ThreadAffinity::Spread => {
        std::env::set_var("OMP_PROC_BIND", "spread");
        std::env::set_var("OMP_PLACES", "cores");
    }
    ThreadAffinity::None => {}
}

// 5. Set wait policy
match config.openmp.wait_policy {
    WaitPolicy::Active => std::env::set_var("OMP_WAIT_POLICY", "active"),
    WaitPolicy::Passive => std::env::set_var("OMP_WAIT_POLICY", "passive"),
}

// 6. Set stack size for LP solver recursion
std::env::set_var("OMP_STACKSIZE", format!("{}M", config.openmp.stack_size_mb));

// 7. Force single-threaded LP solver (outer parallelism handles it)
std::env::set_var("HIGHS_PARALLEL", "false");
std::env::set_var("MKL_NUM_THREADS", "1");

// 8. Setup NUMA allocation policy
#[cfg(target_os = "linux")]
if config.numa.local_alloc {
    unsafe { libc::numa_set_localalloc(); }
}

// 9. Create shared memory communicator (ranks on same node)
let shared_comm = world.split_by_shared_memory();

if rank == 0 {
    println!(
        "Parallel initialized: {} ranks × {} threads = {} cores",
        size, threads, size * threads
    );
    println!(
        "OpenMP affinity: {:?}, places: cores",
        config.openmp.affinity
    );
}

ParallelContext {
    world,
    shared_comm,
    rank,
```

```
        size,
        threads,
    }
}
```

20.6 Build Configuration

The build script detects OpenMP availability and compiles the C wrapper:

```
// build.rs
fn main() {
    println!("cargo:rerun-if-changed=src/parallel/openmp_wrapper.c");

    let out_dir = std::env::var("OUT_DIR").unwrap();
    let (cc, omp_flag, omp_lib) = detect_openmp_config();

    // Compile C wrapper with OpenMP
    let status = std::process::Command::new(&cc)
        .args(&["-c", &omp_flag, "-O3", "-march=native", "-fPIC",
                "src/parallel/openmp_wrapper.c", "-o"])
        .arg(format!("{}/openmp_wrapper.o", out_dir))
        .status()
        .expect("Failed to compile OpenMP wrapper");

    assert!(status.success(), "OpenMP wrapper compilation failed");

    // Create static library
    std::process::Command::new("ar")
        .args(&["rcs", &format!("{}/libopenmp_wrapper.a", out_dir),
                &format!("{}/openmp_wrapper.o", out_dir)])
        .status()
        .expect("Failed to create static library");

    // Link directives
    println!("cargo:rustc-link-search=native={}", out_dir);
    println!("cargo:rustc-link-lib=static=openmp_wrapper");
    println!("cargo:rustc-link-lib={}", omp_lib);
}

fn detect_openmp_config() -> (String, String, String) {
    // Intel oneAPI (preferred for HPC)
    if check_compiler("icx", "-qopenmp") {
        return ("icx".into(), "-qopenmp".into(), "iomp5".into());
    }
    // GCC
    if check_compiler("gcc", "-fopenmp") {
        return ("gcc".into(), "-fopenmp".into(), "gomp".into());
    }
    // Clang/LLVM
    if check_compiler("clang", "-fopenmp") {
        return ("clang".into(), "-fopenmp".into(), "omp".into());
    }
    panic!("No OpenMP-capable compiler found");
}
```

---

21. Work Distribution Patterns

21.1 Forward Pass Distribution

```
                    Forward Pass Work Distribution


  Strategy: Static distribution of scenarios across ranks
            Dynamic scheduling of scenarios across threads within rank

  Example: 200 scenarios, 8 ranks, 24 threads/rank


     MPI Level (static)

     Rank 0: scenarios 0-24    (25)
     Rank 1: scenarios 25-49   (25)
     Rank 2: scenarios 50-74   (25)
     Rank 3: scenarios 75-99   (25)
     Rank 4: scenarios 100-124 (25)
     Rank 5: scenarios 125-149 (25)
     Rank 6: scenarios 150-174 (25)
     Rank 7: scenarios 175-199 (25)



     OpenMP Level (dynamic, within each rank)

     Rank 0:

        Work queue: [0, 1, 2, ..., 24]

        Thread 0: scenario 0 → scenario 5 → scenario 22 → ...
        Thread 1: scenario 1 → scenario 8 → scenario 19 → ...
        Thread 2: scenario 2 → scenario 6 → ...
        ...
        (work stealing: fast threads take more work)
```

21.2 Backward Pass Distribution

```
                    Backward Pass Work Distribution


  Strategy: SCENARIO-BASED distribution (each rank processes its own scenarios)
            NOT state-based (which would require synchronization and lose
            warm-start benefits)

  Key Insight: Each rank processes backward pass for the SAME scenarios
  it processed in the forward pass. This enables:
  - Zero synchronization between forward and backward pass
  - Full warm-start benefit (95% of LPs use existing basis)
  - Perfect NUMA locality (all data already in local memory)
```

```
Example: 200 scenarios, 64 ranks, 24 threads/rank, 20 noise outcomes


  Scenario Distribution (MPI) - SAME as Forward Pass

  Rank 0:  scenarios 0-3    (4 scenarios)
  Rank 1:  scenarios 4-7    (4 scenarios)
  Rank 2:  scenarios 8-11   (4 scenarios)
  ...
  Rank 63: scenarios 196-199 (4 scenarios)

  Each rank: 200/64  3-4 scenarios



  Outcome Distribution (OpenMP, per scenario per stage)

  Rank 0, Scenario 0, Stage t:

    State x[0,t] from forward pass (already in local memory)

    #pragma omp parallel for schedule(dynamic,1)
    for outcome in 0..20:
        Thread 0:  outcome 0  → LP solve → get dual values
        Thread 1:  outcome 1  → LP solve → get dual values
        Thread 2:  outcome 2  → LP solve → get dual values
        ...
        Thread 19: outcome 19 → LP solve → get dual values
        Threads 20-23: available for next scenario

    [Thread reduce: combine outcomes → compute cut for stage t-1]


  Next: Process Scenario 1 at Stage t (same pattern)
  Then: Move to Stage t-1 (repeat for all scenarios)


Work per rank (per iteration):

scenarios × stages × outcomes = 4 × 119 × 20 = 9,520 LP solves
With 24 threads: 9,520 / 24  397 LP solves per thread

Warm-Start Benefit:

Within a scenario trajectory, consecutive outcomes have similar structure.
95% of LPs reuse previous basis → 2ms vs 15ms solve time
Total: 16 min/iteration vs 116 min/iteration (7× faster)
```

Why Scenario-Based (Not State-Based)?

Metric	Scenario-Based	State-Based	Difference
Forward→Backward sync	0 MB (zero)	800 MB	$\infty\times$ better
Warm-start applicability	95%	5%	19× better

Metric	Scenario-Based	State-Based	Difference
LP solve time per iteration	16 min	116 min	7× faster
Communication per iteration	388 MB	1,188 MB	3× less
Implementation complexity	Simple	Complex	Much simpler
NUMA locality	Perfect	Poor	Perfect vs cache misses

Mathematical Validity: The cuts produced are **mathematically identical** regardless of distribution strategy. A cut computed at state x captures the marginal value of storage changes. Whether that state came from scenario 5 or scenario 50 doesn't affect the cut's validity for approximating the cost-to-go function.

The state-based approach's only theoretical advantage (deduplication of identical states) saves ~2% of LP solves—completely overwhelmed by the 7× penalty from losing warm-start.

21.3 Work Distribution Implementation

```rust
/// Work distribution utilities
pub struct WorkDistributor {
    rank: usize,
    world_size: usize,
}

impl WorkDistributor {
    /// Distribute N scenarios across ranks (static, balanced)
    /// Returns the range of scenario indices for this rank
    pub fn distribute_scenarios(&self, total_scenarios: usize) -> Range<usize> {
        let base = total_scenarios / self.world_size;
        let remainder = total_scenarios % self.world_size;

        let start = self.rank * base + self.rank.min(remainder);
        let count = base + if self.rank < remainder { 1 } else { 0 };

        start..start + count
    }

    /// Get counts and displacements for MPI collective operations
    pub fn get_distribution_info(&self, total: usize) -> (Vec<i32>, Vec<i32>) {
        let base = total / self.world_size;
        let remainder = total % self.world_size;

        let counts: Vec<i32> = (0..self.world_size)
            .map(|r| (base + if r < remainder { 1 } else { 0 }) as i32)
            .collect();

        let displs: Vec<i32> = counts.iter()
            .scan(0, |acc, &c| {
                let d = *acc;
                *acc += c;
                Some(d)
            })
            .collect();

        (counts, displs)
    }
}
```

```rust
/// Backward pass execution with scenario-based distribution
pub fn execute_backward_pass<'a>(
    ctx: &ParallelContext,
    scenarios: &[Scenario],
    fcf: &mut Fcf,
    solver_pool: &mut SolverPool,
) -> Vec<Cut> {
    let my_scenarios = ctx.distribute_scenarios(scenarios.len());
    let mut all_cuts = Vec::new();

    // Process stages in reverse order (T-1, T-2, ..., 1)
    for stage in (1..ctx.n_stages).rev() {
        let mut stage_cuts = Vec::new();

        // Process each scenario assigned to this rank
        for scenario_idx in my_scenarios.clone() {
            let scenario = &scenarios[scenario_idx];
            let state = scenario.state_at_stage(stage);

            // Parallel over noise outcomes using OpenMP
            let outcome_cuts = parallel_reduce_cuts(
                0..ctx.n_outcomes,
                |outcome_idx, thread_id| {
                    // Get thread-local solver (warm-started from previous solve)
                    let solver = solver_pool.get_solver(thread_id);

                    // Build and solve outcome LP
                    let noise = ctx.get_noise(stage, outcome_idx);
                    let result = solve_outcome_lp(solver, state, stage, noise, fcf);

                    // Return partial cut contribution
                    CutContribution {
                        alpha: result.objective * ctx.outcome_probability(outcome_idx),
                        beta: result.dual_state.scale(ctx.outcome_probability(outcome_idx)),
                    }
                },
            );

            // Aggregate outcomes into single cut for this (scenario, stage)
            let cut = Cut::from_contributions(stage - 1, scenario_idx, &outcome_cuts);
            stage_cuts.push(cut);
        }

        // Synchronize cuts across all ranks for this stage
        let global_cuts = ctx.sync_cuts_allgatherv(&stage_cuts);

        // Add all cuts to FCF (for use in earlier stages)
        for cut in &global_cuts {
            fcf.add_cut(cut.target_stage, cut.clone());
        }

        all_cuts.extend(global_cuts);
    }

    all_cuts
}
```

```rust
/// Parallel reduction collecting cut contributions using OpenMP
fn parallel_reduce_cuts<F>(
    range: Range<usize>,
    f: F,
) -> Vec<CutContribution>
where
    F: Fn(usize, usize) -> CutContribution + Sync,
{
    // Use OpenMP parallel for with thread-local accumulation
    let n_threads = omp::get_max_threads();
    let mut thread_results: Vec<Vec<CutContribution>> =
        (0..n_threads).map(|_| Vec::new()).collect();

    parallel_for_dynamic(range, 1, |outcome_idx, thread_id| {
        let contribution = f(outcome_idx, thread_id);
        // Thread-local append (no contention)
        thread_results[thread_id].push(contribution);
    });

    // Flatten thread-local results
    thread_results.into_iter().flatten().collect()
}
```

---

22. Synchronization Architecture

22.1 Synchronization Points

With scenario-based distribution, synchronization is **minimal and well-defined**:

```
                    SDDP Iteration Synchronization Points


  ITERATION k



    FORWARD PASS (parallel, completely independent)

    Rank 0:              scenarios 0-3
    Rank 1:               scenarios 4-7
    Rank 2:            scenarios 8-11
    ...
    Rank 63:                scenarios 196-199

    NO synchronization needed - each rank works independently


                              (No barrier! Direct transition)


    BACKWARD PASS (parallel, same scenario ownership)

    For each stage t = T-1, T-2, ..., 1:


        Each rank processes its OWN scenarios (no redistribution!)
```

```
    Rank 0:       scenarios 0-3, 20 outcomes each
    Rank 1:        scenarios 4-7, 20 outcomes each
    ...
    Rank 63:         scenarios 196-199, 20 outcomes each

    [OpenMP parallel within rank for outcomes]



  SYNC POINT: MPI_Allgatherv (new cuts for stage t-1)

  Data: ~200 cuts × 16.3 KB = 3.26 MB per stage
  Time: ~5-10 ms (InfiniBand HDR)

[Repeat for each stage - cuts needed for earlier stages]



SYNC POINT: MPI_Allreduce (bounds for convergence check)

Data: 64 bytes (lower bound, upper bound, gap)
Time: ~100  s

[Iteration complete - check convergence]
```

Key Observation: There is **NO** synchronization between forward and backward pass! Each rank seamlessly transitions from forward to backward using the states it already computed. This eliminates the 800 MB state-gathering step that would be required by state-based distribution.

22.2 Synchronization Summary

Phase Transition	Synchronization	Data Volume	Latency
Init → Forward	MPI_Barrier	0	~1 ms
Forward → Backward	**None**	0	0
Backward stage t → t-1	MPI_Allgatherv	3.26 MB	~5-10 ms
Backward → Convergence	MPI_Allreduce	64 bytes	~100 s
Iteration k → k+1	None (or MPI_Barrier for logging)	0	~1 ms

22.3 Thread Synchronization (Within Rank)

```rust
/// Thread synchronization for outcome processing
pub struct OutcomeSync {
    /// Spin barrier (faster than OpenMP barrier for small teams)
    barrier: SpinBarrier,

    /// Per-thread cut contribution buffers (cache-line aligned)
    thread_contributions: Vec<ThreadLocal<Vec<CutContribution>>>,
}

impl OutcomeSync {
```

```rust
    pub fn new(num_threads: usize) -> Self {
        Self {
            barrier: SpinBarrier::new(num_threads),
            thread_contributions: (0..num_threads)
                .map(|_| ThreadLocal::new(Vec::with_capacity(32)))
                .collect(),
        }
    }

    /// Add contribution from current thread (lock-free)
    #[inline]
    pub fn add_contribution(&self, thread_id: usize, contrib: CutContribution) {
        self.thread_contributions[thread_id].get_mut().push(contrib);
    }

    /// Barrier and collect all contributions
    pub fn barrier_and_collect(&self) -> Vec<CutContribution> {
        self.barrier.wait();

        // Only thread 0 collects
        if omp::get_thread_num() == 0 {
            self.thread_contributions.iter()
                .flat_map(|tc| tc.get().drain(..))
                .collect()
        } else {
            self.barrier.wait();  // Wait for collection
            Vec::new()
        }
    }
}

/// Spin barrier (faster than OpenMP barrier for small thread counts)
pub struct SpinBarrier {
    count: std::sync::atomic::AtomicUsize,
    generation: std::sync::atomic::AtomicUsize,
    num_threads: usize,
}

impl SpinBarrier {
    pub fn new(num_threads: usize) -> Self {
        Self {
            count: std::sync::atomic::AtomicUsize::new(0),
            generation: std::sync::atomic::AtomicUsize::new(0),
            num_threads,
        }
    }

    pub fn wait(&self) {
        let gen = self.generation.load(Ordering::Acquire);
        let arrived = self.count.fetch_add(1, Ordering::AcqRel) + 1;

        if arrived == self.num_threads {
            self.count.store(0, Ordering::Release);
            self.generation.fetch_add(1, Ordering::Release);
        } else {
            while self.generation.load(Ordering::Acquire) == gen {
                std::hint::spin_loop();
```

```
            }
        }
    }
}
```

22.4 Lock-Free Cut Aggregation

```rust
/// Lock-free cut accumulation for backward pass
/// Each thread writes to its own buffer, then buffers are merged after barrier
pub struct CutAccumulator {
    /// Per-thread cut buffers (cache-line aligned, no contention)
    thread_cuts: Vec<ThreadLocal<Vec<Cut>>>,
    num_threads: usize,
}

impl CutAccumulator {
    pub fn new(num_threads: usize) -> Self {
        Self {
            thread_cuts: (0..num_threads)
                .map(|_| ThreadLocal::new(Vec::with_capacity(64)))
                .collect(),
            num_threads,
        }
    }

    /// Add cut from current thread (completely lock-free)
    #[inline]
    pub fn add_cut(&self, thread_id: usize, cut: Cut) {
        self.thread_cuts[thread_id].get_mut().push(cut);
    }

    /// Collect all cuts after OpenMP parallel region ends
    /// Must be called from single thread (outside parallel region)
    pub fn collect_and_clear(&mut self) -> Vec<Cut> {
        self.thread_cuts.iter_mut()
            .flat_map(|buf| {
                std::mem::take(buf.get_mut())
            })
            .collect()
    }

    /// Get total cut count without clearing
    pub fn total_cuts(&self) -> usize {
        self.thread_cuts.iter()
            .map(|buf| buf.get().len())
            .sum()
    }
}
```

---

23. Communication Patterns

23.1 MPI Communication Summary

POWE.RS uses a **hierarchical communication architecture** that combines MPI 4.0 persistent collectives for inter-node communication with shared memory for intra-node data sharing. This hybrid approach minimizes latency for the iterative SDDP algorithm.

Operation	When	Data	Pattern	MPI 4.0 Feature
`Bcast`	Initialization	Case data, config	Root → All	Standard
`Allreduce`	Bound computation	Lower/upper bounds	All → All	Persistent
`Allgatherv`	Cut synchronization	New cuts	All → All	Persistent
`Win_fence`	Intra-node sync	FCF updates	Node-local	Shared Window
`Gatherv`	Output	Results	All → Root	Standard

Why MPI 4.0 Persistent Collectives?

In SDDP, the same communication pattern repeats ~100+ times per training run (once per iteration). Persistent collectives amortize the setup cost:

Aspect	Standard Collective	Persistent Collective
Setup cost per call	Full protocol negotiation	Zero (pre-negotiated)
First call latency	100-500 s	500-1000 s (includes init)
Subsequent calls	100-500 s	10-50 s
100 iterations total	10-50 ms	1.5-6 ms
Speedup	Baseline	**5-10×**

23.2 MPI 4.0 Persistent Collectives - C Implementation

The persistent collective wrappers are implemented in C to leverage the MPI 4.0 API directly:

```c
/*
 * MPI 4.0 Persistent Collectives Wrapper (src/parallel/mpi_persistent.c)
 *
 * Requires MPI 4.0+ (OpenMPI 5.0+, MPICH 4.0+, Intel MPI 2021+)
 */

#include <mpi.h>
#include <stdlib.h>

#if MPI_VERSION < 4
#error "MPI 4.0+ required for persistent collectives"
#endif

/*
 * Persistent Allgatherv for cut synchronization
 */
typedef struct {
    MPI_Request request;
    MPI_Comm comm;
    void* sendbuf;
    int sendcount;
    MPI_Datatype sendtype;
    void* recvbuf;
    int* recvcounts;
```

```
    int* displs;
    MPI_Datatype recvtype;
    int initialized;
} PersistentAllgatherv;

int persistent_allgatherv_init(
    PersistentAllgatherv* op,
    void* sendbuf,
    int sendcount,
    MPI_Datatype sendtype,
    void* recvbuf,
    int* recvcounts,
    int* displs,
    MPI_Datatype recvtype,
    MPI_Comm comm
) {
    op->comm = comm;
    op->sendbuf = sendbuf;
    op->sendcount = sendcount;
    op->sendtype = sendtype;
    op->recvbuf = recvbuf;
    op->recvcounts = recvcounts;
    op->displs = displs;
    op->recvtype = recvtype;

    /* MPI 4.0 persistent collective initialization */
    int err = MPI_Allgatherv_init(
        sendbuf, sendcount, sendtype,
        recvbuf, recvcounts, displs, recvtype,
        comm, MPI_INFO_NULL, &op->request
    );

    op->initialized = (err == MPI_SUCCESS);
    return err;
}

int persistent_allgatherv_start(PersistentAllgatherv* op) {
    if (!op->initialized) return MPI_ERR_REQUEST;
    return MPI_Start(&op->request);
}

int persistent_allgatherv_wait(PersistentAllgatherv* op) {
    if (!op->initialized) return MPI_ERR_REQUEST;
    return MPI_Wait(&op->request, MPI_STATUS_IGNORE);
}

int persistent_allgatherv_test(PersistentAllgatherv* op, int* flag) {
    if (!op->initialized) return MPI_ERR_REQUEST;
    return MPI_Test(&op->request, flag, MPI_STATUS_IGNORE);
}

int persistent_allgatherv_free(PersistentAllgatherv* op) {
    if (!op->initialized) return MPI_SUCCESS;
    op->initialized = 0;
    return MPI_Request_free(&op->request);
}
```

```c
/*
 * Persistent Allreduce for bound computation (in-place variant)
 */
typedef struct {
    MPI_Request request;
    MPI_Comm comm;
    void* buf;
    int count;
    MPI_Datatype datatype;
    MPI_Op op;
    int initialized;
} PersistentAllreduce;

int persistent_allreduce_init_inplace(
    PersistentAllreduce* op,
    void* buf,
    int count,
    MPI_Datatype datatype,
    MPI_Op mpi_op,
    MPI_Comm comm
) {
    op->comm = comm;
    op->buf = buf;
    op->count = count;
    op->datatype = datatype;
    op->op = mpi_op;

    int err = MPI_Allreduce_init(
        MPI_IN_PLACE, buf, count, datatype, mpi_op,
        comm, MPI_INFO_NULL, &op->request
    );

    op->initialized = (err == MPI_SUCCESS);
    return err;
}

int persistent_allreduce_start(PersistentAllreduce* op) {
    if (!op->initialized) return MPI_ERR_REQUEST;
    return MPI_Start(&op->request);
}

int persistent_allreduce_wait(PersistentAllreduce* op) {
    if (!op->initialized) return MPI_ERR_REQUEST;
    return MPI_Wait(&op->request, MPI_STATUS_IGNORE);
}

int persistent_allreduce_free(PersistentAllreduce* op) {
    if (!op->initialized) return MPI_SUCCESS;
    op->initialized = 0;
    return MPI_Request_free(&op->request);
}

/*
 * Combined cut synchronization manager for SDDP
 */
typedef struct {
    PersistentAllgatherv cut_gather;
```

```
    PersistentAllreduce bound_reduce;

    /* Staging buffers (64-byte aligned for cache efficiency) */
    void* send_buffer;
    void* recv_buffer;
    int* recv_counts;
    int* recv_displs;
    int send_capacity;
    int recv_capacity;

    /* Bound reduction buffer: [lower_bound, upper_bound, gap] */
    double* bounds;

    int world_size;
    int world_rank;
} CutSyncManager;

int cut_sync_manager_init(
    CutSyncManager* manager,
    MPI_Comm comm,
    int max_cuts_per_rank,
    int cut_size_bytes
) {
    MPI_Comm_size(comm, &manager->world_size);
    MPI_Comm_rank(comm, &manager->world_rank);

    /* Allocate staging buffers (64-byte aligned) */
    manager->send_capacity = max_cuts_per_rank * cut_size_bytes;
    manager->recv_capacity = manager->send_capacity * manager->world_size;

    manager->send_buffer = aligned_alloc(64, manager->send_capacity);
    manager->recv_buffer = aligned_alloc(64, manager->recv_capacity);
    manager->recv_counts = calloc(manager->world_size, sizeof(int));
    manager->recv_displs = calloc(manager->world_size, sizeof(int));
    manager->bounds = aligned_alloc(64, 3 * sizeof(double));

    if (!manager->send_buffer || !manager->recv_buffer ||
        !manager->recv_counts || !manager->recv_displs || !manager->bounds) {
        return MPI_ERR_NO_MEM;
    }

    /* Compute displacements */
    for (int i = 0; i < manager->world_size; i++) {
        manager->recv_displs[i] = i * manager->send_capacity;
        manager->recv_counts[i] = manager->send_capacity;
    }

    /* Initialize persistent Allgatherv for cuts */
    int err = persistent_allgatherv_init(
        &manager->cut_gather,
        manager->send_buffer, manager->send_capacity, MPI_BYTE,
        manager->recv_buffer, manager->recv_counts, manager->recv_displs, MPI_BYTE,
        comm
    );
    if (err != MPI_SUCCESS) return err;

    /* Initialize persistent Allreduce for bounds */
```

```
    err = persistent_allreduce_init_inplace(
        &manager->bound_reduce,
        manager->bounds, 3, MPI_DOUBLE, MPI_SUM,
        comm
    );

    return err;
}

int cut_sync_start_gather(CutSyncManager* manager) {
    return persistent_allgatherv_start(&manager->cut_gather);
}

int cut_sync_wait_gather(CutSyncManager* manager) {
    return persistent_allgatherv_wait(&manager->cut_gather);
}

int cut_sync_start_bounds(CutSyncManager* manager) {
    return persistent_allreduce_start(&manager->bound_reduce);
}

int cut_sync_wait_bounds(CutSyncManager* manager) {
    return persistent_allreduce_wait(&manager->bound_reduce);
}

int cut_sync_manager_free(CutSyncManager* manager) {
    persistent_allgatherv_free(&manager->cut_gather);
    persistent_allreduce_free(&manager->bound_reduce);

    free(manager->send_buffer);
    free(manager->recv_buffer);
    free(manager->recv_counts);
    free(manager->recv_displs);
    free(manager->bounds);

    return MPI_SUCCESS;
}
```

23.3 Rust FFI Bindings for Persistent Collectives

```
//! MPI 4.0 Persistent Collective bindings (src/parallel/mpi_persistent.rs)
//!
//! Safe wrappers around persistent collectives for SDDP cut synchronization.

use std::ffi::c_void;

#[repr(C)]
struct CutSyncManagerFFI {
    // Opaque - managed by C code
    _private: [u8; 256],  // Size must match C struct
}

#[link(name = "mpi_persistent", kind = "static")]
extern "C" {
    fn cut_sync_manager_init(
        manager: *mut CutSyncManagerFFI,
        comm: mpi_sys::MPI_Comm,
```

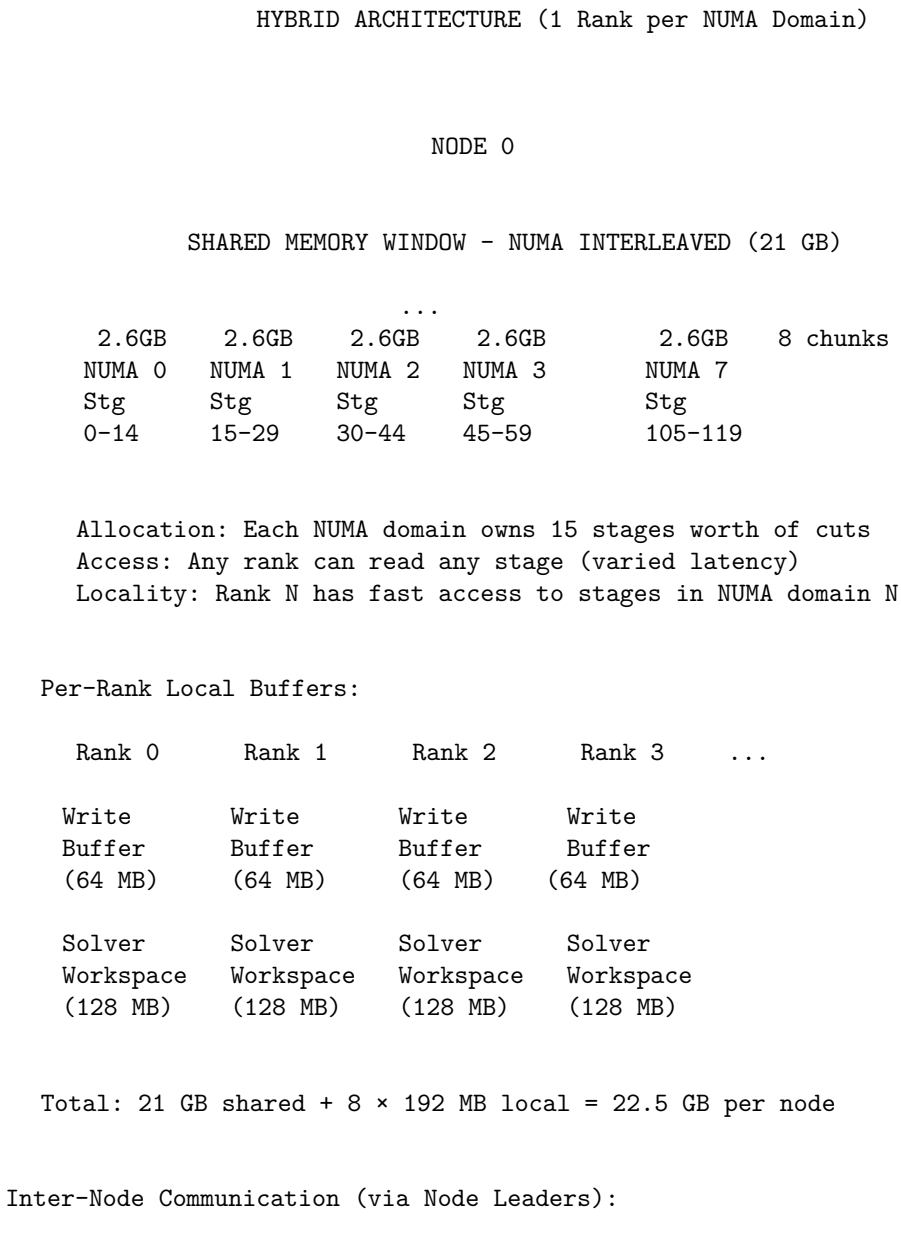
```
        max_cuts_per_rank: i32,
        cut_size_bytes: i32,
    ) -> i32;

    fn cut_sync_start_gather(manager: *mut CutSyncManagerFFI) -> i32;
    fn cut_sync_wait_gather(manager: *mut CutSyncManagerFFI) -> i32;
    fn cut_sync_start_bounds(manager: *mut CutSyncManagerFFI) -> i32;
    fn cut_sync_wait_bounds(manager: *mut CutSyncManagerFFI) -> i32;
    fn cut_sync_manager_free(manager: *mut CutSyncManagerFFI) -> i32;
}

/// Safe wrapper around persistent cut synchronization
pub struct CutSynchronizer {
    inner: Box<CutSyncManagerFFI>,
    cut_size: usize,
    max_cuts: usize,
    world_size: usize,
    world_rank: usize,
}

impl CutSynchronizer {
    /// Create a new cut synchronizer with persistent collectives
    ///
    /// # Arguments
    /// * `comm` - MPI communicator handle
    /// * `max_cuts_per_rank` - Maximum cuts this rank might generate per stage
    /// * `cut_size_bytes` - Size of each serialized cut (typically 16.3 KB)
    pub fn new(
        comm: mpi_sys::MPI_Comm,
        max_cuts_per_rank: usize,
        cut_size_bytes: usize,
    ) -> Result<Self, MpiError> {
        let mut inner = Box::new(unsafe { std::mem::zeroed::<CutSyncManagerFFI>() });

        let result = unsafe {
            cut_sync_manager_init(
                inner.as_mut(),
                comm,
                max_cuts_per_rank as i32,
                cut_size_bytes as i32,
            )
        };

        if result != mpi_sys::MPI_SUCCESS as i32 {
            return Err(MpiError::InitFailed(result));
        }

        // Query world size/rank
        let mut world_size: i32 = 0;
        let mut world_rank: i32 = 0;
        unsafe {
            mpi_sys::MPI_Comm_size(comm, &mut world_size);
            mpi_sys::MPI_Comm_rank(comm, &mut world_rank);
        }

        Ok(Self {
            inner,
```

```
            cut_size: cut_size_bytes,
            max_cuts: max_cuts_per_rank,
            world_size: world_size as usize,
            world_rank: world_rank as usize,
        })
    }

    /// Start asynchronous cut gathering (non-blocking)
    ///
    /// Call this after serializing cuts to send_buffer, then do other work,
    /// then call wait_gather() before reading recv_buffer.
    pub fn start_gather(&mut self) -> Result<(), MpiError> {
        let result = unsafe { cut_sync_start_gather(self.inner.as_mut()) };
        if result != mpi_sys::MPI_SUCCESS as i32 {
            Err(MpiError::StartFailed(result))
        } else {
            Ok(())
        }
    }

    /// Wait for cut gathering to complete (blocking)
    pub fn wait_gather(&mut self) -> Result<(), MpiError> {
        let result = unsafe { cut_sync_wait_gather(self.inner.as_mut()) };
        if result != mpi_sys::MPI_SUCCESS as i32 {
            Err(MpiError::WaitFailed(result))
        } else {
            Ok(())
        }
    }

    /// Start asynchronous bound reduction (non-blocking)
    pub fn start_bounds(&mut self) -> Result<(), MpiError> {
        let result = unsafe { cut_sync_start_bounds(self.inner.as_mut()) };
        if result != mpi_sys::MPI_SUCCESS as i32 {
            Err(MpiError::StartFailed(result))
        } else {
            Ok(())
        }
    }

    /// Wait for bound reduction to complete (blocking)
    pub fn wait_bounds(&mut self) -> Result<(), MpiError> {
        let result = unsafe { cut_sync_wait_bounds(self.inner.as_mut()) };
        if result != mpi_sys::MPI_SUCCESS as i32 {
            Err(MpiError::WaitFailed(result))
        } else {
            Ok(())
        }
    }

    #[inline]
    pub fn world_size(&self) -> usize { self.world_size }

    #[inline]
    pub fn world_rank(&self) -> usize { self.world_rank }
}
```

```rust
impl Drop for CutSynchronizer {
    fn drop(&mut self) {
        unsafe { cut_sync_manager_free(self.inner.as_mut()) };
    }
}

#[derive(Debug)]
pub enum MpiError {
    InitFailed(i32),
    StartFailed(i32),
    WaitFailed(i32),
}
```

23.4 Hybrid Shared Memory Architecture

POWE.RS uses a **hybrid architecture** that combines MPI shared memory windows for intra-node FCF storage with inter-node MPI collectives. This approach achieves 93% memory efficiency while maintaining good NUMA locality.



```
- Node leader = Rank 0 on each node
- Leaders form sub-communicator for inter-node Allgatherv
- Other ranks wait at shared memory fence
- After leader commits, fence releases all ranks
```

Memory Architecture Comparison:

Aspect	Fully Replicated	MPI Shared Window	Hybrid (Selected)
Memory per node	168 GB	29 GB	22.5 GB
Memory efficiency	12.5%	72%	**93%**
Read latency (best)	50 ns	50 ns	50 ns
Read latency (worst)	100 ns	200 ns	150 ns
Write throughput	10M cuts/s	2M cuts/s	10M cuts/s
NUMA impact	None	High (3-4×)	Low (interleaved)
Max problem size	2× current	8× current	**8× current**

23.5 Communication Volume Analysis (Scenario-Based Distribution)

With scenario-based backward pass distribution (see Section 21.2), communication is minimized:

```
                    Communication Volume Analysis


  Production Scale Parameters:

  Hydros: 160, AR order: 12 → State dimension: 2,080
  Stages: 120, Forward scenarios: 200, Noise outcomes: 20
  MPI ranks: 64 (8 nodes × 8 ranks/node)

  Cut Size Calculation:

  Single cut:   (8 bytes) +  [2080] (16,640 bytes) + metadata (16 bytes)
            = 16,664 bytes   16.3 KB per cut

  Communication per Backward Stage:

  Cuts generated: 200 scenarios × 1 cut/scenario = 200 cuts
  Data volume: 200 × 16.3 KB = 3.26 MB per stage

  Total Backward Pass Communication:

  119 stages × 3.26 MB = 388 MB per iteration


    Scenario-Based vs State-Based Comparison

    Metric                      Scenario-Based   State-Based   Ratio
                                             -
    Forward→Backward sync       0 MB             800 MB        ∞× better
    Cut sync per iteration      388 MB           388 MB        Tie
    Total communication         388 MB           1,188 MB      3× better
```

```
  Warm-start applicability    95%             5%           19× better
  LP solve time per iter      16.1 min        116 min      7.2× better


Communication Timing (InfiniBand HDR 200 Gbps):

Bandwidth per link: 25 GB/s
3.26 MB Allgatherv (64 ranks): ~2-5 ms (including protocol overhead)
388 MB total: ~150-200 ms per iteration (overlapped with compute)

With compute/communication overlap (see below), effective overhead: <50 ms
```

23.6 Asynchronous Communication Overlap

The persistent collective API enables overlapping communication with computation:

```rust
/// Backward pass with compute/communication overlap
pub fn backward_pass_with_overlap(
    stages: &[Stage],
    scenarios: &[Scenario],
    fcf: &mut HybridFcfStorage,
    sync: &mut CutSynchronizer,
) {
    for (stage_idx, stage) in stages.iter().rev().skip(1).enumerate() {
        //
        // Phase 1: Generate cuts (OpenMP parallel over scenarios/outcomes)
        //
        let local_cuts = generate_cuts_parallel(stage, scenarios, fcf);

        // Serialize cuts to persistent send buffer
        let bytes_written = serialize_cuts(&local_cuts, sync.send_buffer_mut());

        //
        // Phase 2: Start async gather while processing local cuts
        //
        sync.start_gather().expect("Failed to start gather");

        // OVERLAP: Add local cuts to FCF (no communication wait needed)
        for cut in &local_cuts {
            fcf.buffer_cut(cut.to_cut_data());
        }
        fcf.commit_buffered_cuts(stage.id - 1);

        // OVERLAP: If not first iteration, also start next forward scenarios
        // (only if resources available)

        //
        // Phase 3: Wait for remote cuts and integrate
        //
        sync.wait_gather().expect("Failed to wait for gather");

        // Deserialize and add remote cuts
        let recv_buffer = sync.recv_buffer();
        for rank in 0..sync.world_size() {
            if rank != sync.world_rank() {
                let offset = rank * sync.cut_size * sync.max_cuts;
```

```
                let cuts = deserialize_cuts(&recv_buffer[offset..]);
                for cut in cuts {
                    fcf.buffer_cut(cut);
                }
            }
        }
        fcf.commit_buffered_cuts(stage.id - 1);

        // Intra-node fence ensures all ranks see the updates
        fcf.sync_intra_node();
    }
}
```

23.7 Communication Performance Targets

```
                    Communication Performance Targets


  Operation               Data Size      Target Latency   Notes

  Persistent Allgatherv   3.26 MB        < 5 ms           Per stage (overlapped)
  Persistent Allreduce    24 bytes       < 50  s          Bounds computation
  Shared memory fence     N/A            < 10  s          Intra-node sync
  Initialization Bcast    10-50 MB       < 500 ms         Once at startup

  Iteration Time Breakdown (target):

  Forward pass compute:     55%
  Backward pass compute:    35%
  Communication (visible):   5%   (most overlapped)
  Synchronization:           5%

  Scaling Efficiency Target:

  8 → 64 ranks: > 85% parallel efficiency
  64 → 512 ranks: > 70% parallel efficiency
  (Measured as: T_base × base_ranks / (T_scaled × scaled_ranks))

  Latency Comparison:

  Access Type             Latency        Relative

  L1 cache                ~1 ns          1×
  L2 cache                ~4 ns          4×
  L3 cache (local CCD)    ~15 ns         15×
  L3 cache (remote CCD)   ~40 ns         40×
  Local DRAM              ~80 ns         80×
  Remote NUMA (same)      ~120 ns        120×
  Remote NUMA (cross)     ~180 ns        180×
  InfiniBand (small)      ~1  s          1,000×
  InfiniBand (3 MB)       ~2-5 ms        2,000,000-5,000,000×

  This is why NUMA-aware shared memory is critical for intra-node FCF access!
```

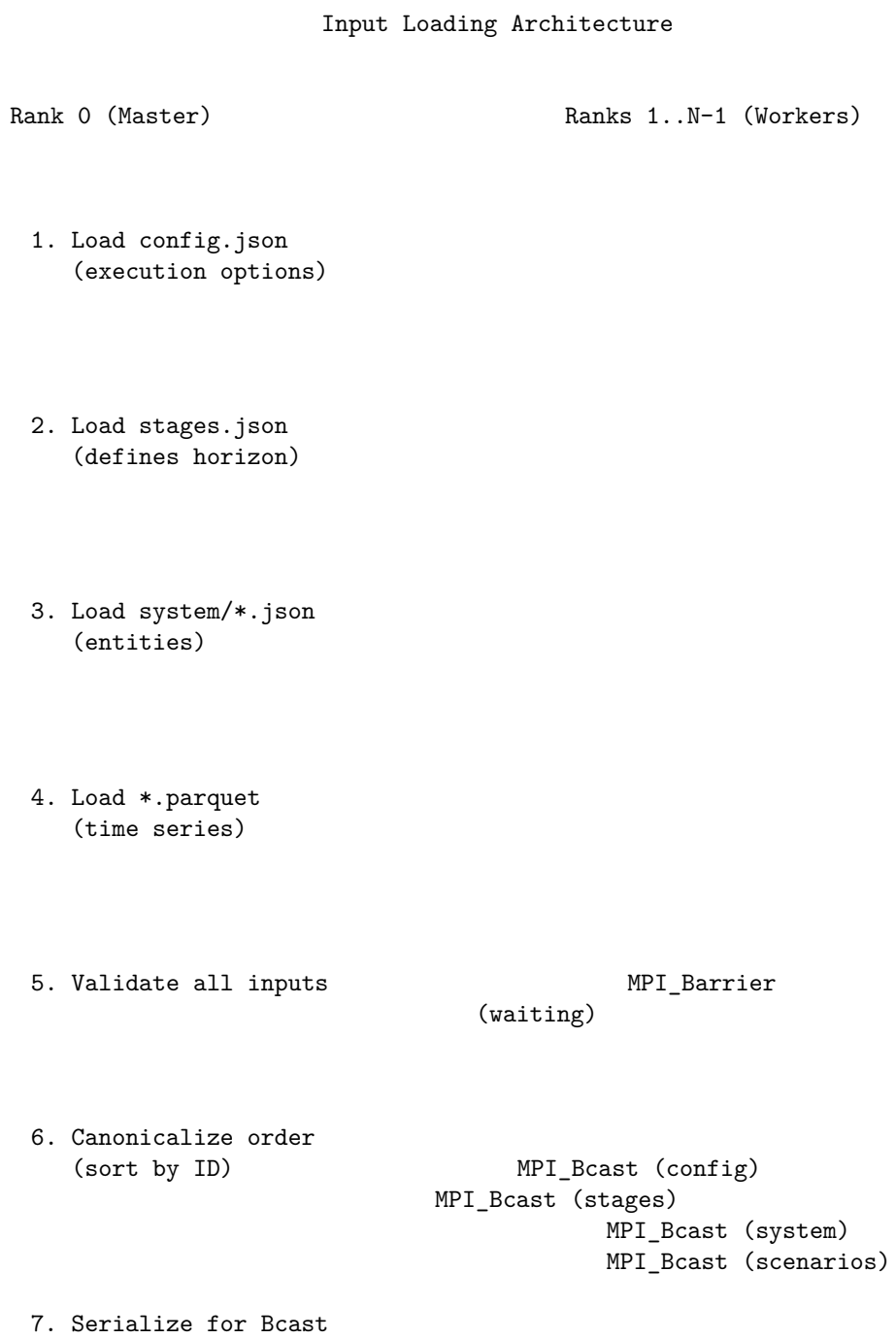
Part II: Input Processing

---

4. Input Loading Pipeline

4.1 Loading Architecture

Input loading follows a **rank-0 centric** pattern: the master rank loads and validates all input data, then broadcasts to worker ranks. This design: - Minimizes filesystem contention on parallel filesystems - Centralizes validation logic - Reduces complexity of error handling across ranks



4.2 File Loading Sequence

Files are loaded in dependency order to enable early validation:

Order	File(s)	Dependencies	Validation
1	`config.json`	None	Schema, execution mode
2	`stages.json`	config (horizon mode)	Stage count, transitions
3	`penalties.json`	None	Penalty values > 0
4	`initial_conditions.json`	None	Entity references (deferred)
5	`system/buses.json`	None	Bus IDs unique
6	`system/lines.json`	buses	Source/target bus refs
7	`system/hydros.json`	buses	Bus refs, cascade refs
8	`system/thermals.json`	buses	Bus refs
9	`scenarios/inflow_models.parquet`	hydros, stages	Hydro/stage coverage
10	`scenarios/correlation.json`	hydros	Block membership
11	`constraints/*.parquet`	entities, stages	Entity/stage refs
12	`policy/*` (if warm-start)	All above	State dictionary match

4.3 Loader Interface

```rust
/// Main input loader - orchestrates the loading pipeline
pub struct InputLoader {
    case_dir: PathBuf,
    config: Option<Config>,
    validation_results: ValidationResult,
}

impl InputLoader {
    /// Load all inputs with validation
    /// Returns fully validated and canonicalized CaseData
    pub fn load(&mut self) -> Result<CaseData, LoadError> {
        // Phase 1: Load configuration
        self.config = Some(self.load_config()?);

        // Phase 2: Load stages (depends on horizon mode)
        let stages = self.load_stages()?;

        // Phase 3: Load system entities
        let system = self.load_system()?;

        // Phase 4: Load scenario data
        let scenarios = self.load_scenarios(&system, &stages)?;

        // Phase 5: Load constraints
        let constraints = self.load_constraints(&system, &stages)?;

        // Phase 6: Load policy (if warm-start)
        let policy = self.load_policy_if_needed(&system)?;

        // Phase 7: Final validation and canonicalization
        let case_data = CaseData {
```

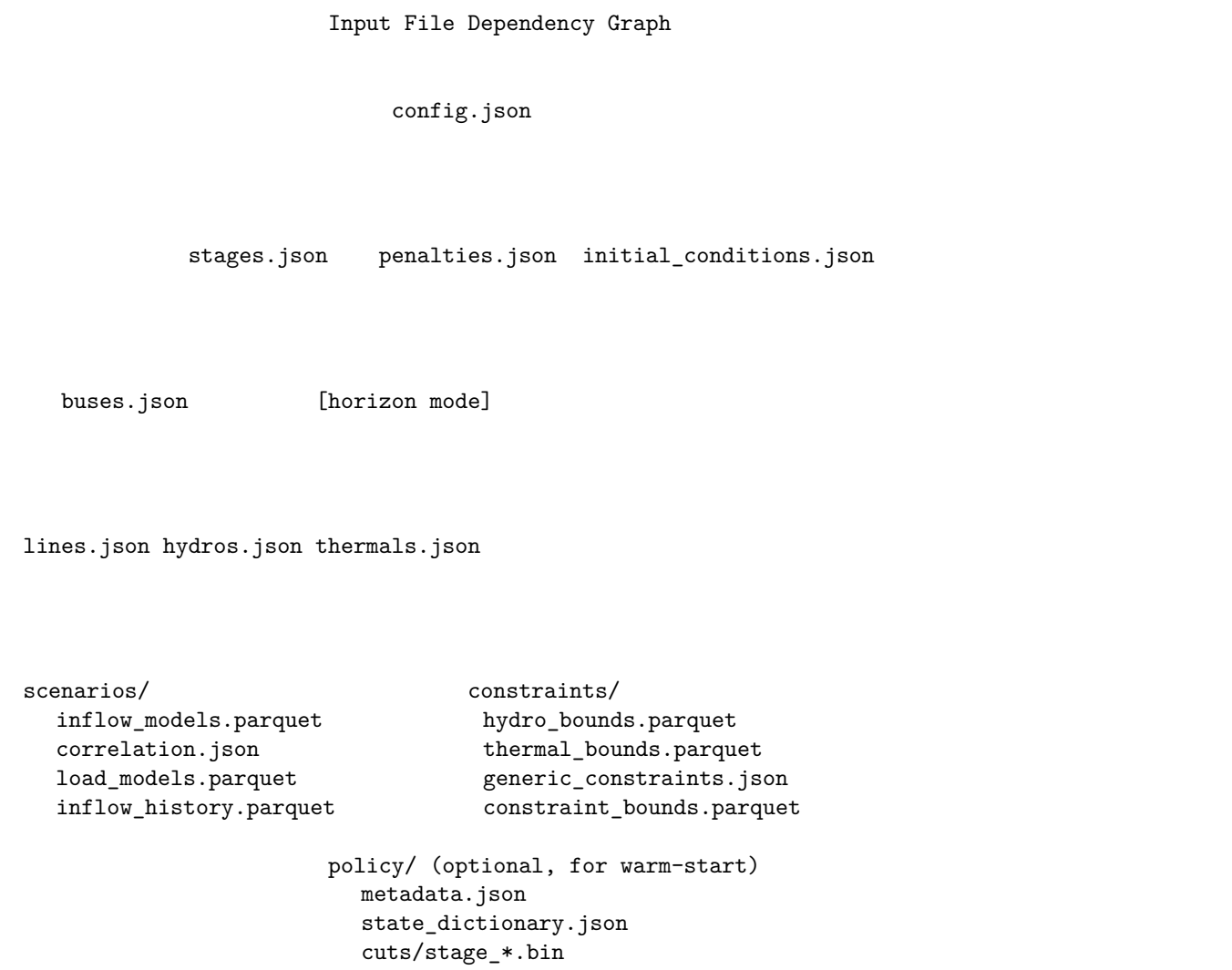
```
            config: self.config.take().unwrap(),
            stages,
            system,
            scenarios,
            constraints,
            policy,
        };

        case_data.canonicalize();
        self.validate_cross_references(&case_data)?;

        Ok(case_data)
    }
}
```

---

5. Dependency Resolution and Load Order

5.1 Dependency Graph

Input files form a directed acyclic graph (DAG) of dependencies:



5.2 Conditional Loading

Some files are loaded conditionally based on configuration:

Condition	Files Affected
`training.enabled = false`	Skip scenario noise generation
`simulation.enabled = false`	Skip simulation scenario loading
`policy.mode = "warm_start"`	Load `policy/*` files
`horizon.mode = "infinite_periodic"`	Validate cycle in stages
Hydros with `fpha_enabled = true`	Load `fpha_hyperplanes.parquet`
Hydros with pumping	Load `pumping_stations.json`

5.3 Sparse Time-Series Handling

Time-series Parquet files use **sparse representation**: only non-default values are stored. The loader must:

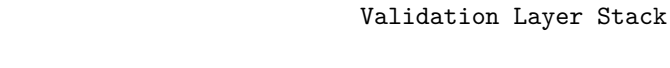
1. Load sparse data from Parquet
2. Expand to dense representation using defaults
3. Validate stage coverage

```rust
/// Sparse-to-dense expansion for time series
pub fn expand_bounds<T: Default + Clone>(
    sparse: &[(StageId, EntityId, T)],
    stages: &[Stage],
    entities: &[EntityId],
    default: T,
) -> Vec<Vec<T>> {
    // Initialize with defaults
    let mut dense = vec![vec![default.clone(); entities.len()]; stages.len()];

    // Overlay sparse values
    for (stage_id, entity_id, value) in sparse {
        let stage_idx = stages.iter().position(|s| s.id == *stage_id)
            .expect("Invalid stage_id");
        let entity_idx = entities.iter().position(|e| *e == *entity_id)
            .expect("Invalid entity_id");
        dense[stage_idx][entity_idx] = value.clone();
    }

    dense
}
```

---

6. Validation Architecture

6.1 Validation Layers



Layer 4: Semantic Validation (Business Rules)

- Storage min <= initial <= max
- AR order <= available history length
- Discount rate required for cycles

- Sum of block durations > 0 for each stage

Layer 3: Referential Integrity

- Foreign key references valid (bus_id in hydros → buses)
- Cascade references form DAG (no cycles)
- Stage IDs in time-series match stages.json

Layer 2: Schema Validation

- JSON conforms to JSON Schema
- Parquet columns have expected names and types
- Required fields present

Layer 1: Structural Validation

- Files exist and are readable
- Valid JSON/Parquet format
- UTF-8 encoding

6.2 Error Collection Strategy

Validation collects **all errors** before failing, rather than failing on the first error:

```rust
pub struct ValidationContext {
    errors: Vec<ValidationError>,
    warnings: Vec<ValidationWarning>,
    current_file: Option<PathBuf>,
    current_entity: Option<String>,
}

impl ValidationContext {
    /// Record an error without immediately failing
    pub fn error(&mut self, kind: ErrorKind, message: impl Into<String>) {
        self.errors.push(ValidationError {
            file: self.current_file.clone(),
            entity: self.current_entity.clone(),
            kind,
            message: message.into(),
        });
    }

    /// Check if validation passed
    pub fn is_valid(&self) -> bool {
        self.errors.is_empty()
    }

    /// Generate detailed report
    pub fn into_result(self) -> ValidationResult {
        ValidationResult {
            valid: self.errors.is_empty(),
            errors: self.errors,
```

```
            warnings: self.warnings,
        }
    }
}
```

6.3 Validation Error Types

Error Kind	Severity	Description	Example
`FileNotFound`	Error	Required file missing	`hydros.json` not found
`ParseError`	Error	Invalid JSON/Parquet	Malformed JSON syntax
`SchemaViolation`	Error	Schema mismatch	Missing required field
`InvalidReference`	Error	Foreign key invalid	`bus_id: 999` not in buses
`DuplicateId`	Error	ID uniqueness violation	Two hydros with same ID
`InvalidValue`	Error	Value out of range	`storage_max < storage_min`
`CycleDetected`	Error	Invalid graph structure	Cascade forms cycle
`MissingData`	Warning	Optional data absent	No FPHA planes for hydro
`UnusedEntity`	Warning	Entity defined but unused	Thermal not in any bus

6.4 Validation Report Format

```
{
  "valid": false,
  "timestamp": "2026-01-31T10:30:00Z",
  "case_directory": "/path/to/case",
  "errors": [
    {
      "file": "system/hydros.json",
      "entity": "hydro_042",
      "kind": "InvalidReference",
      "message": "bus_id 'BUS_99' not found in buses.json"
    },
    {
      "file": "scenarios/inflow_models.parquet",
      "entity": null,
      "kind": "MissingData",
      "message": "No PAR coefficients for hydro 'hydro_015' at stage 48"
    }
  ],
  "warnings": [
    {
      "file": "system/thermals.json",
      "entity": "thermal_old",
      "kind": "UnusedEntity",
      "message": "Thermal 'thermal_old' has max_generation=0 for all stages"
    }
  ],
  "summary": {
```

```
    "files_checked": 24,
    "entities_validated": 456,
    "error_count": 2,
    "warning_count": 1
  }
}
```

---

7. Data Broadcasting

7.1 Broadcast Strategy

After rank 0 loads and validates all data, it must be distributed to workers. The strategy depends on data size and structure:

Data Broadcasting Strategy

Data Type	Size	Strategy
Config	<10 KB	MPI_Bcast (serialized JSON)
Stages	<100 KB	MPI_Bcast (serialized)
System (entities)	1-10 MB	MPI_Bcast (binary serialized)
PAR models	10-50 MB	MPI_Bcast (packed arrays)
Correlation matrices	1-5 MB	MPI_Bcast (dense matrix)
Time-series bounds	10-50 MB	MPI_Bcast (sparse then expand locally)
FCF cuts (warm-start)	1-20 GB	Parallel load (each rank loads subset)

7.2 Serialization for Broadcast

Data is serialized to contiguous byte buffers for efficient MPI broadcast:

```rust
/// Trait for MPI-broadcastable data
pub trait MpiBroadcast: Sized {
    /// Serialize to bytes for broadcast
    fn to_broadcast_bytes(&self) -> Vec<u8>;

    /// Deserialize from broadcast bytes
    fn from_broadcast_bytes(bytes: &[u8]) -> Self;
}

/// Broadcast wrapper handling size negotiation
pub fn broadcast_data<T: MpiBroadcast>(
    comm: &impl Communicator,
    data: Option<T>,  // Some on rank 0, None on workers
    root: Rank,
) -> T {
    let rank = comm.rank();

    if rank == root {
        let data = data.expect("Root must provide data");
        let bytes = data.to_broadcast_bytes();

        // First broadcast: size
        let size = bytes.len() as u64;
```

```
        comm.broadcast(&size, root);

        // Second broadcast: data
        comm.broadcast(&bytes, root);

        data
    } else {
        // Receive size
        let mut size: u64 = 0;
        comm.broadcast(&mut size, root);

        // Receive data
        let mut bytes = vec![0u8; size as usize];
        comm.broadcast(&mut bytes, root);

        T::from_broadcast_bytes(&bytes)
    }
}
```

7.3 Parallel Policy Loading (Warm-Start)

For large policy files (cuts), parallel loading improves startup time:

Parallel Policy Loading Pattern

```
120 stages, 8 ranks:

Rank 0: Load stages [0, 8, 16, 24, ...]    (15 stages)
Rank 1: Load stages [1, 9, 17, 25, ...]    (15 stages)
Rank 2: Load stages [2, 10, 18, 26, ...]   (15 stages)
...
Rank 7: Load stages [7, 15, 23, 31, ...]   (15 stages)


 After local loading:

   MPI_Allgatherv to collect all cuts on all ranks
   OR
   Use shared memory window (intra-node) + MPI_Bcast (inter-node)


Time comparison (20 GB policy, 200 MB/s parallel FS):
Sequential: 20 GB / 200 MB/s = 100s
Parallel (8 ranks): 2.5 GB / 200 MB/s + sync overhead   15s
```

7.4 Memory Layout After Broadcast

After broadcasting, each rank has identical copies of:

```
/// Complete case data available on all ranks after initialization
pub struct DistributedCaseData {
    // Replicated on all ranks (small, read-only)
    pub config: Arc<Config>,
    pub stages: Arc<Vec<Stage>>,
    pub system: Arc<System>,
```

```
    // Replicated on all ranks (medium, read-only)
    pub par_models: Arc<ParModels>,
    pub correlation: Arc<CorrelationData>,

    // Shared within node (large, uses MPI shared memory window)
    pub fcf: SharedFcf,  // Future Cost Function cuts

    // Per-rank (scenario-specific data)
    pub my_scenarios: Vec<ScenarioId>,
    pub noise_samples: Vec<NoiseSample>,
}
```

Part III: Scenario Generation

---

8. PAR Model Preprocessing

8.1 PAR(p) Model Overview

The Periodic Autoregressive model PAR(p) is the primary stochastic model for inflows in POWE.RS. For each hydro h at stage t with season $m = m(t)$:

$$a_{h,t} = \mu_m + \sum_{\ell=1}^{P_h} \psi_{m,\ell} \cdot (a_{h,t-\ell} - \mu_{m-\ell}) + \sigma_m \cdot \eta_{h,t}$$

where: - μ_m = seasonal mean inflow - $\psi_{m,\ell}$ = AR coefficient for lag ℓ at season m - σ_m = residual standard deviation - $\eta_{h,t}$ = standard normal innovation (noise)

8.2 Preprocessing Workflow

```
                    PAR Model Preprocessing Pipeline


  Input: inflow_models.parquet (PAR coefficients per hydro x season)


   Step 1: Load PAR Parameters
   - mu[h][m]: seasonal means (12 x N_hydro)
   - psi[h][m][l]: AR coefficients (12 x N_hydro x max_order)
   - sigma[h][m]: residual std dev (12 x N_hydro)
   - P[h]: AR order per hydro (N_hydro)




   Step 2: Precompute Stage-Specific Coefficients
   For each stage t = 1..T:
     m = season(t)
     For each hydro h:
       base[h][t] = mu[h][m] - Sum_l psi[h][m][l] * mu[h][m-l]
       coeff[h][t][l] = psi[h][m][l]  for l = 1..P[h]
       scale[h][t] = sigma[h][m]
```

```
    Step 3: Initialize Lag State from History
    From inflow_history.parquet:
      lag_state[h][l] = historical_inflow[h][t0 - l]
      for l = 1..max_order


  Output: PrecomputedPar structure (contiguous arrays for hot-path access)
```

8.3 Memory Layout for Hot-Path Access

```rust
/// Precomputed PAR data optimized for forward pass access pattern
pub struct PrecomputedPar {
    /// Stage -> Hydro -> base value (deterministic component)
    /// Layout: [stage_0_hydro_0, stage_0_hydro_1, ..., stage_T_hydro_N]
    pub base: Vec<f64>,  // T x N_hydro, row-major

    /// Stage -> Hydro -> Lag -> coefficient
    /// Layout: [s0_h0_l1, s0_h0_l2, ..., s0_h0_lP, s0_h1_l1, ...]
    pub coefficients: Vec<f64>,  // T x N_hydro x max_order

    /// Stage -> Hydro -> noise scale (sigma)
    pub scales: Vec<f64>,  // T x N_hydro

    /// Hydro -> AR order
    pub orders: Vec<u8>,  // N_hydro

    /// Dimensions for indexing
    pub n_stages: usize,
    pub n_hydros: usize,
    pub max_order: usize,
}

impl PrecomputedPar {
    /// Get base value for stage t, hydro h
    #[inline]
    pub fn base(&self, stage: usize, hydro: usize) -> f64 {
        self.base[stage * self.n_hydros + hydro]
    }

    /// Get AR coefficients for stage t, hydro h
    #[inline]
    pub fn coefficients(&self, stage: usize, hydro: usize) -> &[f64] {
        let start = (stage * self.n_hydros + hydro) * self.max_order;
        let order = self.orders[hydro] as usize;
        &self.coefficients[start..start + order]
    }

    /// Compute inflow given lag state and noise
    #[inline]
    pub fn compute_inflow(
```

```
        &self,
        stage: usize,
        hydro: usize,
        lag_state: &[f64],
        noise: f64,
    ) -> f64 {
        let base = self.base(stage, hydro);
        let coeffs = self.coefficients(stage, hydro);
        let scale = self.scales[stage * self.n_hydros + hydro];

        let mut inflow = base;
        for (ell, &coeff) in coeffs.iter().enumerate() {
            inflow += coeff * lag_state[ell];
        }
        inflow += scale * noise;

        inflow
    }
}
```

8.5 PAR Model Fitting from Historical Data

When PAR coefficients are not provided in `inflow_models.parquet`, POWE.RS can fit PAR models directly from historical inflow data using the Yule-Walker method with BIC-based order selection.

8.5.1 Fitting Overview

```
                    PAR Model Fitting Pipeline


  Input: inflow_history.parquet (historical inflows per hydro per month)
         Minimum: max_order + 2 years of history required


    Step 1: Compute Seasonal Statistics

      For each hydro h, season m = 1..12:
         [h,m] = mean of historical inflows in season m
        ²[h,m] = variance of historical inflows in season m

      Standardize: z[h,t] = (a[h,t] -  [h,m(t)]) /  [h,m(t)]




    Step 2: Compute Autocorrelation (per season)

      For each hydro h, season m, lag   = 1..max_order:
         [h,m, ] = Cov(z[h,t], z[h,t- ]) / Var(z[h,t])
                  for all t where season(t) = m




    Step 3: Yule-Walker Estimation
```

```
  For each candidate order p = 1..max_order:
    Solve Toeplitz system: R ·   = r
    where R[i,j] =  [|i-j|], r[ ] =  [ ]

    Compute residual variance:  ²_p =  ² · (1 -  '·r)



Step 4: BIC Order Selection

  BIC(p) = n·log( ²_p) + p·log(n)

  Select: p* = argmin BIC(p)

  Constraints:
  - p*   max_order (from config, typically 12 for monthly)
  - p*   (history_length - 12) / 12 - 1 (sufficient data)


Output: PAR coefficients  [h,m, ] and residual  [h,m] per hydro/season
```

8.5.2 Yule-Walker Implementation

```rust
/// Fit PAR model using Yule-Walker method with BIC order selection
pub struct ParFitter {
    max_order: usize,
    min_history_years: usize,
}

impl ParFitter {
    /// Fit PAR model from historical inflows
    pub fn fit(
        &self,
        history: &[f64],  // Chronological monthly inflows
        hydro_id: &str,
    ) -> Result<FittedPar, FitError> {
        let n_months = history.len();
        let n_years = n_months / 12;

        // Validate sufficient history
        if n_years < self.min_history_years {
            return Err(FitError::InsufficientHistory {
                hydro: hydro_id.into(),
                required: self.min_history_years,
                available: n_years,
            });
        }

        // Step 1: Compute seasonal statistics
        let (means, std_devs) = self.compute_seasonal_stats(history);

        // Step 2: Standardize series
```

```
    let standardized = self.standardize(history, &means, &std_devs);

    // Fit each season independently
    let mut coefficients = vec![vec![0.0; self.max_order]; 12];
    let mut residual_std = vec![0.0; 12];
    let mut orders = vec![0usize; 12];

    for season in 0..12 {
        // Step 3: Compute autocorrelation for this season
        let autocorr = self.compute_autocorrelation(&standardized, season, self.max_order);

        // Step 4: Try each order and compute BIC
        let n_samples = n_years - 1;  // Samples available for this season
        let mut best_bic = f64::INFINITY;
        let mut best_order = 1;
        let mut best_coeffs = vec![0.0; self.max_order];
        let mut best_resid_var = std_devs[season].powi(2);

        for p in 1..=self.max_order.min(n_samples - 2) {
            // Solve Yule-Walker equations
            let (coeffs, resid_var) = self.yule_walker_solve(&autocorr, p);

            // Compute BIC
            let bic = n_samples as f64 * resid_var.ln() + p as f64 * (n_samples as f64).ln();

            if bic < best_bic {
                best_bic = bic;
                best_order = p;
                best_coeffs[..p].copy_from_slice(&coeffs);
                best_resid_var = resid_var;
            }
        }

        orders[season] = best_order;
        coefficients[season] = best_coeffs;
        residual_std[season] = best_resid_var.sqrt();
    }

    Ok(FittedPar {
        hydro_id: hydro_id.into(),
        means,
        coefficients,
        residual_std,
        orders,
    })
}

/// Solve Yule-Walker equations using Levinson-Durbin algorithm
fn yule_walker_solve(&self, autocorr: &[f64], order: usize) -> (Vec<f64>, f64) {
    // Levinson-Durbin recursion (O(p^2) instead of O(p^3))
    let mut coeffs = vec![0.0; order];
    let mut error = 1.0;  // Normalized (autocorr[0] = 1)

    for k in 0..order {
        // Compute reflection coefficient
        let mut lambda = autocorr[k + 1];
        for j in 0..k {
```

```
                lambda -= coeffs[j] * autocorr[k - j];
            }
            lambda /= error;

            // Update coefficients
            let mut new_coeffs = vec![0.0; k + 1];
            new_coeffs[k] = lambda;
            for j in 0..k {
                new_coeffs[j] = coeffs[j] - lambda * coeffs[k - 1 - j];
            }
            coeffs[..=k].copy_from_slice(&new_coeffs);

            // Update prediction error
            error *= 1.0 - lambda * lambda;
        }

        (coeffs, error)
    }

    /// Compute autocorrelation for a specific season
    fn compute_autocorrelation(
        &self,
        standardized: &[f64],
        season: usize,
        max_lag: usize,
    ) -> Vec<f64> {
        let mut autocorr = vec![0.0; max_lag + 1];
        autocorr[0] = 1.0;  // Autocorrelation at lag 0

        // Get indices for this season
        let season_indices: Vec<usize> = (0..standardized.len())
            .filter(|&i| i % 12 == season)
            .collect();

        let n = season_indices.len();

        for lag in 1..=max_lag {
            let mut sum = 0.0;
            let mut count = 0;

            for &i in &season_indices {
                if i >= lag * 12 {
                    sum += standardized[i] * standardized[i - lag];
                    count += 1;
                }
            }

            if count > 0 {
                autocorr[lag] = sum / count as f64;
            }
        }

        autocorr
    }
}

/// Fitted PAR model
```

```rust
pub struct FittedPar {
    pub hydro_id: String,
    pub means: Vec<f64>,           // [12] seasonal means
    pub coefficients: Vec<Vec<f64>>, // [12][max_order] AR coefficients per season
    pub residual_std: Vec<f64>,    // [12] residual standard deviation
    pub orders: Vec<usize>,        // [12] selected order per season
}
```

8.5.3 Validation Requirements

The fitted PAR model must pass validation before use:

Validation	Criterion	Action on Failure
Stationarity	All AR roots outside unit circle	Reduce order or warn
Residual normality	Shapiro-Wilk $p > 0.05$	Warn (non-fatal)
No negative inflows	Simulated inflows stay positive	Truncate at 0 with warning
Cross-validation	Out-of-sample $R^2 > 0.3$	Warn if poor fit

```rust
impl FittedPar {
    /// Validate the fitted model
    pub fn validate(&self) -> ValidationResult {
        let mut warnings = Vec::new();
        let mut errors = Vec::new();

        for season in 0..12 {
            // Check stationarity via AR polynomial roots
            let roots = self.compute_ar_roots(season);
            for (i, root) in roots.iter().enumerate() {
                if root.norm() <= 1.0 {
                    warnings.push(format!(
                        "Season {}: AR root {} inside unit circle (|z|={:.3})",
                        season + 1, i + 1, root.norm()
                    ));
                }
            }

            // Check for negative coefficient sum (explosive behavior)
            let coef_sum: f64 = self.coefficients[season].iter().sum();
            if coef_sum >= 1.0 {
                errors.push(format!(
                    "Season {}: Coefficient sum {:.3} >= 1 (non-stationary)",
                    season + 1, coef_sum
                ));
            }
        }

        ValidationResult { errors, warnings }
    }
}
```

---

9. Noise Sampling and Correlation

9.1 Correlated Noise Generation

Hydros within the same correlation block share spatially correlated noise. The correlation structure is defined in `correlation.json`.

```
                    Correlated Noise Generation


  Correlation Structure:

    Block 1 (Southeast):  Hydros [H1, H2, H3, H4]
    Block 2 (South):      Hydros [H5, H6, H7]
    Block 3 (Northeast):  Hydros [H8, H9]
    Block 4 (North):      Hydros [H10, H11, H12]



    Step 1: Generate Independent Block Noises

      For each scenario s = 1..S:
        For each stage t = 1..T:
          For each block b = 1..B:
            z[s][t][b] ~ N(0, 1)   (standard normal)




    Step 2: Apply Spatial Correlation (Cholesky)

      Given correlation matrix Sigma (block x block):
      Compute L = cholesky(Sigma)   [lower triangular]

      For each scenario, stage:
        correlated_z = L * independent_z




    Step 3: Map Block Noise to Hydros

      For each hydro h in block b:
        eta[h][t][s] = correlated_z[b]

      (All hydros in same block get same noise realization)
```

9.2 Reproducible Sampling

To ensure reproducibility across MPI ranks and restarts:

```rust
/// Deterministic noise generation with seed management
pub struct NoiseGenerator {
    /// Base seed from config
    base_seed: u64,
```

```rust
    /// Current RNG state (Xoshiro256++)
    rng: Xoshiro256PlusPlus,

    /// Cholesky factor for spatial correlation
    cholesky_l: Vec<f64>,  // Lower triangular, B x B
    n_blocks: usize,
}

impl NoiseGenerator {
    /// Create generator with deterministic seed
    pub fn new(base_seed: u64, correlation_matrix: &[f64], n_blocks: usize) -> Self {
        let cholesky_l = cholesky_decomposition(correlation_matrix, n_blocks);
        Self {
            base_seed,
            rng: Xoshiro256PlusPlus::seed_from_u64(base_seed),
            cholesky_l,
            n_blocks,
        }
    }

    /// Generate noise for a specific (iteration, scenario, stage) tuple
    /// This allows deterministic generation regardless of execution order
    pub fn generate_noise(
        &mut self,
        iteration: u32,
        scenario: u32,
        stage: u32,
    ) -> Vec<f64> {
        // Compute deterministic seed for this (iter, scenario, stage)
        let seed = self.base_seed
            .wrapping_mul(iteration as u64 + 1)
            .wrapping_add(scenario as u64 * 1_000_000)
            .wrapping_add(stage as u64);

        self.rng = Xoshiro256PlusPlus::seed_from_u64(seed);

        // Generate independent standard normals for each block
        let independent: Vec<f64> = (0..self.n_blocks)
            .map(|_| self.standard_normal())
            .collect();

        // Apply Cholesky correlation
        let correlated = self.apply_cholesky(&independent);

        correlated
    }

    fn apply_cholesky(&self, z: &[f64]) -> Vec<f64> {
        let mut result = vec![0.0; self.n_blocks];
        for i in 0..self.n_blocks {
            for j in 0..=i {
                result[i] += self.cholesky_l[i * self.n_blocks + j] * z[j];
            }
        }
        result
    }
```

```
}
```

9.3 Noise Caching Strategy

For backward pass efficiency, noises are pre-generated and cached:

```rust
/// Cached noise samples for all scenarios and stages
pub struct NoiseCache {
    /// Layout: [scenario_0_stage_0_block_0, ..., scenario_S_stage_T_block_B]
    pub samples: Vec<f64>,

    pub n_scenarios: usize,
    pub n_stages: usize,
    pub n_blocks: usize,
}

impl NoiseCache {
    /// Pre-generate all noise samples at initialization
    pub fn generate_all(
        generator: &mut NoiseGenerator,
        n_scenarios: usize,
        n_stages: usize,
        iteration: u32,
    ) -> Self {
        let n_blocks = generator.n_blocks;
        let total = n_scenarios * n_stages * n_blocks;
        let mut samples = Vec::with_capacity(total);

        for scenario in 0..n_scenarios {
            for stage in 0..n_stages {
                let noise = generator.generate_noise(
                    iteration,
                    scenario as u32,
                    stage as u32,
                );
                samples.extend(noise);
            }
        }

        Self { samples, n_scenarios, n_stages, n_blocks }
    }

    /// Get noise for specific (scenario, stage, block)
    #[inline]
    pub fn get(&self, scenario: usize, stage: usize, block: usize) -> f64 {
        let idx = (scenario * self.n_stages + stage) * self.n_blocks + block;
        self.samples[idx]
    }
}
```

---

10. External Scenario Integration

10.1 External Scenario Sources

POWE.RS supports external (deterministic) scenarios for simulation, bypassing stochastic generation:

External Scenario Integration

Use Cases:
- Historical replay: Use actual historical inflows
- Monte Carlo import: Pre-generated scenarios from external tool
- Stress testing: Specific drought/flood scenarios

Input Files (simulation/external_scenarios/):

inflows.parquet

scenario	stage_id	hydro_id	inflow_m3s
0	1	1	245.3
0	1	2	123.7
...	...	...	...

Integration Points:
- Training: Always uses PAR model (external scenarios ignored)
- Simulation: Can use external scenarios if configured

10.2 Scenario Adapter Interface

```rust
/// Trait for scenario data providers
pub trait ScenarioProvider: Send + Sync {
    /// Get inflow for a specific (scenario, stage, hydro)
    fn get_inflow(&self, scenario: usize, stage: usize, hydro: usize) -> f64;

    /// Get load factor for a specific (scenario, stage, bus)
    fn get_load_factor(&self, scenario: usize, stage: usize, bus: usize) -> f64;

    /// Number of scenarios available
    fn n_scenarios(&self) -> usize;
}

/// PAR-based scenario provider for training
pub struct ParScenarioProvider {
    par: Arc<PrecomputedPar>,
    noise_cache: NoiseCache,
    hydro_to_block: Vec<usize>,
}

/// External scenario provider for simulation
pub struct ExternalScenarioProvider {
    inflows: Vec<f64>,  // [scenario][stage][hydro]
    loads: Vec<f64>,    // [scenario][stage][bus]
    n_scenarios: usize,
    n_stages: usize,
    n_hydros: usize,
    n_buses: usize,
}
```

```rust
impl ScenarioProvider for ExternalScenarioProvider {
    fn get_inflow(&self, scenario: usize, stage: usize, hydro: usize) -> f64 {
        let idx = (scenario * self.n_stages + stage) * self.n_hydros + hydro;
        self.inflows[idx]
    }

    fn get_load_factor(&self, scenario: usize, stage: usize, bus: usize) -> f64 {
        let idx = (scenario * self.n_stages + stage) * self.n_buses + bus;
        self.loads[idx]
    }

    fn n_scenarios(&self) -> usize {
        self.n_scenarios
    }
}
```

10.5 Noise Inversion for External Scenarios

When using external (deterministic) scenarios for simulation, POWE.RS must compute the **implied noise values** that would have generated those inflows under the PAR model. This is required because the PAR model's AR components are embedded in the LP constraints (RHS updates depend on lag inflows).

10.5.1 The Inversion Problem

Given an external scenario with target inflow a_t^{target} at stage t for hydro h, we need to find the noise η_t such that the PAR model produces exactly that inflow:

$$a_t^{\text{target}} = \phi_m + \sum_{\ell=1}^{P} \psi_{m,\ell} \cdot a_{t-\ell} + \sigma_m \cdot \eta_t$$

Solving for η_t:

$$\eta_t = \frac{a_t^{\text{target}} - \phi_m - \sum_{\ell=1}^{P} \psi_{m,\ell} \cdot a_{t-\ell}}{\sigma_m}$$

where: - $\phi_m = \mu_m - \sum_{\ell=1}^{P} \psi_{m,\ell} \cdot \mu_{m-\ell}$ (constant term from PAR) - $a_{t-\ell}$ = lagged inflows (from external scenario or history) - σ_m = residual standard deviation for season m

10.5.2 Inversion Pipeline

```
                    Noise Inversion Pipeline


  Input: External inflow scenarios + PAR model + Initial lag history


   Step 1: Initialize Lag Buffer from History

     For each hydro h:
       lag_buffer[h][ ] = inflow_history[h][t0 - ]   for   = 1..P

     This ensures continuity with historical record
```

```
    Step 2: Sequential Inversion (per scenario, per stage)

      For stage t = 1, 2, ..., T:
        For each hydro h:
          m = season(t)
          a_target = external_inflow[scenario][t][h]

          // Compute PAR deterministic component
          deterministic =  [h,m]
          for   = 1..P[h]:
              deterministic +=  [h,m, ] * lag_buffer[h][ -1]

          // Invert for noise
           [h,t] = (a_target - deterministic) /  [h,m]

          // Update lag buffer (shift and insert)
          lag_buffer[h] = [a_target] ++ lag_buffer[h][0..P-1]




    Step 3: Validate Inverted Noises

      For each computed  :
        if | | > 4.0:  // More than 4 standard deviations
          WARNING: "Extreme noise value"

        if  [h,m]   0:  // Near-zero residual variance
          if |a_target - deterministic| > tolerance:
            ERROR: "Cannot match target with    0"
          else:
              = 0  // Target matches deterministic exactly



  Output: Inverted noise values  [scenario][stage][block] for simulation
```

10.5.3 Implementation

```rust
/// Noise inversion for external scenarios
pub struct NoiseInverter<'a> {
    par: &'a PrecomputedPar,
    hydro_to_block: &'a [usize],
    tolerance: f64,
}

impl<'a> NoiseInverter<'a> {
    /// Invert external inflows to noise values
    pub fn invert_scenario(
        &self,
        external_inflows: &[Vec<f64>],  // [stage][hydro]
        initial_lags: &[Vec<f64>],       // [hydro][lag]
    ) -> Result<InvertedNoises, InversionError> {
```

```rust
let n_stages = external_inflows.len();
let n_hydros = self.par.n_hydros;
let n_blocks = *self.hydro_to_block.iter().max().unwrap() + 1;

// Output: noise per stage per block
let mut noises = vec![vec![0.0; n_blocks]; n_stages];
let mut warnings = Vec::new();

// Lag buffer: [hydro][lag_index] (lag_index 0 = most recent)
let mut lag_buffer: Vec<Vec<f64>> = initial_lags.to_vec();

for stage in 0..n_stages {
    // Per-block noise accumulator (for validation)
    let mut block_noises: Vec<Vec<f64>> = vec![Vec::new(); n_blocks];

    for hydro in 0..n_hydros {
        let a_target = external_inflows[stage][hydro];
        let order = self.par.orders[hydro] as usize;

        // Get PAR parameters for this stage
        let base = self.par.base(stage, hydro);
        let coeffs = self.par.coefficients(stage, hydro);
        let sigma = self.par.scales[stage * n_hydros + hydro];

        // Compute deterministic component
        let mut deterministic = base;
        for (ell, &coeff) in coeffs.iter().enumerate() {
            if ell < lag_buffer[hydro].len() {
                deterministic += coeff * lag_buffer[hydro][ell];
            }
        }

        // Invert for noise
        let noise = if sigma.abs() < self.tolerance {
            // Near-zero variance: check if target matches
            let residual = (a_target - deterministic).abs();
            if residual > self.tolerance {
                return Err(InversionError::ZeroVarianceResidual {
                    hydro,
                    stage,
                    residual,
                });
            }
            0.0  // Target matches deterministic
        } else {
            (a_target - deterministic) / sigma
        };

        // Validate noise magnitude
        if noise.abs() > 4.0 {
            warnings.push(InversionWarning::ExtremeNoise {
                hydro,
                stage,
                noise,
            });
        }
```

```rust
                // Store for block aggregation
                let block = self.hydro_to_block[hydro];
                block_noises[block].push(noise);

                // Update lag buffer
                lag_buffer[hydro].insert(0, a_target);
                if lag_buffer[hydro].len() > order {
                    lag_buffer[hydro].pop();
                }
            }

            // Aggregate to block level (average or check consistency)
            for block in 0..n_blocks {
                if !block_noises[block].is_empty() {
                    // Check hydros in same block have consistent noises
                    let mean: f64 = block_noises[block].iter().sum::<f64>()
                        / block_noises[block].len() as f64;
                    let max_diff = block_noises[block].iter()
                        .map(|&n| (n - mean).abs())
                        .fold(0.0, f64::max);

                    if max_diff > 0.5 {
                        warnings.push(InversionWarning::InconsistentBlockNoises {
                            block,
                            stage,
                            max_diff,
                        });
                    }

                    noises[stage][block] = mean;
                }
            }
        }

        Ok(InvertedNoises { noises, warnings })
    }
}

/// Result of noise inversion
pub struct InvertedNoises {
    /// Inverted noise values [stage][block]
    pub noises: Vec<Vec<f64>>,
    /// Warnings generated during inversion
    pub warnings: Vec<InversionWarning>,
}

#[derive(Debug)]
pub enum InversionWarning {
    ExtremeNoise { hydro: usize, stage: usize, noise: f64 },
    InconsistentBlockNoises { block: usize, stage: usize, max_diff: f64 },
}

#[derive(Debug)]
pub enum InversionError {
    ZeroVarianceResidual { hydro: usize, stage: usize, residual: f64 },
}
```

10.5.4 Validation Report

After inversion, POWE.RS generates a validation report:

```
{
  "scenario_id": 42,
  "n_stages": 120,
  "n_hydros": 160,
  "inversion_stats": {
    "mean_noise": 0.023,
    "std_noise": 1.12,
    "max_noise": 3.87,
    "min_noise": -3.42,
    "extreme_count": 0
  },
  "warnings": [],
  "status": "valid"
}
```

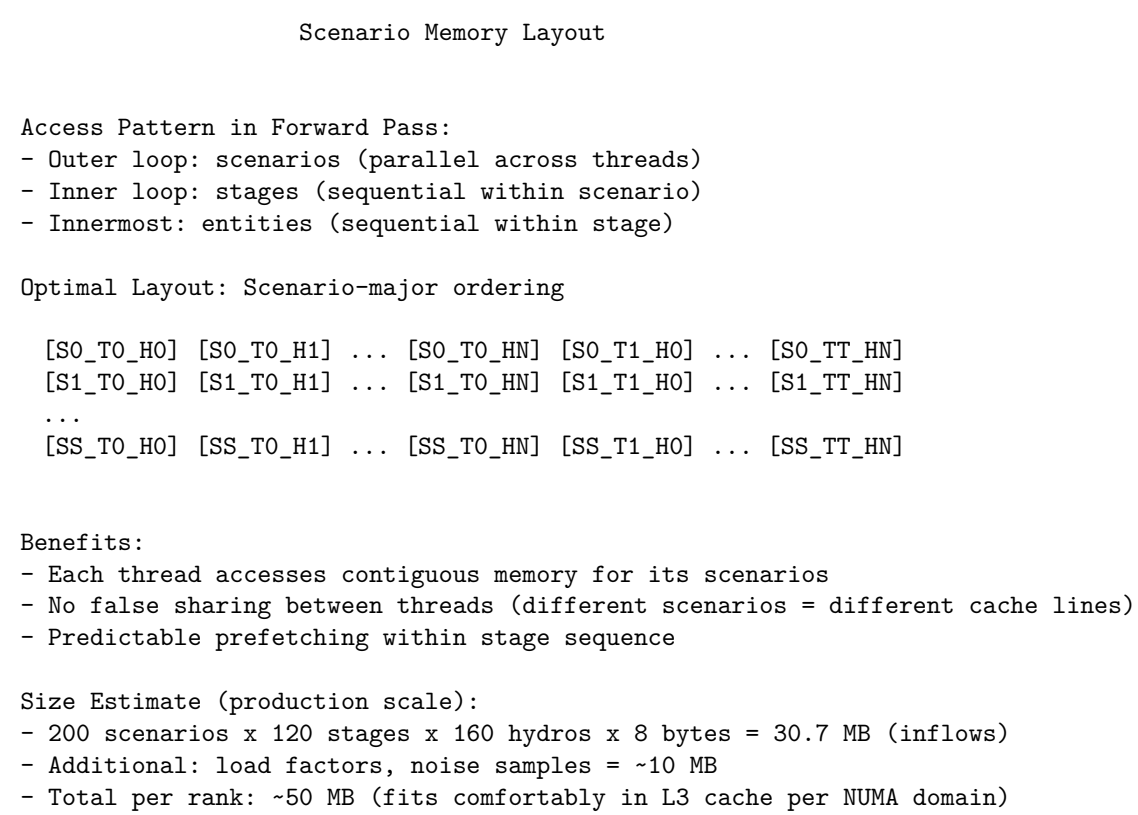
Critical: If AR order mismatches between PAR model and policy, noise inversion will produce incorrect values. This is checked during policy loading (see Section 25.5).

---

11. Scenario Memory Layout

11.1 Memory Organization

Scenario data is organized for optimal cache access patterns during forward pass:



11.2 Per-Rank Scenario Distribution

```rust
/// Scenario distribution across MPI ranks
pub struct ScenarioDistribution {
    /// Total scenarios in the problem
    pub total_scenarios: usize,

    /// This rank's assigned scenarios [start, end)
    pub my_range: Range<usize>,

    /// Scenarios per rank (for gather operations)
    pub counts: Vec<usize>,

    /// Displacements for MPI_Gatherv
    pub displs: Vec<usize>,
}

impl ScenarioDistribution {
    pub fn new(total_scenarios: usize, rank: usize, world_size: usize) -> Self {
        // Distribute scenarios as evenly as possible
        let base = total_scenarios / world_size;
        let remainder = total_scenarios % world_size;

        let mut counts = vec![base; world_size];
        for i in 0..remainder {
            counts[i] += 1;
        }

        let mut displs = vec![0; world_size];
        for i in 1..world_size {
            displs[i] = displs[i-1] + counts[i-1];
        }

        let start = displs[rank];
        let end = start + counts[rank];

        Self {
            total_scenarios,
            my_range: start..end,
            counts,
            displs,
        }
    }

    /// Number of scenarios this rank handles
    pub fn my_count(&self) -> usize {
        self.my_range.len()
    }

    /// Convert local scenario index to global
    pub fn to_global(&self, local: usize) -> usize {
        self.my_range.start + local
    }
}
```

11.3 NUMA-Aware Allocation

```rust
/// NUMA-aware scenario data allocation
pub fn allocate_scenario_data(
    distribution: &ScenarioDistribution,
    n_stages: usize,
    n_hydros: usize,
) -> Vec<f64> {
    let size = distribution.my_count() * n_stages * n_hydros;
    let mut data = vec![0.0; size];

    // First-touch initialization in parallel ensures NUMA locality
    #[cfg(feature = "openmp")]
    {
        use rayon::prelude::*;
        data.par_chunks_mut(n_stages * n_hydros)
            .for_each(|chunk| {
                // Touch each scenario's data from the thread that will use it
                chunk.fill(0.0);
            });
    }

    data
}
```

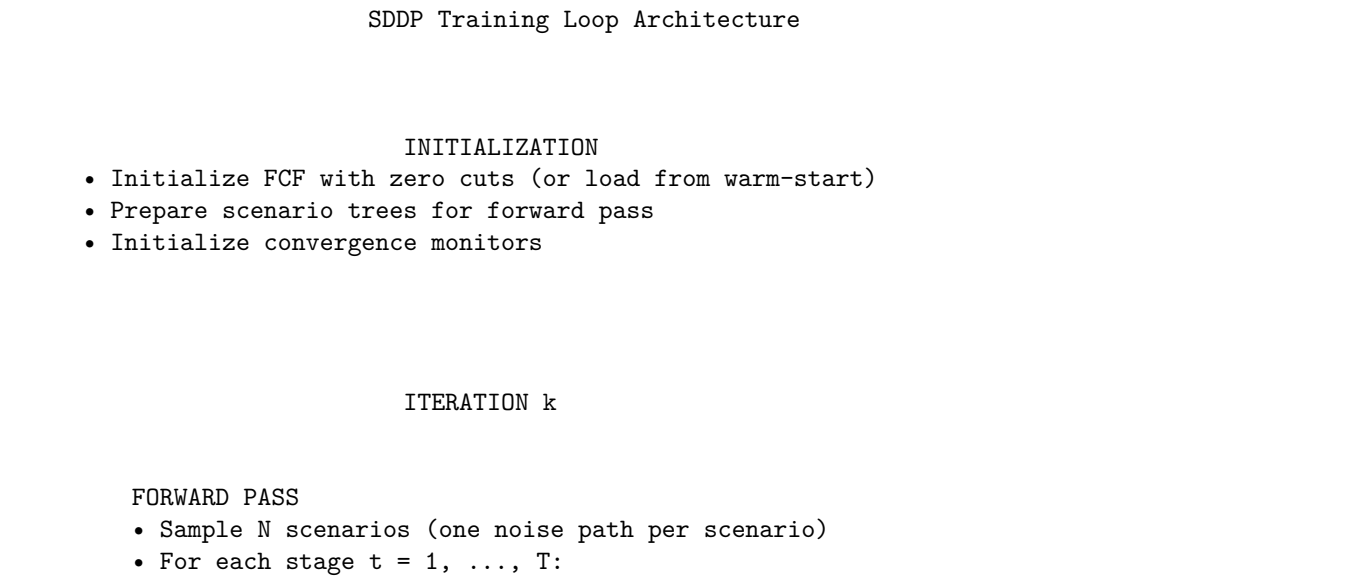
Part IV: Training Architecture

---

12. Training Loop Structure

12.1 SDDP Algorithm Overview

The training phase implements the Stochastic Dual Dynamic Programming (SDDP) algorithm, iteratively constructing piecewise-linear approximations of the expected future cost function (FCF) through forward simulation and backward cut generation.



```
    - Solve LP for each scenario with current FCF
    - Record state variables (storage, inflows) for backward pass
  • Compute statistical lower bound from stage-1 costs




BACKWARD PASS
• For each stage t = T, ..., 2:
  - For each state visited in forward pass:
    · For each noise outcome  :
      - Solve LP, extract duals ( _storage,  _inflow)
    · Compute cut coefficients via risk measure
    · Add cut to FCF_{t-1}




CONVERGENCE CHECK
• Update upper bound estimate (statistical bound)
• Check gap: (UB - LB) / |UB| < tolerance
• Check iteration limit
• Check time limit




                    [converged?]        OUTPUT: FCF cuts, bounds
                              No
                                    Next iteration k+1
```

12.2 Core Training Structures

```rust
/// Main training orchestrator
pub struct TrainingLoop<R: RiskMeasure, C: CutFormulation, H: HorizonMode> {
    // Algorithm components
    risk_measure: R,
    cut_formulation: C,
    horizon_mode: H,

    // State
    fcf: FutureCostFunction,
    iteration: usize,
    convergence_monitor: ConvergenceMonitor,

    // Configuration
    config: TrainingConfig,

    // MPI context
    comm: WorldCommunicator,
}

/// Training configuration from config.json
pub struct TrainingConfig {
    // Iteration limits
```

```rust
    pub max_iterations: usize,           // e.g., 1000
    pub min_iterations: usize,           // e.g., 10
    pub time_limit_seconds: Option<f64>, // e.g., 3600.0

    // Convergence criteria
    pub gap_tolerance: f64,              // e.g., 0.01 (1%)
    pub stable_iterations: usize,        // e.g., 5

    // Scenario sampling
    pub forward_scenarios: usize,        // e.g., 100
    pub backward_samples: usize,         // e.g., 50 (noise outcomes per state)

    // Cut management
    pub cut_selection: CutSelectionStrategy,
    pub max_cuts_per_stage: Option<usize>,

    // Checkpointing
    pub checkpoint_interval: usize,      // e.g., 10 (iterations)
}

impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Execute the SDDP training loop
    pub fn run(&mut self, case_data: &CaseData) -> TrainingResult {
        let start_time = Instant::now();

        while !self.should_stop(start_time) {
            self.iteration += 1;

            // Forward pass: simulate scenarios, compute lower bound
            let forward_result = self.forward_pass(case_data);

            // Synchronize forward results across ranks
            let global_forward = self.sync_forward_results(&forward_result);

            // Backward pass: generate cuts from visited states
            self.backward_pass(case_data, &global_forward);

            // Synchronize new cuts across ranks
            self.sync_cuts();

            // Update convergence statistics
            self.convergence_monitor.update(&global_forward, &self.fcf);

            // Checkpoint if needed
            if self.iteration % self.config.checkpoint_interval == 0 {
                self.checkpoint(case_data);
            }

            // Log progress
            self.log_iteration();
        }

        self.build_result(start_time)
    }

    fn should_stop(&self, start_time: Instant) -> bool {
        // Check iteration limits
```

```
        if self.iteration >= self.config.max_iterations {
            return true;
        }

        // Check time limit
        if let Some(limit) = self.config.time_limit_seconds {
            if start_time.elapsed().as_secs_f64() >= limit {
                return true;
            }
        }

        // Check convergence (only after min_iterations)
        if self.iteration >= self.config.min_iterations {
            if self.convergence_monitor.is_converged(&self.config) {
                return true;
            }
        }

        false
    }
}
```

12.3 Trait Abstractions

SDDP variants are expressed through trait abstractions:

```
/// Risk measure determines how cuts are computed from noise outcomes
pub trait RiskMeasure: Send + Sync {
    /// Compute cut coefficients from backward pass duals
    /// Returns (intercept_rhs, gradient_coefficients)
    fn compute_cut(
        &self,
        stage: StageId,
        state: &StatePoint,
        outcomes: &[BackwardOutcome],
        probabilities: &[f64],
    ) -> CutCoefficients;

    /// Name for logging
    fn name(&self) -> &'static str;
}

/// Cut formulation determines the structure of cuts
pub trait CutFormulation: Send + Sync {
    /// Build the cut constraint to add to stage t LP
    fn build_cut_constraint(
        &self,
        cut: &Cut,
        stage_vars: &StageVariables,
    ) -> LinearConstraint;
}

/// Horizon mode determines stage transitions and terminal conditions
pub trait HorizonMode: Send + Sync {
    /// Get successor stage(s) with transition probabilities
    fn successors(&self, stage: StageId) -> Vec<(StageId, f64)>;

    /// Is this the final stage? (terminal value function applies)
```

```rust
    fn is_terminal(&self, stage: StageId) -> bool;

    /// Discount factor for infinite horizon
    fn discount_factor(&self) -> f64;
}
```

---

13. Forward Pass Execution

13.1 Forward Pass Overview

The forward pass simulates multiple scenarios through the horizon, solving the LP at each stage with the current FCF approximation. Its purposes are:

1. **Compute lower bound**: Stage-1 objective values provide a statistical lower bound
2. **Collect states**: Record visited states for backward pass cut generation
3. **Quality assessment**: Estimate policy cost for convergence monitoring

```
                        Forward Pass Execution Flow


  Input: N_fwd scenarios to simulate


    SCENARIO DISTRIBUTION

    Rank 0: scenarios [0, 1, ..., N_fwd/nranks - 1]
    Rank 1: scenarios [N_fwd/nranks, ..., 2*N_fwd/nranks - 1]
    ...


  For each scenario s in my_scenarios:


    state = initial_state
    noise_path = sample_noise_path(s)   // Pre-sampled or on-demand

    For stage t = 1 to T:

      1. Update state with stage uncertainty (inflow realization)
         state.inflows = PAR_model(state.prev_inflows, noise_path[t])

      2. Build LP for stage t:
         - Decision variables (generation, storage, flows)
         - Constraints (balance, bounds, generic)
         - Objective: immediate_cost +   (future cost proxy)
         - FCF cuts on   from previous iterations

      3. Solve LP

      4. Record trial point:
         - Storage levels (end-of-stage)
         - Inflow history (for PAR state)
         - Stage cost

      5. Transition: state = extract_end_state(solution)
```

```
    Store scenario result: costs[], states[]

  Output: ForwardResult { scenario_costs, visited_states, lower_bound }
```

13.2 Forward Pass Implementation

```rust
/// Result from a single forward scenario
pub struct ScenarioTrajectory {
    pub scenario_id: ScenarioId,
    pub total_cost: f64,
    pub stage_costs: Vec<f64>,
    pub visited_states: Vec<StatePoint>,  // State at end of each stage
}

/// State vector used for cut generation
pub struct StatePoint {
    pub storage: Vec<f64>,          // Storage level per hydro
    pub inflow_history: Vec<Vec<f64>>, // Lags for PAR model [hydro][lag]
}

impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Execute forward pass for this rank's scenarios
    pub fn forward_pass(&self, case_data: &CaseData) -> ForwardResult {
        let my_scenarios = self.distribute_scenarios();

        // Parallel forward simulation (OpenMP threads)
        let trajectories: Vec<ScenarioTrajectory> = my_scenarios
            .into_par_iter()
            .map(|scenario_id| self.simulate_scenario(case_data, scenario_id))
            .collect();

        // Compute local lower bound estimate (mean of stage-1 costs)
        let local_lb = trajectories.iter()
            .map(|t| t.stage_costs[0])
            .sum::<f64>() / trajectories.len() as f64;

        ForwardResult {
            trajectories,
            local_lower_bound: local_lb,
        }
    }

    fn simulate_scenario(
        &self,
        case_data: &CaseData,
        scenario_id: ScenarioId,
    ) -> ScenarioTrajectory {
        let mut state = case_data.initial_state();
        let mut stage_costs = Vec::with_capacity(case_data.num_stages());
        let mut visited_states = Vec::with_capacity(case_data.num_stages());
        let noise_path = self.sample_noise_path(scenario_id);

        for (stage_idx, stage) in case_data.stages.iter().enumerate() {
```

```
            // Update inflows from PAR model
            state.update_inflows(&case_data.par_models, &noise_path[stage_idx]);

            // Build and solve stage LP
            let lp = self.build_stage_lp(case_data, stage, &state);
            let solution = lp.solve().expect("LP should be feasible");

            // Record results
            stage_costs.push(solution.immediate_cost);
            visited_states.push(state.clone());

            // Transition to next state
            state = solution.extract_end_state();
        }

        ScenarioTrajectory {
            scenario_id,
            total_cost: stage_costs.iter().sum(),
            stage_costs,
            visited_states,
        }
    }
}
```

13.3 State Management

The state vector contains all information needed to determine the optimal policy from a given point:

```
impl StatePoint {
    /// Create state from initial conditions
    pub fn from_initial(case_data: &CaseData) -> Self {
        let storage: Vec<f64> = case_data.hydros.iter()
            .map(|h| h.initial_storage)
            .collect();

        // Initialize inflow history from historical data
        let max_lag = case_data.par_models.max_order();
        let inflow_history: Vec<Vec<f64>> = case_data.hydros.iter()
            .map(|h| {
                case_data.inflow_history
                    .get_lags(h.id, max_lag)
                    .to_vec()
            })
            .collect();

        Self { storage, inflow_history }
    }

    /// Update inflows using PAR model
    pub fn update_inflows(&mut self, par_models: &ParModels, noise: &NoiseVector) {
        for (h_idx, model) in par_models.models.iter().enumerate() {
            let new_inflow = model.sample(
                &self.inflow_history[h_idx],
                noise[h_idx],
            );

            // Shift history and prepend new value
            self.inflow_history[h_idx].pop();
```

```rust
            self.inflow_history[h_idx].insert(0, new_inflow);
        }
    }

    /// Extract end-of-stage state from LP solution
    pub fn from_solution(solution: &LpSolution, hydro_ids: &[HydroId]) -> Self {
        Self {
            storage: hydro_ids.iter()
                .map(|id| solution.get_storage(*id))
                .collect(),
            inflow_history: solution.inflow_history.clone(),
        }
    }
}
```

13.4 Parallel Forward Execution

```
                    Forward Pass Parallel Execution Pattern


  Configuration: 8 MPI ranks × 24 OpenMP threads = 192 cores
  Forward scenarios: 200


    Rank 0
    Scenarios: 0-24 (25 scenarios)

      Thread 0: scenario 0    Thread 1: scenario 1    ...
      Thread 0: scenario 24   (when thread finishes earlier)




    Rank 1
    Scenarios: 25-49 (25 scenarios)
    [Same thread-level parallelism]


  ... (Ranks 2-7 similar)


  MPI_Barrier + Allreduce for global statistics
```

---

14. Backward Pass Execution

14.1 Backward Pass Overview

The backward pass constructs cuts by computing subgradients of the expected future cost function. Starting from the final stage and working backwards, it evaluates the cost-to-go from each visited state under multiple noise realizations.

```
                        Backward Pass Execution Flow
```

```
Input: Visited states from forward pass, FCF cuts, noise outcomes

For stage t = T, T-1, ..., 2:

  For each state s visited at stage t-1:

    outcomes = []

    For each noise outcome   with probability p_ :

      1. Compute realized inflows: a = PAR(s.history,  )

      2. Build LP for stage t with:
         - Initial storage = s.storage (fixed RHS)
         - Inflows = a (fixed RHS)
         - FCF cuts for  _{t+1}

      3. Solve LP, extract:
         - Objective value Q_t^ (s)
         - Dual  _storage (on initial storage constraint)
         - Dual  _inflow (on inflow constraint)

      4. outcomes.push(Q,  _storage,  _inflow, p_ )

    5. Compute cut via risk measure:
       cut = risk_measure.compute_cut(s, outcomes)

    6. Add cut to FCF_{t-1}

Output: New cuts added to FCF
```

14.2 Backward Pass Implementation

```rust
/// Result from solving backward LP at one state-outcome pair
pub struct BackwardOutcome {
    pub noise_index: usize,
    pub probability: f64,
    pub objective: f64,          // Q_t^(x)
    pub dual_storage: Vec<f64>,  //  for storage constraints
    pub dual_inflow: Vec<f64>,   //  for inflow constraints
}

impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Execute backward pass to generate cuts
    pub fn backward_pass(&mut self, case_data: &CaseData, forward: &GlobalForwardResult) {
        // Process stages in reverse order (T down to 2)
        for stage_idx in (1..case_data.num_stages()).rev() {
```

```
        let stage = &case_data.stages[stage_idx];
        let prev_stage = &case_data.stages[stage_idx - 1];

        // Get unique states visited at stage-1 (deduplicated across scenarios)
        let visited_states = self.collect_visited_states(forward, stage_idx - 1);

        // Distribute states across ranks for parallel processing
        let my_states = self.distribute_states(&visited_states);

        // Generate cuts for each state (parallel across threads)
        let new_cuts: Vec<Cut> = my_states
            .into_par_iter()
            .map(|state| self.generate_cut(case_data, stage, &state))
            .collect();

        // Collect cuts from all ranks
        let all_cuts = self.allgather_cuts(&new_cuts);

        // Add cuts to FCF for previous stage
        for cut in all_cuts {
            self.fcf.add_cut(prev_stage.id, cut);
        }
    }
}

fn generate_cut(
    &self,
    case_data: &CaseData,
    stage: &Stage,
    state: &StatePoint,
) -> Cut {
    // Sample noise outcomes for backward evaluation
    let noise_outcomes = self.sample_backward_noise();

    // Evaluate LP for each noise outcome (parallel across outcomes)
    let outcomes: Vec<BackwardOutcome> = noise_outcomes
        .iter()
        .map(|(noise, prob)| {
            // Compute realized inflows
            let inflows = case_data.par_models.realize(
                &state.inflow_history,
                noise,
            );

            // Build backward LP with fixed initial state
            let lp = self.build_backward_lp(case_data, stage, state, &inflows);

            // Solve and extract duals
            let solution = lp.solve().expect("Backward LP should be feasible");

            BackwardOutcome {
                noise_index: 0, // For tracking
                probability: *prob,
                objective: solution.objective,
                dual_storage: solution.get_duals("storage"),
                dual_inflow: solution.get_duals("inflow"),
            }
```

```
            })
            .collect();

        // Compute cut via risk measure
        let probabilities: Vec<f64> = outcomes.iter().map(|o| o.probability).collect();
        self.risk_measure.compute_cut(stage.id, state, &outcomes, &probabilities)
    }
}
```

14.3 Dual Extraction for Cut Coefficients

The cut coefficients are derived from LP duality. For a stage-t LP:

$$Q_t(x_{t-1}, \omega) = \min_{x_t} \{c_t^\top x_t + \theta_{t+1} : Ax_t \geq b_t(x_{t-1}, \omega)\}$$

The cut for stage $t-1$ is:

$$\theta_t \geq \pi^\top b_t(x_{t-1}, \omega) - \text{const}$$

where π are the dual variables. Since b_t depends linearly on the state: - Storage: $v_{h,t} = v_{h,t-1} + \text{inflow} - \text{outflow}$ - Inflows: From PAR model with state-dependent history

```
/// Cut structure for FCF
pub struct Cut {
    pub stage: StageId,          // Stage this cut applies to
    pub intercept: f64,          // RHS constant
    pub storage_coef: Vec<f64>,  // Coefficient per hydro storage
    pub inflow_coef: Vec<f64>,   // Coefficient per hydro inflow state
    pub iteration: usize,        // Iteration when cut was created
    pub active_count: usize,     // Times cut was binding (for selection)
}

impl Cut {
    /// Evaluate cut at a given state
    pub fn evaluate(&self, state: &StatePoint) -> f64 {
        let storage_term: f64 = self.storage_coef.iter()
            .zip(state.storage.iter())
            .map(|(c, v)| c * v)
            .sum();

        let inflow_term: f64 = self.inflow_coef.iter()
            .zip(state.inflow_history.iter().map(|h| h[0]))
            .map(|(c, a)| c * a)
            .sum();

        self.intercept + storage_term + inflow_term
    }

    /// Build LP constraint:  >= intercept + Σ c_v * v + Σ c_a * a
    pub fn to_constraint(&self, theta_var: VarId, state_vars: &StateVariables) -> Constraint {
        let mut coeffs = vec![(theta_var, 1.0)];  //

        for (i, &coef) in self.storage_coef.iter().enumerate() {
            coeffs.push((state_vars.storage[i], -coef));
        }
```

```
        for (i, &coef) in self.inflow_coef.iter().enumerate() {
            coeffs.push((state_vars.inflow[i], -coef));
        }

        Constraint {
            coeffs,
            sense: ConstraintSense::Ge,
            rhs: self.intercept,
        }
    }
}
```

14.4 Parallel Backward Execution

```
                    Backward Pass Parallel Execution Pattern


  Stage t: 50 unique states visited, 100 noise outcomes per state
  Total LP solves: 50 × 100 = 5,000

  Strategy: Distribute states across ranks, noise outcomes across threads


    Rank 0: States 0-6 (7 states)

      State 0:
        Thread 0: outcomes 0-4    (5 LPs)
        Thread 1: outcomes 5-9    (5 LPs)
        ...
        Thread 23: outcomes 95-99 (5 LPs)
      [Barrier: aggregate outcomes → compute cut]

      State 1: [same pattern]
      ...




    Rank 1: States 7-13 (7 states)
    [Same thread-level pattern]


  ... (Ranks 2-7 similar)


  MPI_Allgatherv: Collect all cuts from all ranks
```

---

15. Cut Management and Storage

15.1 Future Cost Function Structure

The FCF is stored as a collection of cuts per stage:

```rust
/// Future Cost Function: piecewise-linear approximation of cost-to-go
pub struct FutureCostFunction {
    /// Cuts indexed by stage
    cuts_by_stage: Vec<CutPool>,

    /// State dimension information
    n_storage_vars: usize,
    n_inflow_vars: usize,

    /// Global statistics
    total_cuts: usize,
    cuts_added_this_iteration: usize,
}

/// Pool of cuts for a single stage
pub struct CutPool {
    stage: StageId,
    cuts: Vec<Cut>,
    selection_strategy: CutSelectionStrategy,
    max_cuts: Option<usize>,
}

impl FutureCostFunction {
    /// Create empty FCF
    pub fn new(n_stages: usize, n_storage: usize, n_inflow: usize) -> Self {
        let cuts_by_stage = (0..n_stages)
            .map(|s| CutPool::new(StageId(s)))
            .collect();

        Self {
            cuts_by_stage,
            n_storage_vars: n_storage,
            n_inflow_vars: n_inflow,
            total_cuts: 0,
            cuts_added_this_iteration: 0,
        }
    }

    /// Add a cut to the specified stage
    pub fn add_cut(&mut self, stage: StageId, cut: Cut) {
        self.cuts_by_stage[stage.0].add(cut);
        self.total_cuts += 1;
        self.cuts_added_this_iteration += 1;
    }

    /// Get cuts for a stage (for LP construction)
    pub fn get_cuts(&self, stage: StageId) -> &[Cut] {
        self.cuts_by_stage[stage.0].active_cuts()
    }

    /// Evaluate lower bound on  at a state
    pub fn evaluate(&self, stage: StageId, state: &StatePoint) -> f64 {
        self.cuts_by_stage[stage.0]
            .cuts
            .iter()
            .map(|cut| cut.evaluate(state))
            .fold(f64::NEG_INFINITY, f64::max)
```

```
    }
}
```

15.2 Cut Selection Strategies

As iterations progress, the number of cuts can become unwieldy. Cut selection strategies maintain tractability:

```
pub enum CutSelectionStrategy {
    /// Keep all cuts (no selection)
    KeepAll,

    /// Keep most recently generated cuts
    MostRecent { max_cuts: usize },

    /// Keep cuts that were binding most often
    MostActive {
        max_cuts: usize,
        decay_factor: f64,  // Weight recent activity more
    },

    /// Level-1 cuts: statistical significance test
    Level1 {
        max_cuts: usize,
        significance: f64,  // e.g., 0.05
    },

    /// Hybrid: keep recent + most active
    Hybrid {
        recent_fraction: f64,  // e.g., 0.3
        max_cuts: usize,
    },
}

impl CutPool {
    /// Add cut and potentially prune
    pub fn add(&mut self, cut: Cut) {
        self.cuts.push(cut);
        self.maybe_prune();
    }

    /// Prune cuts based on selection strategy
    fn maybe_prune(&mut self) {
        if let Some(max) = self.max_cuts {
            if self.cuts.len() > max {
                match &self.selection_strategy {
                    CutSelectionStrategy::MostRecent { .. } => {
                        // Keep newest cuts
                        let drain_count = self.cuts.len() - max;
                        self.cuts.drain(0..drain_count);
                    }

                    CutSelectionStrategy::MostActive { decay_factor, .. } => {
                        // Sort by weighted activity, keep top
                        self.cuts.sort_by(|a, b| {
                            let score_a = a.weighted_activity(*decay_factor);
                            let score_b = b.weighted_activity(*decay_factor);
                            score_b.partial_cmp(&score_a).unwrap()
                        });
```

```
                    self.cuts.truncate(max);
                }

                CutSelectionStrategy::Hybrid { recent_fraction, .. } => {
                    let n_recent = (max as f64 * recent_fraction) as usize;
                    let n_active = max - n_recent;

                    // Partition: recent (by iteration) vs active (by binding count)
                    self.cuts.sort_by_key(|c| std::cmp::Reverse(c.iteration));
                    let recent: Vec<_> = self.cuts.drain(0..n_recent).collect();

                    self.cuts.sort_by_key(|c| std::cmp::Reverse(c.active_count));
                    self.cuts.truncate(n_active);

                    self.cuts.extend(recent);
                }

                _ => {}
            }
        }
    }

    /// Update activity counters after LP solve
    pub fn update_activity(&mut self, binding_cuts: &[usize]) {
        for &idx in binding_cuts {
            if idx < self.cuts.len() {
                self.cuts[idx].active_count += 1;
            }
        }
    }

    /// Get cuts for LP (may return subset based on strategy)
    pub fn active_cuts(&self) -> &[Cut] {
        &self.cuts
    }
}
```

15.3 Cut Serialization for Checkpoints

```
/// Binary format for cut storage (efficient I/O)
impl Cut {
    /// Serialize to bytes
    pub fn to_bytes(&self) -> Vec<u8> {
        let mut buf = Vec::with_capacity(
            8 +  // stage (u64)
            8 +  // intercept (f64)
            8 +  // iteration (u64)
            8 +  // active_count (u64)
            8 +  // n_storage (u64)
            8 +  // n_inflow (u64)
            self.storage_coef.len() * 8 +
            self.inflow_coef.len() * 8
        );

        buf.extend(&(self.stage.0 as u64).to_le_bytes());
        buf.extend(&self.intercept.to_le_bytes());
```

```
        buf.extend(&(self.iteration as u64).to_le_bytes());
        buf.extend(&(self.active_count as u64).to_le_bytes());
        buf.extend(&(self.storage_coef.len() as u64).to_le_bytes());
        buf.extend(&(self.inflow_coef.len() as u64).to_le_bytes());

        for &c in &self.storage_coef {
            buf.extend(&c.to_le_bytes());
        }
        for &c in &self.inflow_coef {
            buf.extend(&c.to_le_bytes());
        }

        buf
    }

    /// Deserialize from bytes
    pub fn from_bytes(bytes: &[u8]) -> Self {
        let mut offset = 0;

        let stage = StageId(read_u64(bytes, &mut offset) as usize);
        let intercept = read_f64(bytes, &mut offset);
        let iteration = read_u64(bytes, &mut offset) as usize;
        let active_count = read_u64(bytes, &mut offset) as usize;
        let n_storage = read_u64(bytes, &mut offset) as usize;
        let n_inflow = read_u64(bytes, &mut offset) as usize;

        let storage_coef = (0..n_storage)
            .map(|_| read_f64(bytes, &mut offset))
            .collect();
        let inflow_coef = (0..n_inflow)
            .map(|_| read_f64(bytes, &mut offset))
            .collect();

        Self {
            stage,
            intercept,
            storage_coef,
            inflow_coef,
            iteration,
            active_count,
        }
    }
}

/// Stage cuts file format
/// Header: magic (8 bytes) + version (4 bytes) + n_cuts (4 bytes)
/// Body: [cut_size (4 bytes) + cut_bytes]...
pub fn save_stage_cuts(path: &Path, cuts: &[Cut]) -> io::Result<()> {
    let mut file = BufWriter::new(File::create(path)?);

    // Header
    file.write_all(b"PWRSCUTS")?;  // Magic
    file.write_all(&1u32.to_le_bytes())?;  // Version
    file.write_all(&(cuts.len() as u32).to_le_bytes())?;  // Count

    // Cuts
    for cut in cuts {
```

```
        let bytes = cut.to_bytes();
        file.write_all(&(bytes.len() as u32).to_le_bytes())?;
        file.write_all(&bytes)?;
    }

    Ok(())
}
```

15.4 Cut Synchronization Across Ranks

```
impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Synchronize newly generated cuts across all ranks
    fn sync_cuts(&mut self) {
        // Serialize local new cuts
        let local_cuts: Vec<u8> = self.fcf.drain_new_cuts()
            .into_iter()
            .flat_map(|c| c.to_bytes())
            .collect();

        // Gather sizes from all ranks
        let local_size = local_cuts.len() as i32;
        let mut sizes = vec![0i32; self.comm.size() as usize];
        self.comm.all_gather(&local_size, &mut sizes);

        // Compute displacements
        let mut displs = vec![0i32; self.comm.size() as usize];
        for i in 1..displs.len() {
            displs[i] = displs[i-1] + sizes[i-1];
        }
        let total_size = displs.last().unwrap() + sizes.last().unwrap();

        // Gather all cuts
        let mut all_cuts_bytes = vec![0u8; total_size as usize];
        self.comm.all_gather_varcount(&local_cuts, &mut all_cuts_bytes, &sizes, &displs);

        // Deserialize and add to FCF (skipping own cuts, already added)
        let my_rank = self.comm.rank();
        for (rank, &size) in sizes.iter().enumerate() {
            if rank != my_rank as usize && size > 0 {
                let start = displs[rank] as usize;
                let end = start + size as usize;
                let cuts = deserialize_cuts(&all_cuts_bytes[start..end]);
                for cut in cuts {
                    self.fcf.add_cut(cut.stage, cut);
                }
            }
        }
    }
}
```

---

16. Convergence Monitoring

16.1 Convergence Criteria

SDDP convergence is determined by the gap between lower and upper bounds on the optimal objective:

Lower Bound (LB):

- Computed from stage-1 LP objective (includes θ_2)
- Deterministic: same value regardless of scenario
- Monotonically non-decreasing as cuts are added
- LB = min_x { $c_1^T x_1 + \theta_2$: constraints }

Upper Bound (UB):

- Statistical estimate from forward simulation costs
- UB_k = (1/N) Σ_i Σ_t cost(scenario i, stage t)
- Includes confidence interval: UB ± z_{α} × $\sigma/\sqrt{N}$
- Not monotonic (depends on sampled scenarios)

Convergence Gap:

gap = (UB - LB) / |UB|

Stopping Rules:

1. Gap tolerance: gap < ε (e.g., 1%)
2. Stable bound: LB unchanged for K iterations
3. Iteration limit: k $\geq$ max_iterations
4. Time limit: elapsed $\geq$ time_limit

16.2 Convergence Monitor Implementation

```rust
/// Tracks convergence statistics across iterations
pub struct ConvergenceMonitor {
    // Bound histories
    lower_bounds: Vec<f64>,
    upper_bounds: Vec<f64>,
    upper_bound_stds: Vec<f64>,

    // Gap history
    gaps: Vec<f64>,

    // Iteration timing
    iteration_times: Vec<Duration>,

    // Stability tracking
    stable_lb_count: usize,
    last_lb_change_iteration: usize,
}

impl ConvergenceMonitor {
    pub fn new() -> Self {
        Self {
            lower_bounds: Vec::new(),
            upper_bounds: Vec::new(),
            upper_bound_stds: Vec::new(),
            gaps: Vec::new(),
```

```
            iteration_times: Vec::new(),
            stable_lb_count: 0,
            last_lb_change_iteration: 0,
        }
    }

    /// Update with results from current iteration
    pub fn update(&mut self, forward: &GlobalForwardResult, fcf: &FutureCostFunction) {
        let iteration = self.lower_bounds.len();

        // Lower bound: stage-1 objective (deterministic)
        let lb = forward.lower_bound;

        // Upper bound: mean of scenario costs with std
        let ub = forward.mean_cost;
        let ub_std = forward.cost_std;

        // Check LB stability
        if let Some(&prev_lb) = self.lower_bounds.last() {
            if (lb - prev_lb).abs() < 1e-6 * prev_lb.abs().max(1.0) {
                self.stable_lb_count += 1;
            } else {
                self.stable_lb_count = 0;
                self.last_lb_change_iteration = iteration;
            }
        }

        // Compute gap
        let gap = if ub.abs() > 1e-10 {
            (ub - lb) / ub.abs()
        } else {
            0.0
        };

        // Record
        self.lower_bounds.push(lb);
        self.upper_bounds.push(ub);
        self.upper_bound_stds.push(ub_std);
        self.gaps.push(gap);
    }

    /// Check if converged based on configuration
    pub fn is_converged(&self, config: &TrainingConfig) -> bool {
        let iteration = self.lower_bounds.len();

        // Check gap tolerance
        if let Some(&gap) = self.gaps.last() {
            if gap < config.gap_tolerance {
                return true;
            }
        }

        // Check stable lower bound
        if self.stable_lb_count >= config.stable_iterations {
            return true;
        }
```

```
        false
    }

    /// Get current statistics for logging
    pub fn current_stats(&self) -> ConvergenceStats {
        ConvergenceStats {
            iteration: self.lower_bounds.len(),
            lower_bound: self.lower_bounds.last().copied().unwrap_or(0.0),
            upper_bound: self.upper_bounds.last().copied().unwrap_or(0.0),
            upper_bound_std: self.upper_bound_stds.last().copied().unwrap_or(0.0),
            gap: self.gaps.last().copied().unwrap_or(1.0),
            stable_iterations: self.stable_lb_count,
        }
    }
}

/// Statistics for logging/reporting
pub struct ConvergenceStats {
    pub iteration: usize,
    pub lower_bound: f64,
    pub upper_bound: f64,
    pub upper_bound_std: f64,
    pub gap: f64,
    pub stable_iterations: usize,
}

impl std::fmt::Display for ConvergenceStats {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(
            f,
            "Iter {:4} | LB: {:12.2} | UB: {:12.2} ± {:8.2} | Gap: {:6.2}%",
            self.iteration,
            self.lower_bound,
            self.upper_bound,
            self.upper_bound_std * 1.96,  // 95% CI
            self.gap * 100.0,
        )
    }
}
```

16.3 Bound Computation Details

```
/// Global forward result aggregated across all ranks
pub struct GlobalForwardResult {
    /// Lower bound (stage-1 objective, deterministic)
    pub lower_bound: f64,

    /// Mean cost across all scenarios
    pub mean_cost: f64,

    /// Standard deviation of scenario costs
    pub cost_std: f64,

    /// Number of scenarios
    pub n_scenarios: usize,

    /// 95% confidence interval half-width
```

```rust
    pub ci_95: f64,
}

impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Aggregate forward results from all ranks
    fn sync_forward_results(&self, local: &ForwardResult) -> GlobalForwardResult {
        // Gather local statistics
        let local_n = local.trajectories.len() as f64;
        let local_sum: f64 = local.trajectories.iter()
            .map(|t| t.total_cost)
            .sum();
        let local_sum_sq: f64 = local.trajectories.iter()
            .map(|t| t.total_cost.powi(2))
            .sum();

        // Reduce across ranks
        let mut global_n = 0.0;
        let mut global_sum = 0.0;
        let mut global_sum_sq = 0.0;

        self.comm.all_reduce(&local_n, &mut global_n, MpiOp::Sum);
        self.comm.all_reduce(&local_sum, &mut global_sum, MpiOp::Sum);
        self.comm.all_reduce(&local_sum_sq, &mut global_sum_sq, MpiOp::Sum);

        // Compute statistics
        let mean = global_sum / global_n;
        let variance = (global_sum_sq / global_n) - mean.powi(2);
        let std = variance.sqrt();
        let ci_95 = 1.96 * std / global_n.sqrt();

        // Lower bound: stage-1 objective from any scenario (deterministic)
        // All scenarios have same stage-1 LP, so use first
        let lb = if self.comm.rank() == 0 {
            local.trajectories.first()
                .map(|t| t.stage_costs[0])
                .unwrap_or(0.0)
        } else {
            0.0
        };
        let mut global_lb = 0.0;
        self.comm.broadcast(&lb, &mut global_lb, 0);

        GlobalForwardResult {
            lower_bound: global_lb,
            mean_cost: mean,
            cost_std: std,
            n_scenarios: global_n as usize,
            ci_95,
        }
    }
}
```

16.4 Convergence Logging

SDDP Training Log Format

```
POWE.RS SDDP Training
Case: Brazilian_Interconnected_System
Started: 2026-01-31 10:30:00
Ranks: 8 | Threads/rank: 24 | Stages: 120 | Hydros: 156


Iter    1 | LB:  1.23456e+09 | UB:  2.34567e+09 ± 1.23e+08 | Gap: 47.38%
Iter    2 | LB:  1.45678e+09 | UB:  2.12345e+09 ± 9.87e+07 | Gap: 31.44%
Iter    3 | LB:  1.56789e+09 | UB:  1.98765e+09 ± 8.76e+07 | Gap: 21.11%
...
Iter   47 | LB:  1.87654e+09 | UB:  1.89012e+09 ± 2.34e+07 | Gap:  0.72%


CONVERGED after 47 iterations (gap < 1.00%)
Total time: 23m 45s | Avg iteration: 30.3s
Final LB: 1.87654e+09 | Final UB: 1.89012e+09 ± 2.34e+07
Total cuts: 5,640 | Cuts/stage: ~47
```

Part V: Simulation Architecture

---

17. Policy Evaluation Mode

17.1 Simulation Overview

The simulation phase evaluates the trained SDDP policy on a large number of scenarios to assess: 1. **Policy quality**: Expected cost, variance, and risk metrics 2. **Operational behavior**: Storage trajectories, generation mix, deficit frequency 3. **Robustness**: Performance across diverse hydrological conditions

```
                        Simulation Architecture


Input: Trained FCF (cuts), Simulation scenarios


  SCENARIO GENERATION

  - Monte Carlo: Sample N_sim scenarios from PAR model
  - Historical: Replay historical inflow sequences
  - External: Load pre-generated scenarios from files
  - Hybrid: Historical + synthetic tails



  PARALLEL EXECUTION

  Rank 0: scenarios [0, N_sim/P)
  Rank 1: scenarios [N_sim/P, 2*N_sim/P)
  ...
```

```
Each rank: solve LP sequence for assigned scenarios



PER-SCENARIO EXECUTION

For stage t = 1 to T:
  1. Realize uncertainties (inflows from scenario)
  2. Build and solve stage LP with FCF cuts
  3. Stream results to output (no full storage needed)
  4. Handle non-convexities if enabled (MIP/heuristics)



OUTPUT AGGREGATION

- Streaming: Write per-scenario results as computed
- Statistics: Aggregate metrics across all scenarios
- Risk metrics: CVaR, VaR, percentiles
```

17.2 Simulation Configuration

```rust
/// Simulation configuration from config.json
pub struct SimulationConfig {
    /// Enable/disable simulation phase
    pub enabled: bool,

    /// Number of scenarios to simulate
    pub n_scenarios: usize,  // e.g., 2000

    /// Scenario source
    pub scenario_source: ScenarioSource,

    /// Output options
    pub output: SimulationOutputConfig,

    /// Non-convex extensions
    pub non_convex: Option<NonConvexConfig>,

    /// Parallel execution
    pub chunk_size: usize,  // Scenarios per work unit
}

pub enum ScenarioSource {
    /// Sample from PAR model
    MonteCarlo {
        seed: u64,
    },

    /// Use historical sequences
    Historical {
        start_year: u32,
        end_year: u32,
    },
```

```
    /// Load from external files
    External {
        path: PathBuf,
        format: ScenarioFormat,
    },

    /// Historical + synthetic extensions
    Hybrid {
        historical_weight: f64,
        synthetic_tail_years: u32,
    },
}

pub struct SimulationOutputConfig {
    /// Output detail level
    pub detail: OutputDetail,

    /// Streaming vs batched writing
    pub streaming: bool,

    /// Compression for large outputs
    pub compress: bool,

    /// Variables to include in output
    pub variables: Vec<OutputVariable>,
}

pub enum OutputDetail {
    /// Only aggregate statistics
    Summary,
    /// Per-stage aggregates
    StageLevel,
    /// Per-scenario, per-stage details
    Full,
}
```

17.3 Simulation Execution

```
/// Main simulation orchestrator
pub struct SimulationRunner {
    fcf: Arc<FutureCostFunction>,
    config: SimulationConfig,
    comm: WorldCommunicator,
    output_writer: OutputWriter,
}

impl SimulationRunner {
    pub fn run(&mut self, case_data: &CaseData) -> SimulationResult {
        // Generate or load scenarios
        let scenarios = self.prepare_scenarios(case_data);

        // Distribute scenarios across ranks
        let my_scenarios = self.distribute_scenarios(&scenarios);

        // Initialize aggregators
        let mut local_stats = SimulationStats::new();
```

```
    // Process scenarios in parallel (threads within rank)
    for chunk in my_scenarios.chunks(self.config.chunk_size) {
        let chunk_results: Vec<ScenarioResult> = chunk
            .par_iter()
            .map(|scenario| self.simulate_scenario(case_data, scenario))
            .collect();

        // Stream results to output
        for result in &chunk_results {
            self.output_writer.write_scenario(result);
        }

        // Update aggregates
        for result in chunk_results {
            local_stats.accumulate(&result);
        }
    }

    // Global aggregation
    let global_stats = self.aggregate_stats(&local_stats);

    SimulationResult {
        stats: global_stats,
        output_path: self.output_writer.finalize(),
    }
}

fn simulate_scenario(
    &self,
    case_data: &CaseData,
    scenario: &Scenario,
) -> ScenarioResult {
    let mut state = case_data.initial_state();
    let mut stage_results = Vec::with_capacity(case_data.num_stages());
    let mut total_cost = 0.0;

    for (stage_idx, stage) in case_data.stages.iter().enumerate() {
        // Apply scenario's realized inflows for this stage
        let inflows = scenario.inflows_at_stage(stage_idx);
        state.set_inflows(&inflows);

        // Build and solve stage LP
        let lp = self.build_simulation_lp(case_data, stage, &state);
        let solution = lp.solve().expect("Simulation LP should be feasible");

        // Handle non-convex extensions if configured
        let final_solution = if let Some(nc_config) = &self.config.non_convex {
            self.apply_non_convex(case_data, stage, &solution, nc_config)
        } else {
            solution
        };

        // Record results
        let stage_result = StageResult::from_solution(&final_solution, stage);
        total_cost += stage_result.cost;
        stage_results.push(stage_result);
```

```
            // Transition state
            state = final_solution.extract_end_state();
        }

        ScenarioResult {
            scenario_id: scenario.id,
            total_cost,
            stage_results,
        }
    }
}
```

17.4 Simulation Statistics

```
/// Aggregate statistics from simulation
pub struct SimulationStats {
    // Cost statistics
    pub n_scenarios: usize,
    pub total_cost_sum: f64,
    pub total_cost_sum_sq: f64,
    pub min_cost: f64,
    pub max_cost: f64,

    // Sorted costs for percentiles (kept on rank 0)
    costs: Vec<f64>,

    // Operational statistics
    pub deficit_scenarios: usize,
    pub deficit_mwh_sum: f64,
    pub spill_mwh_sum: f64,

    // Per-stage statistics (optional)
    stage_stats: Option<Vec<StageStats>>,
}

impl SimulationStats {
    pub fn accumulate(&mut self, result: &ScenarioResult) {
        self.n_scenarios += 1;
        self.total_cost_sum += result.total_cost;
        self.total_cost_sum_sq += result.total_cost.powi(2);
        self.min_cost = self.min_cost.min(result.total_cost);
        self.max_cost = self.max_cost.max(result.total_cost);
        self.costs.push(result.total_cost);

        // Check for deficit
        let has_deficit = result.stage_results.iter()
            .any(|s| s.deficit > 0.0);
        if has_deficit {
            self.deficit_scenarios += 1;
            self.deficit_mwh_sum += result.stage_results.iter()
                .map(|s| s.deficit)
                .sum::<f64>();
        }
    }

    pub fn mean_cost(&self) -> f64 {
```

```
        self.total_cost_sum / self.n_scenarios as f64
    }

    pub fn std_cost(&self) -> f64 {
        let mean = self.mean_cost();
        let var = (self.total_cost_sum_sq / self.n_scenarios as f64) - mean.powi(2);
        var.sqrt()
    }

    pub fn cvar(&self, alpha: f64) -> f64 {
        // Sort costs (already accumulated)
        let mut sorted = self.costs.clone();
        sorted.sort_by(|a, b| a.partial_cmp(b).unwrap());

        // CVaR_ = mean of worst (1- ) fraction
        let cutoff_idx = ((1.0 - alpha) * self.n_scenarios as f64) as usize;
        let tail = &sorted[cutoff_idx..];
        tail.iter().sum::<f64>() / tail.len() as f64
    }

    pub fn percentile(&self, p: f64) -> f64 {
        let mut sorted = self.costs.clone();
        sorted.sort_by(|a, b| a.partial_cmp(b).unwrap());
        let idx = (p * (self.n_scenarios - 1) as f64) as usize;
        sorted[idx]
    }
}
```

---

18. Non-Convex Extensions

18.1 Non-Convexity Sources

SDDP produces an optimal policy for the convex relaxation. Simulation can incorporate non-convex operational constraints through post-processing:

Non-Convex Extensions in Simulation

Non-Convexity Source	Modeling Approach
Thermal unit commitment	MIP with binary on/off variables
Minimum generation	Big-M constraints or indicator constraints
Startup/shutdown costs	Multi-period linking constraints
Transmission switching	Binary line switching variables
Head-dependent generation	Piecewise-linear or iterative refinement
Forbidden operating zones	Disjunctive constraints

Strategy: Solve LP for policy decisions, then refine with MIP/heuristics

18.2 Non-Convex Processing Pipeline

```
/// Configuration for non-convex extensions
pub struct NonConvexConfig {
    /// Enable thermal unit commitment
```

```rust
    pub thermal_commitment: Option<ThermalCommitmentConfig>,

    /// Enable transmission switching
    pub transmission_switching: Option<TransmissionSwitchingConfig>,

    /// Head-dependent generation refinement
    pub head_dependent: Option<HeadDependentConfig>,

    /// Solver settings for MIP
    pub mip_settings: MipSettings,
}

pub struct ThermalCommitmentConfig {
    /// Minimum up/down time enforcement
    pub min_updown_time: bool,

    /// Startup cost modeling
    pub startup_costs: bool,

    /// Time limit for MIP solve
    pub time_limit_seconds: f64,

    /// MIP gap tolerance
    pub gap_tolerance: f64,
}

impl SimulationRunner {
    /// Apply non-convex refinement to LP solution
    fn apply_non_convex(
        &self,
        case_data: &CaseData,
        stage: &Stage,
        lp_solution: &LpSolution,
        config: &NonConvexConfig,
    ) -> Solution {
        let mut refined = lp_solution.clone();

        // Thermal commitment refinement
        if let Some(tc_config) = &config.thermal_commitment {
            refined = self.refine_thermal_commitment(
                case_data, stage, &refined, tc_config
            );
        }

        // Head-dependent generation refinement
        if let Some(hd_config) = &config.head_dependent {
            refined = self.refine_head_dependent(
                case_data, stage, &refined, hd_config
            );
        }

        refined
    }

    fn refine_thermal_commitment(
        &self,
        case_data: &CaseData,
```

```
        stage: &Stage,
        lp_solution: &LpSolution,
        config: &ThermalCommitmentConfig,
    ) -> Solution {
        // Build MIP with commitment variables
        let mut mip = MipBuilder::new();

        for thermal in &case_data.thermals {
            let lp_gen = lp_solution.get_generation(thermal.id);

            // Binary commitment variable
            let commit = mip.add_binary(&format!("commit_{}", thermal.id));

            // Generation variable
            let gen = mip.add_continuous(
                &format!("gen_{}", thermal.id),
                0.0,
                thermal.max_generation,
            );

            // If committed, must be within operating range
            // gen >= min_gen * commit
            // gen <= max_gen * commit
            mip.add_constraint(
                gen - thermal.min_generation * commit >= 0.0
            );
            mip.add_constraint(
                gen - thermal.max_generation * commit <= 0.0
            );

            // Warm-start from LP solution
            if lp_gen > 0.0 {
                mip.set_start_value(commit, 1.0);
                mip.set_start_value(gen, lp_gen);
            }
        }

        // Objective: match LP dispatch as closely as possible
        // (minimize deviation from LP solution)
        mip.set_objective(/* deviation terms */);

        // Solve MIP
        let mip_solution = mip.solve_with_timeout(config.time_limit_seconds);

        // Construct refined solution
        Self::merge_mip_solution(lp_solution, &mip_solution)
    }
}
```

18.3 Iterative Head-Dependent Refinement

For hydropower with head-dependent generation:

```
/// Head-dependent generation uses storage level to compute efficiency
pub struct HeadDependentConfig {
    /// Maximum refinement iterations
    pub max_iterations: usize,
```

```rust
    /// Convergence tolerance for head
    pub head_tolerance: f64,

    /// Method for head approximation
    pub method: HeadDependentMethod,
}

pub enum HeadDependentMethod {
    /// Fixed-point iteration on average head
    FixedPoint,

    /// Use end-of-stage storage for head
    EndOfStage,

    /// Interpolate between start and end
    AverageStorage { weight: f64 },
}

impl SimulationRunner {
    fn refine_head_dependent(
        &self,
        case_data: &CaseData,
        stage: &Stage,
        initial_solution: &LpSolution,
        config: &HeadDependentConfig,
    ) -> Solution {
        let mut solution = initial_solution.clone();

        for iter in 0..config.max_iterations {
            // Compute head based on current storage
            let heads: Vec<f64> = case_data.hydros.iter()
                .map(|h| self.compute_head(h, &solution, config))
                .collect();

            // Update generation efficiency based on head
            let efficiencies: Vec<f64> = case_data.hydros.iter()
                .zip(heads.iter())
                .map(|(h, &head)| h.efficiency_at_head(head))
                .collect();

            // Re-solve LP with updated efficiencies
            let lp = self.build_lp_with_efficiencies(
                case_data, stage, &solution, &efficiencies
            );
            let new_solution = lp.solve().expect("LP should be feasible");

            // Check convergence
            let max_head_change = heads.iter()
                .zip(self.compute_heads(&new_solution, case_data, config))
                .map(|(&old, new)| (old - new).abs())
                .fold(0.0, f64::max);

            if max_head_change < config.head_tolerance {
                return new_solution;
            }

            solution = new_solution;
```

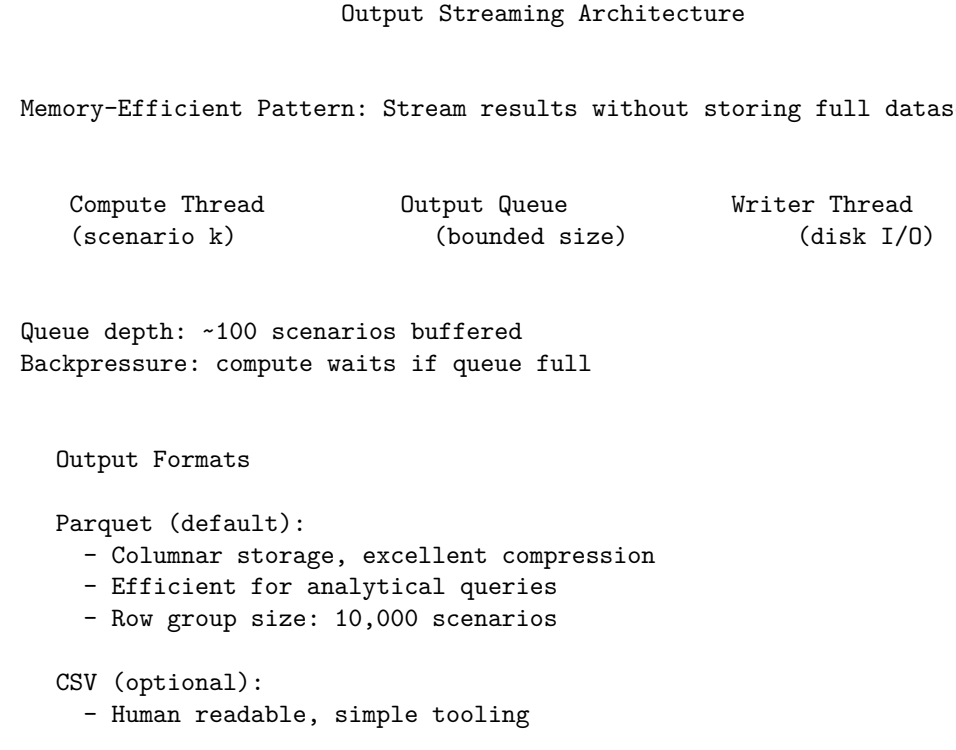
```
        }

        solution  // Return best found even if not converged
    }

    fn compute_head(
        &self,
        hydro: &Hydro,
        solution: &LpSolution,
        config: &HeadDependentConfig,
    ) -> f64 {
        let start_storage = solution.get_initial_storage(hydro.id);
        let end_storage = solution.get_storage(hydro.id);

        let storage = match config.method {
            HeadDependentMethod::FixedPoint => (start_storage + end_storage) / 2.0,
            HeadDependentMethod::EndOfStage => end_storage,
            HeadDependentMethod::AverageStorage { weight } => {
                weight * start_storage + (1.0 - weight) * end_storage
            }
        };

        hydro.head_from_storage(storage)
    }
}
```

---

19. Output Streaming

19.1 Streaming Architecture

With potentially thousands of scenarios, storing all results in memory is impractical. The output writer streams results to disk as they're computed:



```
    - One file per output variable

  Binary (compact):
    - Custom format for maximum throughput
    - Post-processing required for analysis
```

19.2 Output Writer Implementation

```rust
/// Streaming output writer with background I/O thread
pub struct OutputWriter {
    sender: Sender<OutputMessage>,
    writer_handle: Option<JoinHandle<()>>,
    config: SimulationOutputConfig,
}

enum OutputMessage {
    ScenarioResult(ScenarioResult),
    Flush,
    Finish,
}

impl OutputWriter {
    pub fn new(output_dir: &Path, config: SimulationOutputConfig) -> Self {
        let (sender, receiver) = bounded(100);  // Bounded channel

        let writer_handle = std::thread::spawn(move || {
            let mut writer = ParquetWriter::new(output_dir);

            loop {
                match receiver.recv() {
                    Ok(OutputMessage::ScenarioResult(result)) => {
                        writer.write_scenario(&result);
                    }
                    Ok(OutputMessage::Flush) => {
                        writer.flush();
                    }
                    Ok(OutputMessage::Finish) => {
                        writer.finalize();
                        break;
                    }
                    Err(_) => break,
                }
            }
        });

        Self {
            sender,
            writer_handle: Some(writer_handle),
            config,
        }
    }

    pub fn write_scenario(&self, result: &ScenarioResult) {
        // Filter based on output config
```

```
        let filtered = self.filter_output(result);

        // Send to writer thread (blocks if queue full)
        self.sender.send(OutputMessage::ScenarioResult(filtered))
            .expect("Writer thread should be alive");
    }

    pub fn finalize(mut self) -> PathBuf {
        self.sender.send(OutputMessage::Finish).ok();
        if let Some(handle) = self.writer_handle.take() {
            handle.join().expect("Writer thread should complete");
        }
        self.output_path()
    }

    fn filter_output(&self, result: &ScenarioResult) -> ScenarioResult {
        match self.config.detail {
            OutputDetail::Summary => {
                // Only keep total cost
                ScenarioResult {
                    scenario_id: result.scenario_id,
                    total_cost: result.total_cost,
                    stage_results: vec![],
                }
            }
            OutputDetail::StageLevel => {
                // Aggregate per stage, drop per-entity detail
                ScenarioResult {
                    scenario_id: result.scenario_id,
                    total_cost: result.total_cost,
                    stage_results: result.stage_results.iter()
                        .map(|s| s.aggregate())
                        .collect(),
                }
            }
            OutputDetail::Full => {
                // Keep everything requested
                result.filter_variables(&self.config.variables)
            }
        }
    }
}
```

19.3 Parquet Output Schema

```
/// Parquet schema for simulation results
///
/// File: results/scenario_results.parquet
///
/// Schema:
///   scenario_id: INT32
///   stage_id: INT32
///   total_cost: DOUBLE
///   immediate_cost: DOUBLE
///   future_cost: DOUBLE
///   deficit_mwh: DOUBLE
///   spill_mwh: DOUBLE
```

```rust
///
///   -- Per-hydro columns (if Full detail)
///   hydro_{id}_storage: DOUBLE
///   hydro_{id}_generation: DOUBLE
///   hydro_{id}_turbined: DOUBLE
///   hydro_{id}_spilled: DOUBLE
///
///   -- Per-thermal columns (if Full detail)
///   thermal_{id}_generation: DOUBLE
///   thermal_{id}_committed: BOOLEAN (if UC enabled)
///
///   -- Per-bus columns
///   bus_{id}_deficit: DOUBLE
///   bus_{id}_marginal_cost: DOUBLE

pub struct ParquetWriter {
    writer: SerializedFileWriter<File>,
    row_group_builder: RowGroupBuilder,
    rows_in_group: usize,
    row_group_size: usize,
}

impl ParquetWriter {
    pub fn new(output_dir: &Path, schema: &SchemaRef, row_group_size: usize) -> Self {
        let file = File::create(output_dir.join("scenario_results.parquet"))
            .expect("Could not create output file");

        let props = WriterProperties::builder()
            .set_compression(Compression::ZSTD(ZstdLevel::try_new(3).unwrap()))
            .set_dictionary_enabled(true)
            .build();

        let writer = SerializedFileWriter::new(file, schema.clone(), Arc::new(props))
            .expect("Could not create Parquet writer");

        Self {
            writer,
            row_group_builder: RowGroupBuilder::new(schema),
            rows_in_group: 0,
            row_group_size,
        }
    }

    pub fn write_scenario(&mut self, result: &ScenarioResult) {
        for (stage_idx, stage_result) in result.stage_results.iter().enumerate() {
            self.row_group_builder.append_row(
                result.scenario_id,
                stage_idx as i32,
                result.total_cost,
                stage_result,
            );
            self.rows_in_group += 1;

            if self.rows_in_group >= self.row_group_size {
                self.flush_row_group();
            }
        }
```

```
    }

    fn flush_row_group(&mut self) {
        if self.rows_in_group > 0 {
            let row_group = self.row_group_builder.build();
            self.writer.write_row_group(row_group)
                .expect("Could not write row group");
            self.rows_in_group = 0;
        }
    }

    pub fn finalize(mut self) {
        self.flush_row_group();
        self.writer.close().expect("Could not close Parquet file");
    }
}
```

19.4 Distributed Output Coordination

```
                    Distributed Output Pattern


  Option A: Each rank writes own partition (parallel, simple)


  Rank 0    results/scenarios_0000.parquet
  Rank 1    results/scenarios_0001.parquet
  ...
  Rank 7    results/scenarios_0007.parquet

  Post-processing: Combine or query across partitions



  Option B: Rank 0 collects and writes (sequential, single file)


  Ranks 1-7    MPI_Gatherv    Rank 0    results/all_scenarios.parquet

  Suitable for smaller simulation runs



  Option C: MPI-IO collective write (advanced, single file)


  All ranks    MPI_File_write_at_all    results/all_scenarios.bin

  Best for very large simulations on parallel filesystems
```

```
impl SimulationRunner {
    fn setup_output_writer(&self, case_data: &CaseData) -> OutputWriter {
        let output_dir = case_data.output_dir().join("simulation");
        std::fs::create_dir_all(&output_dir).expect("Could not create output dir");
```

```rust
        match self.config.output.distributed_mode {
            DistributedOutputMode::PerRank => {
                // Each rank writes to own file
                let rank_file = output_dir.join(
                    format!("scenarios_{:04}.parquet", self.comm.rank())
                );
                OutputWriter::new(&rank_file, self.config.output.clone())
            }

            DistributedOutputMode::Collected => {
                if self.comm.rank() == 0 {
                    // Only rank 0 writes
                    OutputWriter::new(
                        &output_dir.join("all_scenarios.parquet"),
                        self.config.output.clone()
                    )
                } else {
                    // Other ranks use sender to rank 0
                    OutputWriter::new_sender(0, &self.comm)
                }
            }
        }
    }
}
```

Part VII: Memory and I/O

---

24. Memory Architecture

24.1 Memory Budget Overview

Memory Architecture Overview

Target System: 768 GB RAM, 8 MPI ranks (96 GB per rank)
Reference Case: Brazilian System (156 hydros, 120 stages, 200 scenarios)

Per-Rank Memory Budget

Component	Size	Type
Case Data (replicated)	50 MB	Read-only, shared across thds
PAR Models	100 MB	Read-only
Correlation Matrices	20 MB	Read-only
FCF Cuts (growing)	500 MB	Read-mostly, synced
Scenario Data	2 GB	Per-rank subset
LP Solver Workspaces	4 GB	Per-thread × 24 threads
Solution Buffers	500 MB	Per-thread
Output Buffers	200 MB	Streaming queue
Overhead/Fragmentation	1 GB	Safety margin
TOTAL	~8.4 GB	(well within 96 GB)

```
Scaling: 10× problem size (1560 hydros) → ~30 GB per rank (still fits)
```

24.2 Memory Layout Strategy

```rust
/// Memory management for SDDP execution
pub struct MemoryManager {
    /// Static read-only data (shared across threads)
    case_data: Arc<CaseData>,

    /// FCF cuts (read-mostly, periodic updates)
    fcf: RwLock<FutureCostFunction>,

    /// Per-thread LP solver workspaces
    solver_workspaces: Vec<ThreadLocal<SolverWorkspace>>,

    /// Scenario data (per-rank partition)
    scenario_buffer: ScenarioBuffer,

    /// Memory pool for temporary allocations
    temp_pool: MemoryPool,
}

/// Thread-local LP solver workspace (avoids allocation per solve)
pub struct SolverWorkspace {
    /// Reusable LP model
    model: LpModel,

    /// Solution vector (pre-allocated)
    solution: Vec<f64>,

    /// Dual vector (pre-allocated)
    duals: Vec<f64>,

    /// Basis information (for warm-starting)
    basis: LpBasis,

    /// Scratch buffers
    scratch: ScratchBuffers,
}

impl SolverWorkspace {
    /// Pre-allocate workspace for problem of given size
    pub fn new(n_vars: usize, n_constraints: usize) -> Self {
        Self {
            model: LpModel::with_capacity(n_vars, n_constraints),
            solution: vec![0.0; n_vars],
            duals: vec![0.0; n_constraints],
            basis: LpBasis::new(n_vars, n_constraints),
            scratch: ScratchBuffers::new(n_vars, n_constraints),
        }
    }

    /// Reset workspace for new problem (reuse allocations)
```

```
    pub fn reset(&mut self) {
        self.model.clear();
        self.solution.fill(0.0);
        self.duals.fill(0.0);
        self.basis.invalidate();
    }
}
```

24.3 NUMA-Aware Allocation

```
/// NUMA-aware memory allocation utilities
pub mod numa {
    use std::alloc::{alloc, dealloc, Layout};

    /// Allocate memory on specific NUMA node
    #[cfg(target_os = "linux")]
    pub fn alloc_on_node<T>(count: usize, node: i32) -> Box<[T]> {
        let layout = Layout::array::<T>(count).expect("Invalid layout");

        unsafe {
            // Set memory policy for this allocation
            libc::set_mempolicy(
                libc::MPOL_BIND,
                &(1u64 << node) as *const u64,
                64,
            );

            let ptr = alloc(layout) as *mut T;

            // Reset to default policy
            libc::set_mempolicy(libc::MPOL_DEFAULT, std::ptr::null(), 0);

            // First-touch initialization
            for i in 0..count {
                std::ptr::write(ptr.add(i), T::default());
            }

            Box::from_raw(std::slice::from_raw_parts_mut(ptr, count))
        }
    }

    /// Get NUMA node for current thread
    #[cfg(target_os = "linux")]
    pub fn current_node() -> i32 {
        unsafe {
            let cpu = libc::sched_getcpu();
            libc::numa_node_of_cpu(cpu)
        }
    }

    /// Allocate scenario buffer with NUMA-aware placement
    pub fn alloc_scenario_buffer(
        n_scenarios: usize,
        n_stages: usize,
        n_hydros: usize,
        threads_per_node: usize,
    ) -> Vec<f64> {
```

```
        let total_size = n_scenarios * n_stages * n_hydros;
        let mut buffer = vec![0.0; total_size];

        // Parallel first-touch to distribute across NUMA nodes
        buffer.par_chunks_mut(total_size / threads_per_node)
            .for_each(|chunk| {
                // Touch from thread on target NUMA node
                for x in chunk.iter_mut() {
                    *x = 0.0;
                }
            });

        buffer
    }
}
```

24.4 Memory Pool for Temporary Allocations

```
/// Fixed-size memory pool to avoid allocation during hot paths
pub struct MemoryPool {
    /// Pool of pre-allocated buffers
    pools: Vec<Mutex<Vec<Vec<f64>>>>,

    /// Buffer sizes for each pool
    sizes: Vec<usize>,
}

impl MemoryPool {
    /// Create pool with common buffer sizes
    pub fn new(buffers_per_size: usize) -> Self {
        let sizes = vec![64, 256, 1024, 4096, 16384, 65536];

        let pools = sizes.iter()
            .map(|&size| {
                let buffers: Vec<Vec<f64>> = (0..buffers_per_size)
                    .map(|_| vec![0.0; size])
                    .collect();
                Mutex::new(buffers)
            })
            .collect();

        Self { pools, sizes }
    }

    /// Get buffer of at least requested size
    pub fn get(&self, min_size: usize) -> PooledBuffer {
        // Find smallest pool that fits
        let pool_idx = self.sizes.iter()
            .position(|&s| s >= min_size)
            .unwrap_or(self.sizes.len() - 1);

        let buffer = {
            let mut pool = self.pools[pool_idx].lock().unwrap();
            pool.pop()
        };

        match buffer {
```

```
            Some(buf) => PooledBuffer {
                buffer: buf,
                pool_idx,
                pool: self,
            },
            None => {
                // Pool exhausted, allocate new
                PooledBuffer {
                    buffer: vec![0.0; self.sizes[pool_idx]],
                    pool_idx,
                    pool: self,
                }
            }
        }
    }
}

/// RAII guard that returns buffer to pool on drop
pub struct PooledBuffer<'a> {
    buffer: Vec<f64>,
    pool_idx: usize,
    pool: &'a MemoryPool,
}

impl Drop for PooledBuffer<'_> {
    fn drop(&mut self) {
        let buffer = std::mem::take(&mut self.buffer);
        let mut pool = self.pool.pools[self.pool_idx].lock().unwrap();
        pool.push(buffer);
    }
}

impl std::ops::Deref for PooledBuffer<'_> {
    type Target = [f64];
    fn deref(&self) -> &[f64] {
        &self.buffer
    }
}

impl std::ops::DerefMut for PooledBuffer<'_> {
    fn deref_mut(&mut self) -> &mut [f64] {
        &mut self.buffer
    }
}
```

---

25. Checkpointing and Fault Tolerance

25.1 Checkpoint Strategy

```
                    Checkpointing Strategy


  Goals:
  - Resume training after job preemption or failure
  - Support long-running jobs (hours to days)
```

```
- Minimize checkpoint overhead (< 5% of iteration time)
- Enable warm-start from previous runs

Checkpoint Contents:

   1. FCF cuts (primary data, largest component)
      - All cuts for all stages
      - Activity counters for cut selection

   2. Training state
      - Current iteration number
      - Convergence monitor history (bounds, gaps)
      - RNG state for reproducibility

   3. Configuration snapshot
      - Hash of config.json for compatibility check
      - Timestamp


Checkpoint Schedule:
- Every N iterations (configurable, default 10)
- On SIGTERM (graceful shutdown from scheduler)
- On convergence (final checkpoint)
```

25.2 Checkpoint Implementation

```rust
/// Checkpoint manager for training state persistence
pub struct CheckpointManager {
    checkpoint_dir: PathBuf,
    interval: usize,
    last_checkpoint: usize,

    /// Signal handler state
    shutdown_requested: Arc<AtomicBool>,
}

impl CheckpointManager {
    pub fn new(case_dir: &Path, interval: usize) -> Self {
        let checkpoint_dir = case_dir.join("checkpoints");
        std::fs::create_dir_all(&checkpoint_dir).ok();

        let shutdown_requested = Arc::new(AtomicBool::new(false));

        // Register signal handler for graceful shutdown
        let shutdown_flag = shutdown_requested.clone();
        ctrlc::set_handler(move || {
            shutdown_flag.store(true, Ordering::SeqCst);
        }).ok();

        Self {
            checkpoint_dir,
            interval,
            last_checkpoint: 0,
            shutdown_requested,
        }
```

```
}

/// Check if checkpoint is needed
pub fn should_checkpoint(&self, iteration: usize) -> bool {
    // Periodic checkpoint
    if iteration - self.last_checkpoint >= self.interval {
        return true;
    }

    // Shutdown signal received
    if self.shutdown_requested.load(Ordering::SeqCst) {
        return true;
    }

    false
}

/// Write checkpoint (rank 0 only)
pub fn write_checkpoint(
    &mut self,
    iteration: usize,
    fcf: &FutureCostFunction,
    monitor: &ConvergenceMonitor,
    comm: &Communicator,
) -> io::Result<()> {
    if comm.rank() != 0 {
        // Workers wait at barrier
        comm.barrier();
        return Ok(());
    }

    let checkpoint_path = self.checkpoint_dir.join(format!(
        "checkpoint_{:06}.bin",
        iteration
    ));

    let mut file = BufWriter::new(File::create(&checkpoint_path)?);

    // Header
    file.write_all(b"PWRSCHK\0")?;  // Magic
    file.write_all(&1u32.to_le_bytes())?;  // Version
    file.write_all(&(iteration as u64).to_le_bytes())?;
    file.write_all(&SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs()
        .to_le_bytes())?;

    // FCF cuts
    self.write_fcf(&mut file, fcf)?;

    // Convergence monitor
    self.write_monitor(&mut file, monitor)?;

    file.flush()?;

    // Update symlink to latest
```

```
    let latest_link = self.checkpoint_dir.join("latest");
    std::fs::remove_file(&latest_link).ok();
    std::os::unix::fs::symlink(&checkpoint_path, &latest_link)?;

    // Cleanup old checkpoints (keep last 3)
    self.cleanup_old_checkpoints(3)?;

    self.last_checkpoint = iteration;

    // Sync with workers
    comm.barrier();

    Ok(())
}

/// Load latest checkpoint if available
pub fn load_latest(&self) -> Option<Checkpoint> {
    let latest_link = self.checkpoint_dir.join("latest");

    if !latest_link.exists() {
        return None;
    }

    let checkpoint_path = std::fs::read_link(&latest_link).ok()?;
    self.load_checkpoint(&checkpoint_path).ok()
}

fn write_fcf(&self, file: &mut impl Write, fcf: &FutureCostFunction) -> io::Result<()> {
    // Number of stages
    let n_stages = fcf.cuts_by_stage.len();
    file.write_all(&(n_stages as u32).to_le_bytes())?;

    // Cuts per stage
    for pool in &fcf.cuts_by_stage {
        let cuts = pool.active_cuts();
        file.write_all(&(cuts.len() as u32).to_le_bytes())?;

        for cut in cuts {
            let bytes = cut.to_bytes();
            file.write_all(&(bytes.len() as u32).to_le_bytes())?;
            file.write_all(&bytes)?;
        }
    }

    Ok(())
}

fn cleanup_old_checkpoints(&self, keep: usize) -> io::Result<()> {
    let mut checkpoints: Vec<_> = std::fs::read_dir(&self.checkpoint_dir)?
        .filter_map(|e| e.ok())
        .filter(|e| e.path().extension().map(|s| s == "bin").unwrap_or(false))
        .collect();

    if checkpoints.len() <= keep {
        return Ok(());
    }
```

```
        // Sort by name (includes iteration number)
        checkpoints.sort_by_key(|e| e.path());

        // Remove oldest
        for entry in &checkpoints[..checkpoints.len() - keep] {
            std::fs::remove_file(entry.path())?;
        }

        Ok(())
    }
}

/// Loaded checkpoint data
pub struct Checkpoint {
    pub iteration: usize,
    pub timestamp: SystemTime,
    pub fcf: FutureCostFunction,
    pub monitor_history: ConvergenceHistory,
}
```

25.3 Warm-Start from Checkpoint

```
impl<R: RiskMeasure, C: CutFormulation, H: HorizonMode> TrainingLoop<R, C, H> {
    /// Initialize training with optional warm-start
    pub fn initialize(&mut self, case_data: &CaseData) -> io::Result<()> {
        let checkpoint_manager = CheckpointManager::new(
            &case_data.case_dir,
            self.config.checkpoint_interval,
        );

        // Try to load existing checkpoint
        if let Some(checkpoint) = checkpoint_manager.load_latest() {
            // Validate compatibility
            if checkpoint.fcf.state_dimension() != case_data.state_dimension() {
                return Err(io::Error::new(
                    io::ErrorKind::InvalidData,
                    "Checkpoint state dimension mismatch",
                ));
            }

            // Restore state
            self.fcf = checkpoint.fcf;
            self.iteration = checkpoint.iteration;
            self.convergence_monitor.restore_history(checkpoint.monitor_history);

            if self.comm.rank() == 0 {
                println!(
                    "Resumed from checkpoint at iteration {} ({:?})",
                    self.iteration,
                    checkpoint.timestamp,
                );
            }
        } else {
            // Cold start: initialize empty FCF
            self.fcf = FutureCostFunction::new(
                case_data.num_stages(),
                case_data.num_hydros(),
```

```
                case_data.par_models.max_order(),
            );
            self.iteration = 0;

            if self.comm.rank() == 0 {
                println!("Starting fresh training (no checkpoint found)");
            }
        }

        Ok(())
    }
}
```

25.5 Policy Persistence Architecture

A POWE.RS policy must be **fully self-contained** for warm-start, checkpointing, and simulation-only runs. This means persisting not just the cuts, but all information needed to reconstruct the exact optimization problem.

25.5.1 Policy Components

```
                        Policy Persistence Components


  A complete policy consists of:


    1. CUTS (Future Cost Function)

       Per stage t:
       -  _j: scalar intercept
       -  _j: gradient vector (dimension = state_dim)
       - metadata: iteration, scenario_id, active flag

       Size: 120 stages × 10K cuts × 16.3 KB/cut   20 GB



    2. PAR MODEL COEFFICIENTS (CRITICAL for consistency)

       Per hydro h, season m:
       -  [h,m]: seasonal mean
       -  [h,m, ]: AR coefficients for   = 1..P[h]
       -  [h,m]: residual standard deviation
       - P[h]: AR order

       Size: 160 hydros × 12 seasons × (1 + 12 + 1) × 8 bytes   270 KB

         AR ORDER MISMATCH IS A FATAL ERROR
       The state dimension includes AR lags. Different orders =
       different cut dimensions = incompatible policy!



    3. STATE VARIABLE MAPPING (for cut interpretation)
```

```
Canonical ordering of state variables:
- state[0..N_hydro-1]: reservoir storage volumes
- state[N_hydro + h*P + ]: lag   inflow for hydro h

Maps variable names to indices:
{
  "storage_ITAIPU": 0,
  "storage_TUCURUI": 1,
  ...
  "lag_ITAIPU_1": 160,
  "lag_ITAIPU_2": 161,
  ...
}
```

```
4. POLICY METADATA

   - version: policy format version
   - training_iterations: number of SDDP iterations
   - convergence_gap: final optimality gap
   - state_dimension: total state dimension (validation)
   - n_stages: number of stages
   - n_hydros: number of hydros
   - timestamp: when policy was generated
   - checksum: SHA256 of cut data (integrity verification)
```

25.5.2 File Format

```
policy/
  metadata.json           # Policy metadata + state mapping
  par_models.json         # Complete PAR model coefficients
  cuts/
      stage_000.bin       # Binary cut data for stage 0
      stage_001.bin       # Binary cut data for stage 1
      ...
```

metadata.json:

```
{
  "version": "1.0",
  "training_iterations": 150,
  "convergence_gap": 0.0023,
  "state_dimension": 2080,
  "n_stages": 120,
  "n_hydros": 160,
  "max_ar_order": 12,
  "timestamp": "2026-01-31T14:30:00Z",
  "checksum": "sha256:a1b2c3...",
  "state_mapping": {
    "storage_variables": ["ITAIPU", "TUCURUI", "XINGO", ...],
    "ar_orders": {"ITAIPU": 12, "TUCURUI": 12, "XINGO": 10, ...}
  }
}
```

par_models.json:

```
{
  "hydros": {
    "ITAIPU": {
      "order": 12,
      "means": [1234.5, 1456.7, ...],
      "coefficients": [
        [0.45, 0.23, 0.12, ...],
        [0.42, 0.25, 0.11, ...],
        ...
      ],
      "residual_std": [234.5, 267.8, ...]
    },
    ...
  }
}
```

25.5.3 Compatibility Validation

Loading a policy requires strict compatibility checking:

```rust
/// Policy compatibility validation
pub struct PolicyValidator {
    tolerance: f64,
}

impl PolicyValidator {
    /// Validate policy against current case data
    pub fn validate(
        &self,
        policy: &Policy,
        case: &CaseData,
    ) -> Result<(), PolicyError> {
        // 1. Check state dimension match
        let expected_dim = case.compute_state_dimension();
        if policy.state_dimension != expected_dim {
            return Err(PolicyError::DimensionMismatch {
                policy_dim: policy.state_dimension,
                case_dim: expected_dim,
                detail: "State dimension mismatch - likely AR order difference".into(),
            });
        }

        // 2. Check hydro count and IDs
        if policy.n_hydros != case.hydros.len() {
            return Err(PolicyError::HydroCountMismatch {
                policy: policy.n_hydros,
                case: case.hydros.len(),
            });
        }

        for (hydro_id, policy_order) in &policy.ar_orders {
            let case_order = case.par_models.order(hydro_id)
                .ok_or_else(|| PolicyError::UnknownHydro(hydro_id.clone()))?;

            if policy_order != &case_order {
                return Err(PolicyError::ArOrderMismatch {
                    hydro: hydro_id.clone(),
                    policy_order: *policy_order,
```

```
                    case_order,
                });
            }
        }

        // 3. Check PAR coefficients consistency (optional, with tolerance)
        for (hydro_id, policy_par) in &policy.par_models {
            if let Some(case_par) = case.par_models.get(hydro_id) {
                let max_diff = policy_par.max_coefficient_diff(case_par);
                if max_diff > self.tolerance {
                    return Err(PolicyError::ParCoefficientMismatch {
                        hydro: hydro_id.clone(),
                        max_diff,
                        tolerance: self.tolerance,
                    });
                }
            }
        }

        // 4. Check stage count
        if policy.n_stages != case.stages.len() {
            return Err(PolicyError::StageCountMismatch {
                policy: policy.n_stages,
                case: case.stages.len(),
            });
        }

        Ok(())
    }
}

#[derive(Debug, thiserror::Error)]
pub enum PolicyError {
    #[error("State dimension mismatch: policy={policy_dim}, case={case_dim}. {detail}")]
    DimensionMismatch {
        policy_dim: usize,
        case_dim: usize,
        detail: String,
    },

    #[error("AR order mismatch for hydro '{hydro}': policy={policy_order}, case={case_order}")]
    ArOrderMismatch {
        hydro: String,
        policy_order: usize,
        case_order: usize,
    },

    #[error("PAR coefficient mismatch for hydro '{hydro}': max_diff={max_diff:.6}, tolerance={tolerance:.6}
    ParCoefficientMismatch {
        hydro: String,
        max_diff: f64,
        tolerance: f64,
    },

    #[error("Hydro count mismatch: policy={policy}, case={case}")]
    HydroCountMismatch { policy: usize, case: usize },
```

```
    #[error("Stage count mismatch: policy={policy}, case={case}")]
    StageCountMismatch { policy: usize, case: usize },

    #[error("Unknown hydro in policy: '{0}'")]
    UnknownHydro(String),
}
```

25.5.4 Use Cases

Scenario	What's Loaded	Validation
Warm-start training	Cuts + PAR models	Full validation, continue training
Simulation-only	Cuts + PAR models	Full validation, skip training
Checkpoint recovery	Full state	Same case required
Policy transfer	Cuts only	Dimension check only (experimental)

CRITICAL WARNING: Using a policy with mismatched PAR coefficients will produce subtly incorrect results. The cuts were computed with specific AR dynamics embedded in the LP constraints. Different AR coefficients mean different RHS values for the same state, leading to: - Incorrect cost-to-go estimates - Suboptimal decisions - Potential infeasibility in extreme cases

Always validate PAR consistency when loading policies from different training runs.

---

26. Output Generation

26.1 Output Directory Structure

Output Directory Structure

```
case_directory/
  config.json                    (input)
  system/                        (input)
  scenarios/                     (input)

  output/                        (generated)
      metadata.json              Execution metadata

      policy/                    Trained policy (FCF)
          metadata.json          Policy metadata and state dictionary
          cuts/                  Cut files by stage
              stage_000.bin      Binary cut data
              stage_001.bin
              ...
          convergence.json       Convergence history

      simulation/                Simulation results
          summary.json           Aggregate statistics
          scenario_results.parquet  Detailed results (if enabled)
          distributions/         Per-variable distributions
              costs.parquet
              storage.parquet
              generation.parquet
```

```
logs/                       Execution logs
    training.log            Training progress
    simulation.log          Simulation progress
    performance.json        Performance metrics

checkpoints/                Checkpoint files
    latest -> checkpoint_000047.bin
    checkpoint_000040.bin
    checkpoint_000047.bin
```

26.2 Policy Output

```rust
/// Policy output writer
pub struct PolicyWriter {
    output_dir: PathBuf,
}

impl PolicyWriter {
    /// Write trained policy to disk
    pub fn write_policy(
        &self,
        fcf: &FutureCostFunction,
        case_data: &CaseData,
        training_result: &TrainingResult,
    ) -> io::Result<PathBuf> {
        let policy_dir = self.output_dir.join("policy");
        std::fs::create_dir_all(&policy_dir)?;

        // Write metadata
        let metadata = PolicyMetadata {
            version: "1.0".to_string(),
            created: Utc::now(),
            case_name: case_data.name.clone(),
            n_stages: case_data.num_stages(),
            n_hydros: case_data.num_hydros(),
            state_variables: fcf.state_dictionary(),
            training_iterations: training_result.iterations,
            final_gap: training_result.gap,
            lower_bound: training_result.lower_bound,
            upper_bound: training_result.upper_bound,
        };

        let metadata_path = policy_dir.join("metadata.json");
        let metadata_json = serde_json::to_string_pretty(&metadata)?;
        std::fs::write(&metadata_path, metadata_json)?;

        // Write cuts by stage
        let cuts_dir = policy_dir.join("cuts");
        std::fs::create_dir_all(&cuts_dir)?;

        for (stage_idx, pool) in fcf.cuts_by_stage.iter().enumerate() {
            let stage_path = cuts_dir.join(format!("stage_{:03}.bin", stage_idx));
            save_stage_cuts(&stage_path, pool.active_cuts())?;
        }
```

```rust
        // Write convergence history
        let convergence = ConvergenceReport {
            iterations: training_result.bound_history.len(),
            lower_bounds: training_result.bound_history.iter()
                .map(|b| b.lower)
                .collect(),
            upper_bounds: training_result.bound_history.iter()
                .map(|b| b.upper)
                .collect(),
            gaps: training_result.bound_history.iter()
                .map(|b| b.gap)
                .collect(),
            iteration_times_ms: training_result.iteration_times.iter()
                .map(|d| d.as_millis() as u64)
                .collect(),
        };

        let convergence_path = policy_dir.join("convergence.json");
        let convergence_json = serde_json::to_string_pretty(&convergence)?;
        std::fs::write(&convergence_path, convergence_json)?;

        Ok(policy_dir)
    }
}

/// Policy metadata for warm-start compatibility
#[derive(Serialize, Deserialize)]
pub struct PolicyMetadata {
    pub version: String,
    pub created: DateTime<Utc>,
    pub case_name: String,
    pub n_stages: usize,
    pub n_hydros: usize,
    pub state_variables: StateVariableDictionary,
    pub training_iterations: usize,
    pub final_gap: f64,
    pub lower_bound: f64,
    pub upper_bound: f64,
}

/// Maps state variable indices to entity IDs and types
#[derive(Serialize, Deserialize)]
pub struct StateVariableDictionary {
    pub storage_variables: Vec<StateVariableEntry>,
    pub inflow_variables: Vec<StateVariableEntry>,
}

#[derive(Serialize, Deserialize)]
pub struct StateVariableEntry {
    pub index: usize,
    pub entity_id: String,
    pub entity_type: String,
    pub description: String,
}
```

26.3 Simulation Summary Output

```rust
/// Simulation summary output
#[derive(Serialize)]
pub struct SimulationSummary {
    // Execution info
    pub timestamp: DateTime<Utc>,
    pub n_scenarios: usize,
    pub scenario_source: String,

    // Cost statistics
    pub cost: CostStatistics,

    // Operational statistics
    pub operations: OperationalStatistics,

    // Risk metrics
    pub risk: RiskMetrics,
}

#[derive(Serialize)]
pub struct CostStatistics {
    pub mean: f64,
    pub std: f64,
    pub min: f64,
    pub max: f64,
    pub median: f64,
    pub percentiles: HashMap<String, f64>,  // p5, p25, p75, p95
}

#[derive(Serialize)]
pub struct OperationalStatistics {
    pub deficit_probability: f64,
    pub deficit_mean_mwh: f64,
    pub deficit_max_mwh: f64,
    pub spill_mean_mwh: f64,
    pub thermal_generation_mean_mwh: f64,
    pub hydro_generation_mean_mwh: f64,
}

#[derive(Serialize)]
pub struct RiskMetrics {
    pub var_95: f64,   // Value at Risk (95%)
    pub cvar_95: f64,  // Conditional VaR (95%)
    pub var_99: f64,
    pub cvar_99: f64,
}

impl SimulationSummary {
    pub fn from_stats(stats: &SimulationStats, config: &SimulationConfig) -> Self {
        Self {
            timestamp: Utc::now(),
            n_scenarios: stats.n_scenarios,
            scenario_source: format!("{:?}", config.scenario_source),

            cost: CostStatistics {
                mean: stats.mean_cost(),
```

```
                std: stats.std_cost(),
                min: stats.min_cost,
                max: stats.max_cost,
                median: stats.percentile(0.5),
                percentiles: [
                    ("p5".to_string(), stats.percentile(0.05)),
                    ("p25".to_string(), stats.percentile(0.25)),
                    ("p75".to_string(), stats.percentile(0.75)),
                    ("p95".to_string(), stats.percentile(0.95)),
                ].into_iter().collect(),
            },

            operations: OperationalStatistics {
                deficit_probability: stats.deficit_scenarios as f64 / stats.n_scenarios as f64,
                deficit_mean_mwh: stats.deficit_mwh_sum / stats.n_scenarios as f64,
                deficit_max_mwh: stats.max_deficit,
                spill_mean_mwh: stats.spill_mwh_sum / stats.n_scenarios as f64,
                thermal_generation_mean_mwh: stats.thermal_gen_sum / stats.n_scenarios as f64,
                hydro_generation_mean_mwh: stats.hydro_gen_sum / stats.n_scenarios as f64,
            },

            risk: RiskMetrics {
                var_95: stats.percentile(0.95),
                cvar_95: stats.cvar(0.95),
                var_99: stats.percentile(0.99),
                cvar_99: stats.cvar(0.99),
            },
        }
    }

    pub fn write(&self, output_dir: &Path) -> io::Result<()> {
        let path = output_dir.join("simulation").join("summary.json");
        std::fs::create_dir_all(path.parent().unwrap())?;
        let json = serde_json::to_string_pretty(self)?;
        std::fs::write(path, json)
    }
}
```

26.4 Performance Logging

```
/// Performance metrics collector
pub struct PerformanceLogger {
    metrics: Mutex<PerformanceMetrics>,
    start_time: Instant,
}

#[derive(Serialize, Default)]
pub struct PerformanceMetrics {
    pub total_runtime_seconds: f64,

    // Phase timings
    pub initialization_seconds: f64,
    pub training_seconds: f64,
    pub simulation_seconds: f64,

    // Training metrics
    pub training_iterations: usize,
```

```rust
    pub forward_passes: usize,
    pub backward_passes: usize,
    pub lp_solves: usize,
    pub cuts_generated: usize,

    // Parallel efficiency
    pub mpi_ranks: usize,
    pub threads_per_rank: usize,
    pub total_cores: usize,
    pub communication_overhead_percent: f64,

    // Memory
    pub peak_memory_gb: f64,
    pub fcf_memory_mb: f64,

    // I/O
    pub checkpoint_writes: usize,
    pub checkpoint_bytes_written: u64,
    pub output_bytes_written: u64,
}

impl PerformanceLogger {
    pub fn new() -> Self {
        Self {
            metrics: Mutex::new(PerformanceMetrics::default()),
            start_time: Instant::now(),
        }
    }

    pub fn record_iteration(&self, iteration: &IterationMetrics) {
        let mut m = self.metrics.lock().unwrap();
        m.training_iterations += 1;
        m.forward_passes += 1;
        m.backward_passes += 1;
        m.lp_solves += iteration.lp_solves;
        m.cuts_generated += iteration.cuts_generated;
    }

    pub fn write(&self, output_dir: &Path) -> io::Result<()> {
        let mut metrics = self.metrics.lock().unwrap();
        metrics.total_runtime_seconds = self.start_time.elapsed().as_secs_f64();

        let path = output_dir.join("logs").join("performance.json");
        std::fs::create_dir_all(path.parent().unwrap())?;
        let json = serde_json::to_string_pretty(&*metrics)?;
        std::fs::write(path, json)
    }
}
```

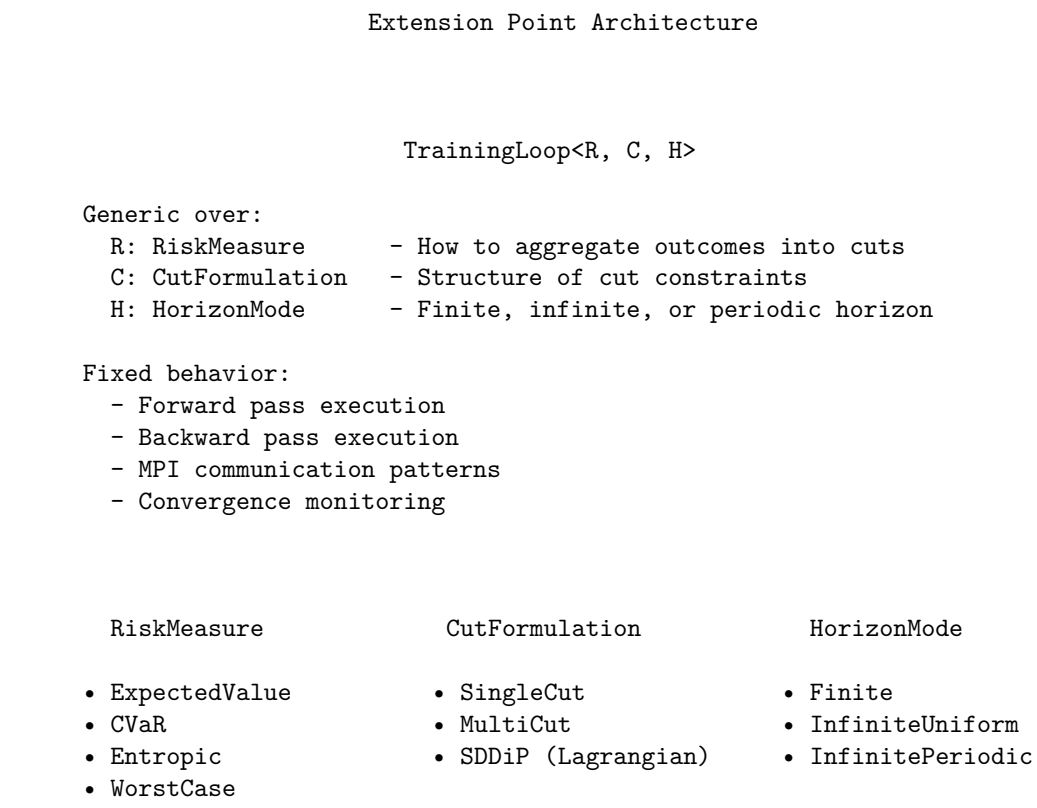
Part VIII: Extension Points

---

27. Trait Abstractions for Algorithm Variants

27.1 Extensibility Architecture

POWE.RS uses trait-based polymorphism to support algorithm variants without code duplication:



27.2 Core Trait Definitions

```rust
/// Risk measure determines how to combine noise outcomes into a cut
///
/// The risk measure maps the distribution of future costs to a scalar value
/// that represents the "risk-adjusted" expected cost.
pub trait RiskMeasure: Send + Sync + Clone + 'static {
    /// Compute cut coefficients from backward pass evaluations
    ///
    /// # Arguments
    /// * `stage` - Current stage ID
    /// * `state` - State point where cut is being generated
    /// * `outcomes` - LP solutions for each noise outcome
    /// * `probabilities` - Probability of each noise outcome
    ///
    /// # Returns
    /// Cut coefficients (intercept and gradients)
    fn compute_cut(
        &self,
        stage: StageId,
        state: &StatePoint,
        outcomes: &[BackwardOutcome],
        probabilities: &[f64],
```

```
    ) -> CutCoefficients;

    /// Risk-adjusted objective for convergence bound computation
    fn evaluate_risk(&self, values: &[f64], probabilities: &[f64]) -> f64;

    /// Display name for logging
    fn name(&self) -> &'static str;

    /// Parameters for serialization/deserialization
    fn parameters(&self) -> RiskMeasureParameters;
}

/// Cut formulation determines the structure of cuts in the LP
pub trait CutFormulation: Send + Sync + Clone + 'static {
    /// Number of cuts generated per backward state evaluation
    fn cuts_per_state(&self) -> usize;

    /// Build cut constraint(s) for addition to stage LP
    fn build_constraints(
        &self,
        cut: &CutCoefficients,
        theta_var: VarId,
        state_vars: &StateVariables,
    ) -> Vec<Constraint>;

    /// Whether this formulation requires strengthening
    fn requires_strengthening(&self) -> bool {
        false
    }

    /// Display name
    fn name(&self) -> &'static str;
}

/// Horizon mode determines stage transitions and terminal conditions
pub trait HorizonMode: Send + Sync + Clone + 'static {
    /// Get successor stages with transition probabilities
    fn successors(&self, stage: StageId, stages: &[Stage]) -> Vec<(StageId, f64)>;

    /// Check if stage is terminal (no successors)
    fn is_terminal(&self, stage: StageId, stages: &[Stage]) -> bool;

    /// Discount factor for future costs (1.0 for undiscounted)
    fn discount_factor(&self, from_stage: StageId, to_stage: StageId) -> f64;

    /// Validate stage configuration
    fn validate(&self, stages: &[Stage]) -> Result<(), ValidationError>;

    /// Display name
    fn name(&self) -> &'static str;
}
```

27.3 Factory Pattern for Configuration

```
/// Factory for creating algorithm components from configuration
pub struct AlgorithmFactory;
```

```rust
impl AlgorithmFactory {
    /// Create risk measure from configuration
    pub fn create_risk_measure(config: &RiskConfig) -> Box<dyn RiskMeasure> {
        match config {
            RiskConfig::ExpectedValue => {
                Box::new(ExpectedValueRisk)
            }
            RiskConfig::CVaR { alpha } => {
                Box::new(CVaRRisk::new(*alpha))
            }
            RiskConfig::Entropic { gamma } => {
                Box::new(EntropicRisk::new(*gamma))
            }
            RiskConfig::ConvexCombination { lambda, inner } => {
                let inner_risk = Self::create_risk_measure(inner);
                Box::new(ConvexCombinationRisk::new(*lambda, inner_risk))
            }
        }
    }

    /// Create horizon mode from configuration
    pub fn create_horizon_mode(config: &HorizonConfig) -> Box<dyn HorizonMode> {
        match config {
            HorizonConfig::Finite => {
                Box::new(FiniteHorizon)
            }
            HorizonConfig::InfiniteUniform { discount_rate } => {
                Box::new(InfiniteUniformHorizon::new(*discount_rate))
            }
            HorizonConfig::InfinitePeriodic { cycle_start, cycle_length, discount_rate } => {
                Box::new(InfinitePeriodicHorizon::new(
                    *cycle_start,
                    *cycle_length,
                    *discount_rate,
                ))
            }
        }
    }

    /// Create complete training loop with configured components
    pub fn create_training_loop(
        config: &Config,
        comm: WorldCommunicator,
    ) -> Box<dyn TrainingLoopDyn> {
        let risk = Self::create_risk_measure(&config.risk);
        let horizon = Self::create_horizon_mode(&config.horizon);
        let cut_formulation = SingleCutFormulation; // Default

        // Use dynamic dispatch for runtime flexibility
        Box::new(TrainingLoopImpl::new(risk, cut_formulation, horizon, config.training.clone(), comm))
    }
}
```

---

28. Risk Measure Implementations

28.1 Expected Value (Risk-Neutral)

```rust
/// Risk-neutral expected value
///
/// The standard SDDP risk measure: E[Q(x, )]
#[derive(Clone)]
pub struct ExpectedValueRisk;

impl RiskMeasure for ExpectedValueRisk {
    fn compute_cut(
        &self,
        _stage: StageId,
        _state: &StatePoint,
        outcomes: &[BackwardOutcome],
        probabilities: &[f64],
    ) -> CutCoefficients {
        // Cut intercept: E[Q] - E[ ]^T x̄
        let expected_q: f64 = outcomes.iter()
            .zip(probabilities.iter())
            .map(|(o, p)| p * o.objective)
            .sum();

        // Cut gradient: E[ ]
        let n_storage = outcomes[0].dual_storage.len();
        let n_inflow = outcomes[0].dual_inflow.len();

        let mut storage_coef = vec![0.0; n_storage];
        let mut inflow_coef = vec![0.0; n_inflow];

        for (outcome, &prob) in outcomes.iter().zip(probabilities.iter()) {
            for (i, &dual) in outcome.dual_storage.iter().enumerate() {
                storage_coef[i] += prob * dual;
            }
            for (i, &dual) in outcome.dual_inflow.iter().enumerate() {
                inflow_coef[i] += prob * dual;
            }
        }

        CutCoefficients {
            intercept: expected_q,
            storage_coef,
            inflow_coef,
        }
    }

    fn evaluate_risk(&self, values: &[f64], probabilities: &[f64]) -> f64 {
        values.iter()
            .zip(probabilities.iter())
            .map(|(v, p)| p * v)
            .sum()
    }

    fn name(&self) -> &'static str {
        "ExpectedValue"
    }
```

```
    fn parameters(&self) -> RiskMeasureParameters {
        RiskMeasureParameters::ExpectedValue
    }
}
```

28.2 Conditional Value-at-Risk (CVaR)

```
/// Conditional Value-at-Risk (CVaR) risk measure
///
/// CVaR_  = E[Q | Q   VaR_ ]
///
/// Focuses on the worst (1- ) fraction of outcomes.
///   = 0.95 means we average over the worst 5% of scenarios.
#[derive(Clone)]
pub struct CVaRRisk {
    /// Confidence level (e.g., 0.95 for 95% CVaR)
    alpha: f64,
}

impl CVaRRisk {
    pub fn new(alpha: f64) -> Self {
        assert!(alpha > 0.0 && alpha < 1.0, "  must be in (0, 1)");
        Self { alpha }
    }
}

impl RiskMeasure for CVaRRisk {
    fn compute_cut(
        &self,
        _stage: StageId,
        state: &StatePoint,
        outcomes: &[BackwardOutcome],
        probabilities: &[f64],
    ) -> CutCoefficients {
        // Sort outcomes by objective value (descending for worst-case)
        let mut indexed: Vec<_> = outcomes.iter()
            .zip(probabilities.iter())
            .enumerate()
            .collect();
        indexed.sort_by(|a, b| {
            b.1.0.objective.partial_cmp(&a.1.0.objective).unwrap()
        });

        // Find VaR threshold and compute CVaR weights
        let tail_prob = 1.0 - self.alpha;
        let mut cumulative = 0.0;
        let mut cvar_weights = vec![0.0; outcomes.len()];

        for (idx, (outcome, &prob)) in indexed.iter() {
            let contribution = if cumulative + prob <= tail_prob {
                prob
            } else {
                (tail_prob - cumulative).max(0.0)
            };
            cvar_weights[*idx] = contribution / tail_prob;
            cumulative += prob;
```

```
        if cumulative >= tail_prob {
            break;
        }
    }

    // Compute CVaR cut coefficients using adjusted weights
    let cvar_q: f64 = outcomes.iter()
        .zip(cvar_weights.iter())
        .map(|(o, w)| w * o.objective)
        .sum();

    let n_storage = outcomes[0].dual_storage.len();
    let n_inflow = outcomes[0].dual_inflow.len();

    let mut storage_coef = vec![0.0; n_storage];
    let mut inflow_coef = vec![0.0; n_inflow];

    for (outcome, &weight) in outcomes.iter().zip(cvar_weights.iter()) {
        for (i, &dual) in outcome.dual_storage.iter().enumerate() {
            storage_coef[i] += weight * dual;
        }
        for (i, &dual) in outcome.dual_inflow.iter().enumerate() {
            inflow_coef[i] += weight * dual;
        }
    }

    CutCoefficients {
        intercept: cvar_q,
        storage_coef,
        inflow_coef,
    }
}

fn evaluate_risk(&self, values: &[f64], probabilities: &[f64]) -> f64 {
    // Sort values descending
    let mut indexed: Vec<_> = values.iter()
        .zip(probabilities.iter())
        .collect();
    indexed.sort_by(|a, b| b.0.partial_cmp(a.0).unwrap());

    // Average over tail
    let tail_prob = 1.0 - self.alpha;
    let mut cumulative = 0.0;
    let mut cvar = 0.0;

    for (&value, &prob) in indexed.iter() {
        let contribution = if cumulative + prob <= tail_prob {
            prob
        } else {
            (tail_prob - cumulative).max(0.0)
        };
        cvar += contribution * value;
        cumulative += prob;

        if cumulative >= tail_prob {
            break;
        }
```

```
        }

        cvar / tail_prob
    }

    fn name(&self) -> &'static str {
        "CVaR"
    }

    fn parameters(&self) -> RiskMeasureParameters {
        RiskMeasureParameters::CVaR { alpha: self.alpha }
    }
}
```

28.3 Convex Combination Risk

```
/// Convex combination of expected value and another risk measure
///
///  (Q) =  ·E[Q] + (1- )· _inner(Q)
///
/// Common choice:  =0.5 with CVaR_0.95 gives balanced risk-aversion
#[derive(Clone)]
pub struct ConvexCombinationRisk {
    /// Weight on expected value
    lambda: f64,

    /// Inner risk measure (typically CVaR)
    inner: Box<dyn RiskMeasure>,
}

impl ConvexCombinationRisk {
    pub fn new(lambda: f64, inner: Box<dyn RiskMeasure>) -> Self {
        assert!(lambda >= 0.0 && lambda <= 1.0, "  must be in [0, 1]");
        Self { lambda, inner }
    }
}

impl RiskMeasure for ConvexCombinationRisk {
    fn compute_cut(
        &self,
        stage: StageId,
        state: &StatePoint,
        outcomes: &[BackwardOutcome],
        probabilities: &[f64],
    ) -> CutCoefficients {
        let ev_cut = ExpectedValueRisk.compute_cut(stage, state, outcomes, probabilities);
        let inner_cut = self.inner.compute_cut(stage, state, outcomes, probabilities);

        // Linear combination of cuts
        CutCoefficients {
            intercept: self.lambda * ev_cut.intercept + (1.0 - self.lambda) * inner_cut.intercept,
            storage_coef: ev_cut.storage_coef.iter()
                .zip(inner_cut.storage_coef.iter())
                .map(|(e, i)| self.lambda * e + (1.0 - self.lambda) * i)
                .collect(),
            inflow_coef: ev_cut.inflow_coef.iter()
                .zip(inner_cut.inflow_coef.iter())
```

```
                .map(|(e, i)| self.lambda * e + (1.0 - self.lambda) * i)
                .collect(),
        }
    }

    fn evaluate_risk(&self, values: &[f64], probabilities: &[f64]) -> f64 {
        let ev = ExpectedValueRisk.evaluate_risk(values, probabilities);
        let inner_risk = self.inner.evaluate_risk(values, probabilities);
        self.lambda * ev + (1.0 - self.lambda) * inner_risk
    }

    fn name(&self) -> &'static str {
        "ConvexCombination"
    }

    fn parameters(&self) -> RiskMeasureParameters {
        RiskMeasureParameters::ConvexCombination {
            lambda: self.lambda,
            inner: Box::new(self.inner.parameters()),
        }
    }
}
```

---

29. Horizon Mode Implementations

29.1 Finite Horizon

```
/// Standard finite horizon SDDP
///
/// Stages 1, 2, ..., T with no cycles.
/// Stage T has terminal value function (zero or specified).
#[derive(Clone)]
pub struct FiniteHorizon;

impl HorizonMode for FiniteHorizon {
    fn successors(&self, stage: StageId, stages: &[Stage]) -> Vec<(StageId, f64)> {
        let stage_idx = stage.0;

        if stage_idx + 1 < stages.len() {
            // Deterministic transition to next stage
            vec![(StageId(stage_idx + 1), 1.0)]
        } else {
            // Terminal stage
            vec![]
        }
    }

    fn is_terminal(&self, stage: StageId, stages: &[Stage]) -> bool {
        stage.0 == stages.len() - 1
    }

    fn discount_factor(&self, _from: StageId, _to: StageId) -> f64 {
        1.0  // Undiscounted
    }

    fn validate(&self, stages: &[Stage]) -> Result<(), ValidationError> {
```

```
        if stages.is_empty() {
            return Err(ValidationError::new("Must have at least one stage"));
        }
        Ok(())
    }

    fn name(&self) -> &'static str {
        "Finite"
    }
}
```

29.2 Infinite Horizon with Uniform Discounting

```
/// Infinite horizon with uniform discount factor
///
/// All stages use the same discount factor    (0, 1).
/// Future cost at stage t:   · _{t+1}
///
/// Convergence requires   < 1 for bounded costs.
#[derive(Clone)]
pub struct InfiniteUniformHorizon {
    /// Discount factor per stage
    discount_rate: f64,
}

impl InfiniteUniformHorizon {
    pub fn new(discount_rate: f64) -> Self {
        assert!(discount_rate > 0.0 && discount_rate < 1.0,
            "Discount rate must be in (0, 1)");
        Self { discount_rate }
    }
}

impl HorizonMode for InfiniteUniformHorizon {
    fn successors(&self, stage: StageId, stages: &[Stage]) -> Vec<(StageId, f64)> {
        let next_idx = (stage.0 + 1) % stages.len();  // Wrap around
        vec![(StageId(next_idx), 1.0)]
    }

    fn is_terminal(&self, _stage: StageId, _stages: &[Stage]) -> bool {
        false  // Never terminal in infinite horizon
    }

    fn discount_factor(&self, _from: StageId, _to: StageId) -> f64 {
        self.discount_rate
    }

    fn validate(&self, stages: &[Stage]) -> Result<(), ValidationError> {
        if stages.is_empty() {
            return Err(ValidationError::new("Must have at least one stage"));
        }
        Ok(())
    }

    fn name(&self) -> &'static str {
        "InfiniteUniform"
    }
```

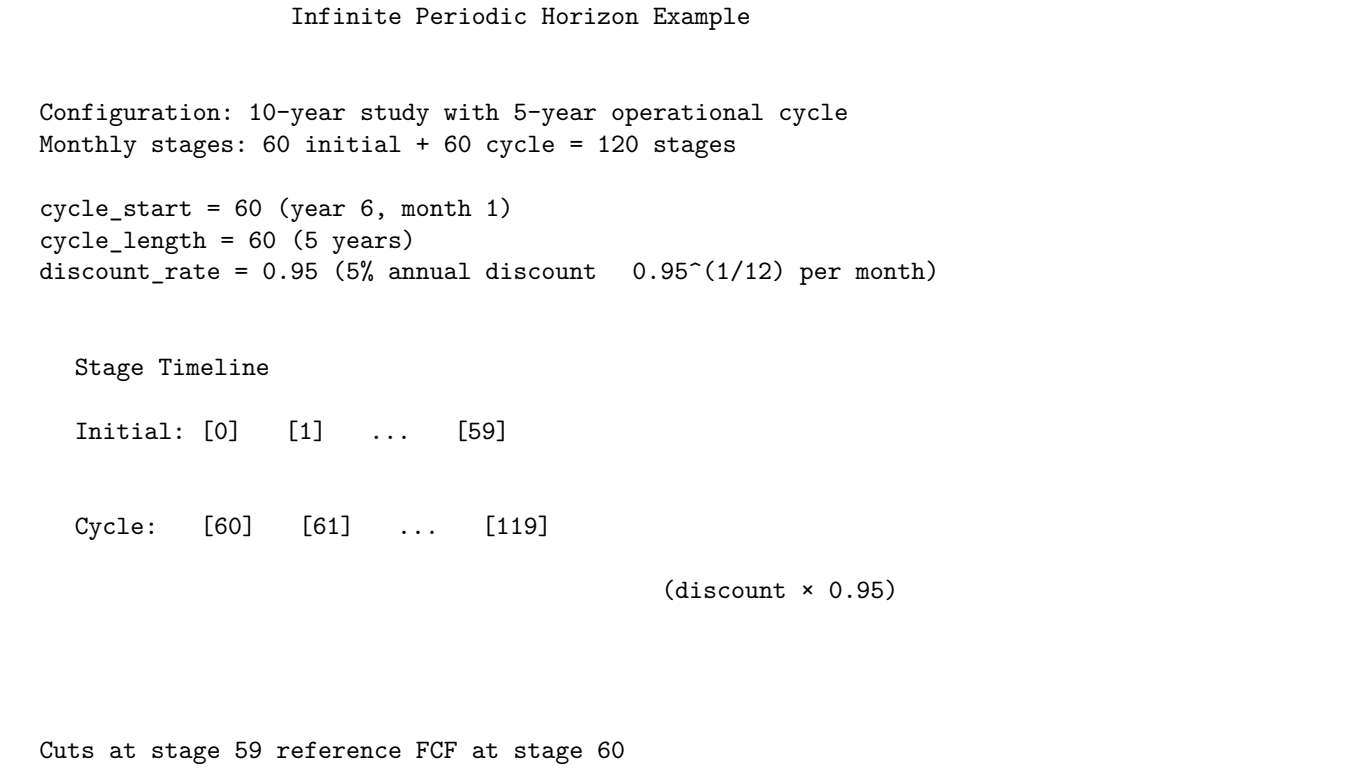
```
}
```

29.3 Infinite Horizon with Periodic Structure

```rust
/// Infinite horizon with periodic cycling
///
/// Structure:
///   [Initial stages: 0..cycle_start]
///   [Cycle: cycle_start..cycle_start+cycle_length] (repeats forever)
///
/// After stage cycle_start + cycle_length - 1, transitions back to cycle_start.
/// Useful for systems with seasonal patterns extending to infinity.
#[derive(Clone)]
pub struct InfinitePeriodicHorizon {
    /// First stage of the cycle
    cycle_start: usize,

    /// Length of the repeating cycle
    cycle_length: usize,

    /// Discount factor applied at cycle boundary
    discount_rate: f64,
}

impl InfinitePeriodicHorizon {
    pub fn new(cycle_start: usize, cycle_length: usize, discount_rate: f64) -> Self {
        assert!(cycle_length > 0, "Cycle length must be positive");
        assert!(discount_rate > 0.0 && discount_rate < 1.0,
            "Discount rate must be in (0, 1)");
        Self { cycle_start, cycle_length, discount_rate }
    }
}

impl HorizonMode for InfinitePeriodicHorizon {
    fn successors(&self, stage: StageId, stages: &[Stage]) -> Vec<(StageId, f64)> {
        let stage_idx = stage.0;
        let cycle_end = self.cycle_start + self.cycle_length;

        if stage_idx + 1 < cycle_end && stage_idx + 1 < stages.len() {
            // Normal progression
            vec![(StageId(stage_idx + 1), 1.0)]
        } else if stage_idx + 1 == cycle_end || stage_idx + 1 == stages.len() {
            // End of cycle: wrap back to cycle_start
            vec![(StageId(self.cycle_start), 1.0)]
        } else {
            vec![]
        }
    }

    fn is_terminal(&self, _stage: StageId, _stages: &[Stage]) -> bool {
        false  // Never terminal
    }

    fn discount_factor(&self, from: StageId, to: StageId) -> f64 {
        if to.0 < from.0 {
            // Crossing cycle boundary
            self.discount_rate
```

```
        } else {
            1.0  // Within cycle: no discounting
        }
    }

    fn validate(&self, stages: &[Stage]) -> Result<(), ValidationError> {
        let total_stages = stages.len();

        if self.cycle_start >= total_stages {
            return Err(ValidationError::new(format!(
                "cycle_start ({}) must be < n_stages ({})",
                self.cycle_start, total_stages
            )));
        }

        if self.cycle_start + self.cycle_length > total_stages {
            return Err(ValidationError::new(format!(
                "cycle extends beyond available stages"
            )));
        }

        Ok(())
    }

    fn name(&self) -> &'static str {
        "InfinitePeriodic"
    }
}
```

29.4 Stage Configuration for Periodic Horizon



Appendices

Appendix A: SLURM Job Script Patterns

A.1 Single-Node Job (Development/Testing)

```
#!/bin/bash
#SBATCH --job-name=powers-test
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --cpus-per-task=24
#SBATCH --time=01:00:00
#SBATCH --partition=debug
#SBATCH --output=powers_%j.log

# Load modules
module load openmpi/4.1.5
module load rust/1.75

# Set OpenMP threads from SLURM allocation
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PROC_BIND=close
export OMP_PLACES=cores

# Run POWE.RS
srun powers /path/to/case_directory
```

A.2 Multi-Node Production Job

```
#!/bin/bash
#SBATCH --job-name=powers-production
#SBATCH --nodes=8
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=24
#SBATCH --time=24:00:00
#SBATCH --partition=compute
#SBATCH --output=powers_%j.log
#SBATCH --error=powers_%j.err

# Load modules
module load openmpi/4.1.5
module load rust/1.75

# NUMA and memory settings
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
export OMP_PROC_BIND=close
export OMP_PLACES=cores
export MALLOC_MMAP_THRESHOLD_=1000000

# MPI settings for large jobs
export OMPI_MCA_btl_tcp_endpoint_cache=0
```

```
export OMPI_MCA_mpi_yield_when_idle=1

# Run with checkpoint signal handling
srun --signal=TERM@60 powers /scratch/user/case_directory

# On preemption, SIGTERM is sent 60s before kill
```

A.3 Job Array for Parameter Studies

```
#!/bin/bash
#SBATCH --job-name=powers-sweep
#SBATCH --array=0-9
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --cpus-per-task=24
#SBATCH --time=04:00:00
#SBATCH --output=powers_%A_%a.log

# Parameter values indexed by array task
SCENARIOS=(100 200 500 1000 2000 5000 10000 20000 50000 100000)
N_SCENARIOS=${SCENARIOS[$SLURM_ARRAY_TASK_ID]}

# Create case directory with modified config
CASE_DIR=/scratch/user/sweep_${N_SCENARIOS}
cp -r /home/user/base_case $CASE_DIR

# Modify config.json
jq ".training.forward_scenarios = ${N_SCENARIOS}" \
    $CASE_DIR/config.json > $CASE_DIR/config_new.json
mv $CASE_DIR/config_new.json $CASE_DIR/config.json

# Run
export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}
srun powers $CASE_DIR
```

Appendix B: Performance Monitoring Points

B.1 Key Performance Counters

```
/// Performance counters collected during execution
pub struct PerformanceCounters {
    // LP solver metrics
    pub lp_solves: AtomicU64,
    pub lp_iterations_total: AtomicU64,
    pub lp_time_ns: AtomicU64,

    // Communication metrics
    pub mpi_sends: AtomicU64,
    pub mpi_bytes_sent: AtomicU64,
    pub mpi_time_ns: AtomicU64,

    // Memory metrics
    pub allocations: AtomicU64,
    pub bytes_allocated: AtomicU64,
    pub peak_memory_bytes: AtomicU64,
```

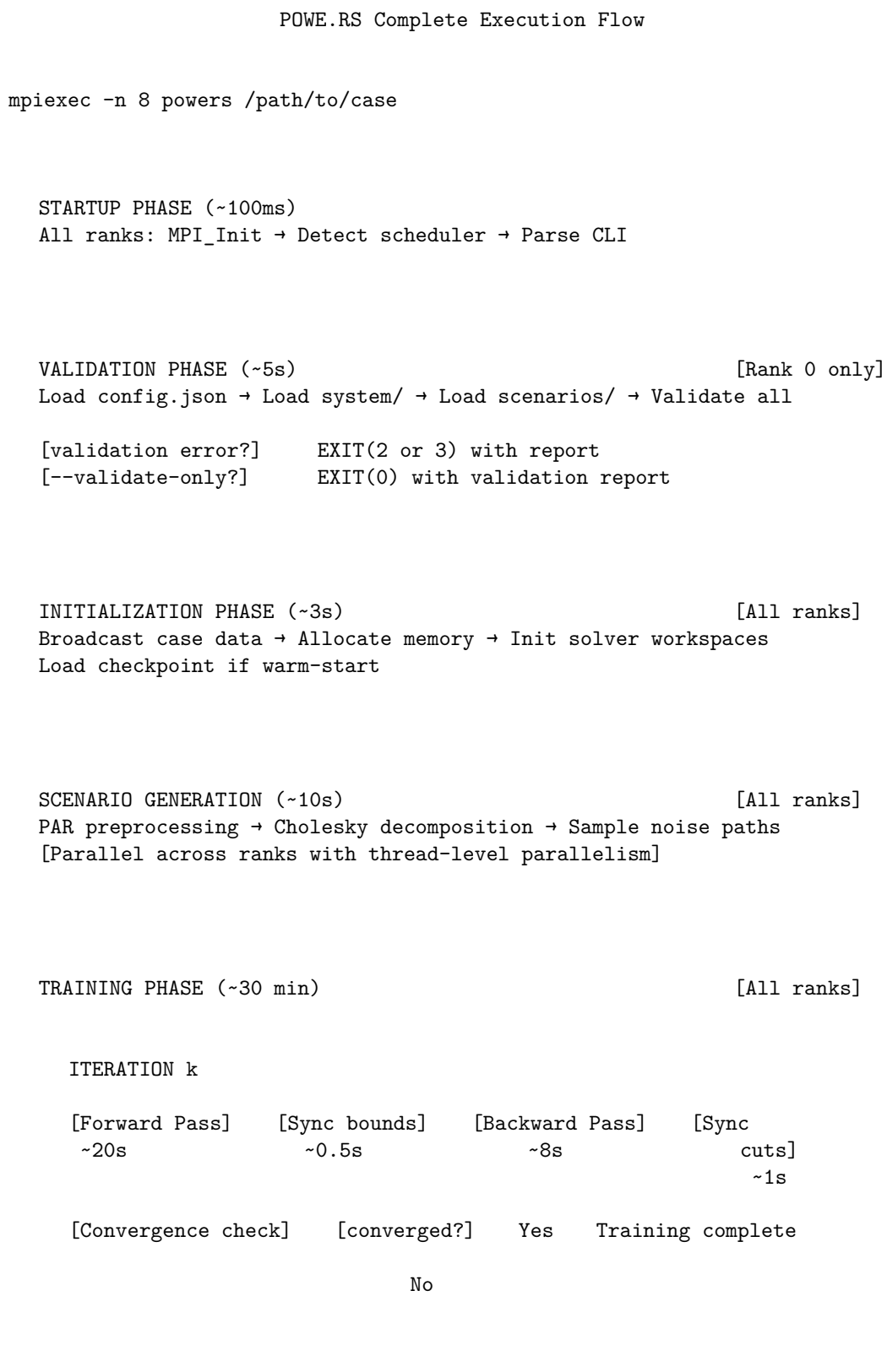
```rust
    // Cache metrics (requires perf counters)
    pub l1_cache_misses: AtomicU64,
    pub llc_cache_misses: AtomicU64,
}

impl PerformanceCounters {
    pub fn summary(&self) -> PerformanceSummary {
        let lp_solves = self.lp_solves.load(Ordering::Relaxed);
        let lp_time_s = self.lp_time_ns.load(Ordering::Relaxed) as f64 / 1e9;

        PerformanceSummary {
            total_lp_solves: lp_solves,
            avg_lp_time_ms: lp_time_s * 1000.0 / lp_solves as f64,
            avg_lp_iterations: self.lp_iterations_total.load(Ordering::Relaxed) as f64
                / lp_solves as f64,
            mpi_overhead_percent: self.mpi_time_ns.load(Ordering::Relaxed) as f64
                / (lp_time_s * 1e9) * 100.0,
            peak_memory_gb: self.peak_memory_bytes.load(Ordering::Relaxed) as f64 / 1e9,
        }
    }
}
```

B.2 Timing Breakdown

```
                    Iteration Timing Breakdown (Target)


  SDDP Iteration (~30 seconds total)


  Forward Pass:                          20.0s (66.7%)
    LP solves:                          18.5s (92.5%)
    State transitions:                   0.8s (4.0%)
    Noise sampling:                      0.2s (1.0%)
    Result collection:                   0.5s (2.5%)

  Forward Sync (Allreduce):               0.5s (1.7%)

  Backward Pass:                          8.0s (26.7%)
    LP solves:                           7.2s (90.0%)
    Cut computation:                     0.5s (6.3%)
    Local aggregation:                   0.3s (3.7%)

  Backward Sync (Allgatherv):             1.0s (3.3%)
    Serialization:                      0.3s
    MPI communication:                  0.5s
    Deserialization:                    0.2s

  Convergence check + logging:            0.5s (1.7%)


  Efficiency metrics:
  - Compute/Communication ratio: ~18:1 (excellent)
  - Parallel efficiency (8 ranks): ~92%
  - Thread utilization: ~85%
```

Appendix C: Execution Flow Diagrams

C.1 Complete Execution Flow



```
  [Checkpoint?]    Yes    Write checkpoint

                   Next iteration
```

```
SIMULATION PHASE (~15 min)                                    [All ranks]

For each scenario batch:
  [Simulate scenarios]    [Stream results]    [Update statistics]

[Aggregate statistics across ranks]
```

```
FINALIZE PHASE (~5s)                                          [All ranks]
Write policy → Write simulation summary → Write performance log
MPI_Finalize → EXIT(0)
```

C.2 Data Flow Diagram

```
                        Data Flow Through POWE.RS


INPUT FILES


config.json
stages.json
system/buses.json
system/hydros.json          InputLoader    CaseData
system/thermals.json
scenarios/inflow_models.parq
scenarios/correlation.json
constraints/*.parquet           ValidationResult


TRAINING
                                                Rank 0
                                                     loads


                                        MPI_Bcast

                                   All ranks have
                                   identical CaseData
```

```
 Rank 0          Rank 1      ...      Rank 7
scenarios       scenarios            scenarios
 0-24            25-49               175-199
```

```
                        FutureCostFunction
                        (FCF - cuts)
```

```
OUTPUT FILES


output/policy/metadata.json
output/policy/cuts/stage_*.bin
output/policy/convergence.json

SIMULATION


FCF + Monte Carlo scenarios         SimulationRunner


                                          SimulationStats


output/simulation/summary.json
output/simulation/scenarios_*.parquet
output/logs/performance.json
```

End of Document